Supplementary Material

Counting Collapse: A Formally Verified Attractor Theory of Repetition Hallucination in Autoregressive Text-to-Speech

This document accompanies the 4-page ICASSP 2026 submission and is self-contained. It reproduces the formal artifact in full, states precisely what it does and does not certify, documents the benchmark and measurement protocol down to the level a re-implementer needs, reports the negative negative result of the main paper's Section 4 in the detail it deserves, answers, with new measurements rather than argument, five alternative explanations for the main results raised across two rounds of review (Section 8), and documents, in Section 9, the audit that found the paper's original behavioural judge (Whisper large-v3) to be biased against exactly the phenomenon under study, the validation of its replacement, and the metric change that followed – the project's single most consequential methodological correction, which post-dates Sections 1–8 and supersedes every Whisper-judged, exact-match number reported in them. Those numbers are not deleted; they are flagged inline, at the point each recurs, with a pointer to Section 9, because the contrast between what the old instrument said and what the new one says is itself part of the evidence. Every claim below is traceable to a specific file in the project repository, cited inline by path.

Contents

1 The formal development in full

The formal artifact lives at `lean/SpectralTTS/` and is organised into two files: `SpectralTTS/CountingCollapse.lean` (Theorem A and its supporting lemmas) and `SpectralTTS/AttentionDilution.lean` (Lemma B). Both import all of `Mathlib` and are checked against `leanprover/lean4:v4.34.0-rc1` with `mathlib` pinned at the same tag (`lean/SpectralTTS/lean-toolchain`, `lean/SpectralTTS/lakefile.toml`).

Typesetting convention. The listings below reproduce the two Lean source files exactly: every identifier, every tactic, every line of code and every word of every doc-comment is preserved verbatim and in its original order. The one substitution made for typesetting is that Lean's Unicode mathematical notation is rendered through the corresponding standard LaTeX math macro, because the base monospace font used in this document does not contain those glyphs and raw Unicode would silently disappear from the printed page (we checked this directly before committing to this approach). Concretely, in the listings that follow: $\forall$ stands for Lean's universal quantifier, $\exists$ for the existential quantifier, $\le$ for the order relation, $\to$ for the function-space/implication arrow, $\in$ for set/type membership, $\sum$ for the finite-sum big operator, $\mathbb{R}$ and $\mathbb{N}$ for the real and natural number types, $\leftarrow$ for Lean's "rewrite backwards" arrow, and $\langle \cdot, \cdot \rangle$ for Lean's anonymous-constructor brackets; δ and ϵ stand for the one-letter Greek identifiers used throughout both files. Separately, mathlib's naming convention of suffixing a lemma name with a subscript zero to mark a positivity/non-degeneracy side condition (rendered here, e.g., as `div_le_iff0`) is printed with a plain digit rather than a Unicode subscript, for the same font reason. No token is added, removed, or reordered.

1.1 CountingCollapse.lean

The file's own header states the scope precisely; we reproduce it first because it is the most honest single paragraph in the whole artifact:

```
/-
Counting Collapse in Autoregressive Decoders
============================================

Formal core of the paper "Counting Collapse: A Formally Verified Attractor
Theory of Repetition Hallucination in Autoregressive TTS".

Setting.  An autoregressive decoder generating speech for a text prompt that
contains a phrase repeated `k` times passes through a sequence of *repetition
boundary* states `s_0, F s_0, F^2 s_0, ...`, where `F : S → S` is the composite update
the decoder applies over one full repetition of the phrase.  `S` is the decoder
state space (in practice ℝ^d with the Euclidean metric, which is a nonempty
complete metric space).

The modelling assumption is that `F` is the *same* map at every boundary.  This
is what Lemma B (`AttentionDilution.lean`) buys us: softmax attention over `k`
near-identical repeated text spans cannot tell occurrence `j` from occurrence
`j'`, so the conditioning the decoder receives at each boundary is the same up
to `O(δ + 1/k)`.

What is proved here.  *If* `F` is a contraction (`ContractingWith q F`, i.e.
`q < 1` and `F` is `q`-Lipschitz) then the number of repetitions that any
Lipschitz readout can distinguish is finite and bounded explicitly by
`log(2LC/margin) / log(1/q)`.

What is NOT proved here -- and must not be claimed.  That a real transformer
decoder satisfies the contraction hypothesis.  `q` is an empirical quantity: the
accompanying experiments estimate it as `q̂` per model from the observed decay of
boundary-state distances.  The Lean artifact certifies the *implication*, not
the premise.
-/
import Mathlib

namespace SpectralTTS

open Filter Function

variable {S : Type*} [MetricSpace S] [CompleteSpace S] [Nonempty S]
variable {F : S → S} {q : NNReal}
```

Every theorem below is stated for a fixed but arbitrary nonempty complete metric space S (the decoder's hidden-state space, in practice $\mathbb{R}^d$ with the Euclidean metric) and a fixed map $F : S \to S$ (one full repetition's worth of decoder update) with contraction modulus q. These are Lean `variable` declarations, so every subsequent theorem is implicitly universally quantified over them.

orbitRadius and dist_iterate_fixedPoint_le

```
/-- The a-priori radius of the boundary-state orbit: every iterate of `F`
starting at `s` stays within `orbitRadius F s q * q ^ n` of the fixed point.
Writing it as a named quantity keeps the later bounds readable; it is the `C` of
the paper. -/
```

```
noncomputable def orbitRadius (F : S → S) (s : S) (q : NNReal) : ℝ :=
  dist s (F s) / (1 - q)

theorem orbitRadius_nonneg (hF : ContractingWith q F) (s : S) :
    0 ≤ orbitRadius F s q :=
  div_nonneg dist_nonneg hF.one_sub_K_pos.le

/-- **Theorem A(i) -- geometric convergence at repetition boundaries.**
The state after `n` repetitions is within `C q^n` of a single fixed point: the
decoder's record of how many repetitions it has produced decays geometrically.
This is `ContractingWith.apriori_dist_iterate_fixedPoint_le` restated in terms of
`orbitRadius`. -/
theorem dist_iterate_fixedPoint_le (hF : ContractingWith q F) (s : S) (n : ℕ) :
    dist (F^[n] s) (ContractingWith.fixedPoint F hF) ≤ orbitRadius F s q * (q : ℝ) ^ n := by
  have h := hF.apriori_dist_iterate_fixedPoint_le s n
  calc dist (F^[n] s) (ContractingWith.fixedPoint F hF)
      ≤ dist s (F s) * (q : ℝ) ^ n / (1 - q) := h
    _ = orbitRadius F s q * (q : ℝ) ^ n := by unfold orbitRadius; ring
```

`orbitRadius` is a bookkeeping device, not new mathematics: it is literally $C := d(s, Fs)/(1-q)$, the quantity the paper's Theorem 1 calls the orbit scale. `orbitRadius_nonneg` is immediate from $q < 1$ (so $1 - q > 0$) and $d \geq 0$.

`dist_iterate_fixedPoint_le` is Theorem A(i). It is *not* proved from scratch: mathlib already contains the Banach fixed-point theorem and, with it, an a-priori error estimate for Picard iteration, `ContractingWith.apriori_dist_iterate_fixedPoint_le`. The classical argument behind that estimate is the standard telescoping bound: since F is a q-contraction, consecutive iterates satisfy $d(F^i s, F^{i+1} s) \leq q^i d(s, Fs)$, and summing the triangle inequality over $i \geq n$ against the (unique, mathlib-constructed) fixed point s^* gives $d(F^n s, s^*) \leq \sum_{i \geq n} q^i \, d(s, Fs) = \frac{q^n}{1-q} \, d(s, Fs)$. The Lean proof here does no more than `calc`-rewrite that existing lemma through the definition of `orbitRadius`, by `ring`. The mathematical content is entirely mathlib's; the paper's contribution at this step is choosing the right restatement for what follows.

`dist_iterate_iterate_le`

```
/-- Two repetition counts are separated by at most `C (q^m + q^n)` in state
space.  Both orbits are pinned to the same fixed point, so their mutual distance
inherits the geometric decay. -/
theorem dist_iterate_iterate_le (hF : ContractingWith q F) (s : S) (m n : ℕ) :
    dist (F^[m] s) (F^[n] s) ≤ orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n) := by
  have hm := dist_iterate_fixedPoint_le hF s m
  have hn := dist_iterate_fixedPoint_le hF s n
  calc dist (F^[m] s) (F^[n] s)
      ≤ dist (F^[m] s) (ContractingWith.fixedPoint F hF)
        + dist (ContractingWith.fixedPoint F hF) (F^[n] s) := dist_triangle _ _ _
    _ = dist (F^[m] s) (ContractingWith.fixedPoint F hF)
        + dist (F^[n] s) (ContractingWith.fixedPoint F hF) := by rw [dist_comm _ (F^[n] s)]
    _ ≤ orbitRadius F s q * (q : ℝ) ^ m + orbitRadius F s q * (q : ℝ) ^ n := by gcongr
    _ = orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n) := by ring
```

This is the triangle inequality routed through the shared fixed point: $d(F^m s, F^n s) \leq d(F^m s, s^*) + d(s^*, F^n s) \leq Cq^m + Cq^n$, each half supplied by the previous theorem. `gcongr` ("generalized congruence") is a mathlib tactic that discharges a monotone combination of already-proved inequalities automatically; here it combines the two $\leq$-facts term by term. This is the state-space statement behind the paper's claim that any two repetition counts land within $C(q^m + q^n)$ of each

other – it does not yet say anything about a readout.

exists_indistinguishable

```
/-- **Theorem A(ii) -- eventual indistinguishability.**
For any tolerance `ϵ` there is a horizon `N` beyond which *all* repetition counts
produce states within `ϵ` of one another.  No amount of additional repetition
creates new state-space structure. -/
theorem exists_indistinguishable (hF : ContractingWith q F) (s : S) {ϵ : ℝ} (hϵ : 0 < ϵ) :
    ∃ N : ℕ, ∀ m n : ℕ, N ≤ m → N ≤ n → dist (F^[m] s) (F^[n] s) < ϵ := by
  set C := orbitRadius F s q with hC
  have hC0 : 0 ≤ C := orbitRadius_nonneg hF s
  have hden : 0 < 2 * C + 1 := by linarith
  obtain ⟨N, hN⟩ : ∃ N : ℕ, (q : ℝ) ^ N < ϵ / (2 * C + 1) :=
    exists_pow_lt_of_lt_one (by positivity) (by exact_mod_cast hF.1)
  refine ⟨N, fun m n hm hn => ?_⟩
  have hq0 : (0 : ℝ) ≤ (q : ℝ) := q.coe_nonneg
  have hq1 : (q : ℝ) ≤ 1 := le_of_lt (by exact_mod_cast hF.1)
  have hpm : (q : ℝ) ^ m ≤ (q : ℝ) ^ N := pow_le_pow_of_le_one hq0 hq1 hm
  have hpn : (q : ℝ) ^ n ≤ (q : ℝ) ^ N := pow_le_pow_of_le_one hq0 hq1 hn
  have hbound := dist_iterate_iterate_le hF s m n
  -- C (q^m + q^n)  ≤  2C q^N  ≤  2C ϵ/(2C+1)  <  ϵ
  have h1 : C * ((q : ℝ) ^ m + (q : ℝ) ^ n) ≤ 2 * C * (q : ℝ) ^ N := by
    have hsum : (q : ℝ) ^ m + (q : ℝ) ^ n ≤ 2 * (q : ℝ) ^ N := by linarith
    calc C * ((q : ℝ) ^ m + (q : ℝ) ^ n)
        ≤ C * (2 * (q : ℝ) ^ N) := mul_le_mul_of_nonneg_left hsum hC0
      _ = 2 * C * (q : ℝ) ^ N := by ring
  have h2 : 2 * C * (q : ℝ) ^ N ≤ 2 * C * (ϵ / (2 * C + 1)) :=
    mul_le_mul_of_nonneg_left hN.le (by linarith)
  have h3 : 2 * C * (ϵ / (2 * C + 1)) < ϵ := by
    have ht : (0 : ℝ) < ϵ / (2 * C + 1) := div_pos hϵ hden
    have heq : ϵ / (2 * C + 1) * (2 * C + 1) = ϵ := div_mul_cancel0 ϵ hden.ne'
    nlinarith [ht, heq]
  linarith [hbound, h1, h2, h3]
```

This is Theorem A(ii). Read operationally, it says: fix any tolerance $\epsilon > 0$; then there is a step count N (a horizon) past which *every pair* of repetition counts $m, n \geq N$ produces states within ϵ of each other. It is the formal counterpart of "the states the decoder visits stop being mutually distinguishable," the empirical claim tested by the capacity-gain measurement, which is reported in Section 8.3 of this supplement rather than in the main paper: it was cut from the body in the revision that made periodicity the headline, and the pointer here went stale with it.

The proof strategy: because $q < 1$, $q^N \to 0$, so mathlib's `exists_pow_lt_of_lt_one` supplies an N with q^N below any target threshold; we choose the threshold $\epsilon/(2C+1)$ (the "+1" is there purely to keep the denominator positive even when $C = 0$, i.e. when s already *is* the fixed point). For $m, n \geq N$, monotonicity of $q^{(\cdot)}$ on $[0, 1]$ gives $q^m, q^n \leq q^N$, so by `dist_iterate_iterate_le`, $d(F^m s, F^n s) \leq C(q^m + q^n) \leq 2Cq^N$. The chain $2Cq^N \leq 2C \cdot \frac{\epsilon}{2C+1} < \epsilon$ (the last step because $\frac{2C}{2C+1} < 1$) closes the proof by `linarith`/`nlinarith` on the accumulated facts. Nothing here is specific to attention or to speech; it is a direct corollary of contraction.

readout_gap_le

```
/-- **Theorem A(iii) -- no Lipschitz readout can count.**
Any `L`-Lipschitz map `g : S → ℝ` -- the model's own stop-token logit, a duration
predictor, or any linear probe -- sees repetition counts `m` and `n` collapsed to
```

```
within `L C (q^m + q^n)`. -/
theorem readout_gap_le (hF : ContractingWith q F) {L : NNReal} {g : S → ℝ}
    (hg : LipschitzWith L g) (s : S) (m n : ℕ) :
    |g (F^[m] s) - g (F^[n] s)| ≤ (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n)) :=
     by
  have h1 : |g (F^[m] s) - g (F^[n] s)| = dist (g (F^[m] s)) (g (F^[n] s)) :=
    (Real.dist_eq _ _).symm
  rw [h1]
  calc dist (g (F^[m] s)) (g (F^[n] s))
      ≤ (L : ℝ) * dist (F^[m] s) (F^[n] s) := hg.dist_le_mul _ _
    _ ≤ (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n)) := by
        exact mul_le_mul_of_nonneg_left (dist_iterate_iterate_le hF s m n) L.coe_nonneg
```

This is the step that turns a statement about the *state space* into a statement about *anything a downstream mechanism can compute from the state.* g is an arbitrary L-Lipschitz map from S to $\mathbb{R}$: it could be the model's own stop-token logit, a linear counting probe trained after the fact, a duration predictor, or any other bounded-sensitivity read-out. The proof is two lines: rewrite $|g(a) - g(b)|$ as $\mathrm{dist}(g(a), g(b))$ on $\mathbb{R}$ (`Real.dist_eq`), bound it by $L \cdot d(a, b)$ (the definition of Lipschitz), then substitute the state-space bound already proved. No property of g beyond its Lipschitz constant is used, which is exactly why the theorem applies uniformly to greedy decoding, sampling, and beam search alike – the readout argument never inspects how a token was chosen, only how sensitive a fixed function of the state can be.

`counting_margin_le`

```
/-- **Theorem A(iv) -- the counting horizon is finite (algebraic form).**
If a readout still resolves two repetition counts `m ≤ n` with decision margin
`margin`, then `q ^ m` cannot have decayed below `margin / (2 L C)`.  Since
`q < 1`, this bounds `m`: only finitely many repetition counts admit a decision
margin at all. -/
theorem counting_margin_le (hF : ContractingWith q F) {L : NNReal} {g : S → ℝ}
    (hg : LipschitzWith L g) (s : S) {margin : ℝ} {m n : ℕ} (hmn : m ≤ n)
    (hsep : margin ≤ |g (F^[m] s) - g (F^[n] s)|) :
    margin ≤ 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := by
  have hq0 : (0 : ℝ) ≤ (q : ℝ) := q.coe_nonneg
  have hq1 : (q : ℝ) ≤ 1 := le_of_lt (by exact_mod_cast hF.1)
  have hpn : (q : ℝ) ^ n ≤ (q : ℝ) ^ m := pow_le_pow_of_le_one hq0 hq1 hmn
  have hgap := readout_gap_le hF hg s m n
  have hC0 : 0 ≤ orbitRadius F s q := orbitRadius_nonneg hF s
  have hstep : (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n))
      ≤ 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := by
    have hsum : (q : ℝ) ^ m + (q : ℝ) ^ n ≤ 2 * (q : ℝ) ^ m := by linarith
    have hinner : orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n)
        ≤ orbitRadius F s q * (2 * (q : ℝ) ^ m) := mul_le_mul_of_nonneg_left hsum hC0
    calc (L : ℝ) * (orbitRadius F s q * ((q : ℝ) ^ m + (q : ℝ) ^ n))
        ≤ (L : ℝ) * (orbitRadius F s q * (2 * (q : ℝ) ^ m)) :=
          mul_le_mul_of_nonneg_left hinner L.coe_nonneg
      _ = 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := by ring
  linarith
```

This is the contrapositive step written out algebraically, before taking logarithms. Assume the readout *does* separate counts $m \le n$ by at least `margin`. Since $m \le n$ and $0 \le q \le 1$, $q^n \le q^m$, so $q^m + q^n \le 2q^m$; feeding this into `readout_gap_le`'s bound $LC(q^m + q^n)$ gives the tighter bound $2LCq^m$. Chaining $\texttt{margin} \le LC(q^m + q^n) \le 2LCq^m$ is exactly the statement proved. In words: whatever separation a Lipschitz readout manages to extract between counts m and n,

that separation is itself capped by a quantity that decays geometrically in the *smaller* of the two counts alone – which is the algebraic seed of the horizon bound.

le_log_div_log_of_le_pow

```
/-- Real-arithmetic bridge: a geometric quantity bounded below is bounded in its
exponent.  Used to turn `counting_margin_le` into an explicit horizon. -/
theorem le_log_div_log_of_le_pow {r c : ℝ} (hr0 : 0 < r) (hr1 : r < 1) (hc : 0 < c)
    {m : ℕ} (h : c ≤ r ^ m) : (m : ℝ) ≤ Real.log c / Real.log r := by
  have hlogr : Real.log r < 0 := Real.log_neg hr0 hr1
  have hlog : Real.log c ≤ Real.log (r ^ m) := Real.log_le_log hc h
  rw [Real.log_pow] at hlog
  rw [le_div_iff_of_neg hlogr]
  linarith [hlog]
```

A general-purpose real-analysis lemma with no reference to decoders, states, or contractions: if $0 < r < 1$, $0 < c$, and $c \le r^m$, then $m \le \log c / \log r$. The subtlety, and the reason this needs its own lemma rather than a one-line `gcongr`, is the sign flip: because $0 < r < 1$, $\log r < 0$ (`Real.log_neg`), so dividing the monotone consequence $\log c \le m \log r$ (from `Real.log_le_log` applied to $c \le r^m$, then `Real.log_pow` to expand $\log(r^m) = m \log r$) by the *negative* quantity $\log r$ reverses the inequality's direction – handled here by mathlib's `le_div_iff_of_neg`, which is precisely the division lemma for a negative denominator. This lemma is the generic engine that turns `counting_margin_le`'s algebraic inequality into an explicit bound on m in the next theorem; note that `scripts/check_lean.sh`'s axiom audit does not list this lemma by name (Section 1.3 explains why that omission is harmless).

counting_horizon

```
/-- **Theorem A -- Counting Collapse (explicit horizon).**
The headline statement.  Under contraction, a Lipschitz readout with decision
margin `margin > 0` can separate the repetition count `m` from any larger count
only while

    m ≤ log(margin / (2 L C)) / log q  =  log(2 L C / margin) / log(1 / q).

Beyond that horizon the decoder is provably unable to tell how many repetitions
it has already produced, so its continue/stop decision is decoupled from the
count -- the failure mode observed empirically as looping or truncation. -/
theorem counting_horizon (hF : ContractingWith q F) {L : NNReal} {g : S → ℝ}
    (hg : LipschitzWith L g) (s : S) {margin : ℝ} {m n : ℕ}
    (hq0 : (0 : ℝ) < (q : ℝ)) (hmargin : 0 < margin) (hmn : m ≤ n)
    (hsep : margin ≤ |g (F^[m] s) - g (F^[n] s)|) :
    (m : ℝ) ≤ Real.log (margin / (2 * (L : ℝ) * orbitRadius F s q)) / Real.log (q : ℝ) := by
  have hq1 : (q : ℝ) < 1 := by exact_mod_cast hF.1
  have hbound := counting_margin_le hF hg s hmn hsep
  have hC0 : 0 ≤ orbitRadius F s q := orbitRadius_nonneg hF s
  -- a nonzero decision margin forces the prefactor to be strictly positive
  have hprefix : (0 : ℝ) ≤ 2 * (L : ℝ) * orbitRadius F s q :=
    mul_nonneg (mul_nonneg (by norm_num) L.coe_nonneg) hC0
  have hpos : 0 < 2 * (L : ℝ) * orbitRadius F s q := by
    rcases lt_or_eq_of_le hprefix with h | h
    · exact h
    · exfalso
      rw [← h, zero_mul] at hbound
```

```
    linarith
  have hratio : margin / (2 * (L : ℝ) * orbitRadius F s q) ≤ (q : ℝ) ^ m := by
    rw [div_le_iff0 hpos]
    calc margin ≤ 2 * (L : ℝ) * orbitRadius F s q * (q : ℝ) ^ m := hbound
      _ = (q : ℝ) ^ m * (2 * (L : ℝ) * orbitRadius F s q) := by ring
  exact le_log_div_log_of_le_pow hq0 hq1 (by positivity) hratio
```

This is Theorem A(iv), the operative statement of the paper, combining `counting_margin_le` with the log bridge just discussed. Two details are worth pointing out because they are exactly the kind of thing a pen-and-paper proof would gloss over and Lean forces to be made explicit.

First, before the log bridge can apply, the prefactor $2LC$ must be shown *strictly* positive (the bridge lemma requires $0 < c$ for $c = \text{margin}/(2LC)$ to be well-formed as a division and for `Real.log` to behave). The proof handles the boundary case directly: if $2LC$ were 0, then `hbound` ($\text{margin} \le 2LC \cdot q^m$) would force $\text{margin} \le 0$, contradicting the hypothesis $\text{margin} > 0$ – so the very existence of a positive decision margin is what rules out the degenerate case $L = 0$ or $C = 0$ (a zero-Lipschitz-constant readout, or an already-converged orbit). This is the `rcases lt_or_eq_of_le` case split ending in `exfalso`.

Second, the theorem as stated in Lean gives $m \le \log\big(\text{margin}/(2LC)\big)/\log q$, while the main paper's Equation (1) writes the horizon as $\log(2LC/\text{margin})/\log(1/q)$. These are the same quantity: since $\text{margin}/(2LC) \in (0,1)$ whenever the readout achieves a real separation (so its log is negative) and $q \in (0,1)$ (so $\log q$ is also negative), $\log(a)/\log(b) = \log(1/a)/\log(1/b)$ for $a = \text{margin}/(2LC)$, $b = q$ – a negative-over-negative flips to a positive-over-positive with the reciprocal arguments. The Lean statement and the paper's Equation (1) are the identical horizon, just written with the sign convention that falls out of `le_log_div_log_of_le_pow` versus the convention chosen for exposition.

1.2 `AttentionDilution.lean`

```
/-
Attention Dilution
==================

Lemma B of "Counting Collapse: A Formally Verified Attractor Theory of
Repetition Hallucination in Autoregressive TTS".

Setting.  A decoder step attends over the `k` text positions occupied by `k`
repetitions of the same phrase.  Because those positions carry (near-)identical
content, their attention logits are near-identical too: we assume only that the
logits lie within a spread `δ` of one another, which is a directly measurable
quantity.

What is proved.  Every occurrence receives attention weight in
`[e^{-δ}/k, e^{δ}/k]`; consequently the attention profile over the repeated
block is within `(e^{δ} - e^{-δ})/k` of uniform, and its entropy is at least
`log k - δ`.  As `k` grows the profile flattens: the decoder cannot use
attention weight to identify *which* occurrence it is currently rendering.

Why it matters.  This licenses the modelling assumption of
`CountingCollapse.lean` -- that the conditioning at every repetition boundary is
the same map `F`.  The link is quantitative but informal (an `O(δ + 1/k)`
perturbation of the conditioning yields an `O(δ + 1/k)` perturbation of `F`); the
paper states it as a remark, not as a formalized theorem.
-/
import Mathlib
```

```
namespace SpectralTTS

open Finset Real

variable {k : ℕ}

/-- Softmax over the `k` attention logits of a repeated text block. -/
noncomputable def softmax (z : Fin k → ℝ) (i : Fin k) : ℝ :=
  Real.exp (z i) / ∑ j, Real.exp (z j)

theorem sum_exp_pos [NeZero k] (z : Fin k → ℝ) : 0 < ∑ j, Real.exp (z j) :=
  Finset.sum_pos (fun j _ => Real.exp_pos (z j))
    ⟨⟨0, Nat.pos_of_ne_zero (NeZero.ne k)⟩, mem_univ _⟩

theorem softmax_nonneg [NeZero k] (z : Fin k → ℝ) (i : Fin k) : 0 ≤ softmax z i :=
  div_nonneg (Real.exp_pos _).le (sum_exp_pos z).le
```

`softmax` is defined exactly as one would expect – $\mathrm{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$ over the k logits of a repeated block – and `sum_exp_pos`/`softmax_nonneg` are routine positivity facts needed to make the subsequent division lemmas well-formed (the denominator is a finite sum of strictly positive terms over a nonempty index type, guaranteed by the `[NeZero k]` instance).

`sum_softmax`

```
/-- The softmax profile is a probability distribution. -/
theorem sum_softmax [NeZero k] (z : Fin k → ℝ) : ∑ i, softmax z i = 1 := by
  unfold softmax
  rw [← Finset.sum_div, div_self (sum_exp_pos z).ne']
```

Establishes that `softmax` really is a probability distribution over the k occurrences: $\sum_i e^{z_i} / \sum_j e^{z_j} = \left(\sum_i e^{z_i}\right) / \sum_j e^{z_j} = 1$, by pulling the common denominator out of the sum (`Finset.sum_div`, used backwards) and cancelling it against itself. This is a prerequisite for the entropy statement later, since Shannon entropy is only meaningful over a genuine distribution.

`softmax_le` and `le_softmax`

```
/-- **Lemma B (upper bound).**  If all logits in the repeated block lie within
`δ` of one another, no single occurrence can capture more than `e^{δ}/k` of the
attention mass. -/
theorem softmax_le [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ) (i : Fin k) :
    softmax z i ≤ Real.exp δ / k := by
  have hk : (0 : ℝ) < k := Nat.cast_pos.mpr (Nat.pos_of_ne_zero (NeZero.ne k))
  -- every term of the denominator is at least `exp (z i - δ)`
  have hlow : (k : ℝ) * Real.exp (z i - δ) ≤ ∑ j, Real.exp (z j) := by
    have hterm : ∀ j ∈ (univ : Finset (Fin k)), Real.exp (z i - δ) ≤ Real.exp (z j) := by
      intro j _
      exact Real.exp_le_exp.mpr (by linarith [hz i j])
    calc (k : ℝ) * Real.exp (z i - δ)
        = ∑ _j : Fin k, Real.exp (z i - δ) := by
          rw [Finset.sum_const, card_univ, Fintype.card_fin, nsmul_eq_mul]
      _ ≤ ∑ j, Real.exp (z j) := Finset.sum_le_sum hterm
  unfold softmax
  rw [div_le_div_iff0 (sum_exp_pos z) hk]
  have hkey : Real.exp (z i) * (k : ℝ) = Real.exp δ * ((k : ℝ) * Real.exp (z i - δ)) := by
```

```
    rw [Real.exp_sub]
    field_simp
  rw [hkey]
  exact mul_le_mul_of_nonneg_left hlow (Real.exp_pos δ).le

/-- **Lemma B (lower bound).**  Symmetrically, no occurrence can be starved
below `e^{-δ}/k`. -/
theorem le_softmax [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ) (i : Fin k) :
    Real.exp (-δ) / k ≤ softmax z i := by
  have hk : (0 : ℝ) < k := Nat.cast_pos.mpr (Nat.pos_of_ne_zero (NeZero.ne k))
  have hhigh : ∑ j, Real.exp (z j) ≤ (k : ℝ) * Real.exp (z i + δ) := by
    have hterm : ∀ j ∈ (univ : Finset (Fin k)), Real.exp (z j) ≤ Real.exp (z i + δ) := by
      intro j _
      exact Real.exp_le_exp.mpr (by linarith [hz j i])
    calc ∑ j, Real.exp (z j)
        ≤ ∑ _j : Fin k, Real.exp (z i + δ) := Finset.sum_le_sum hterm
      _ = (k : ℝ) * Real.exp (z i + δ) := by
          rw [Finset.sum_const, card_univ, Fintype.card_fin, nsmul_eq_mul]
  unfold softmax
  rw [div_le_div_iff0 hk (sum_exp_pos z)]
  have hkey : Real.exp (-δ) * ((k : ℝ) * Real.exp (z i + δ)) = Real.exp (z i) * (k : ℝ) := by
    rw [Real.exp_add, Real.exp_neg]
    field_simp
  calc Real.exp (-δ) * ∑ j, Real.exp (z j)
      ≤ Real.exp (-δ) * ((k : ℝ) * Real.exp (z i + δ)) :=
        mul_le_mul_of_nonneg_left hhigh (Real.exp_pos _).le
    _ = Real.exp (z i) * (k : ℝ) := hkey
```

These two theorems together are Lemma B's headline bound. The hypothesis `hz` says the k logits of the repeated block are mutually within δ of each other – a directly measurable spread, not an assumption about the mechanism producing the logits.

For the upper bound: since $z_i - z_j \le \delta$ for every j, each term of the denominator satisfies $e^{z_j} \ge e^{z_i-\delta}$, so summing k copies of that lower bound gives $\sum_j e^{z_j} \ge k\, e^{z_i-\delta}$. Substituting into the definition of softmax, $\mathrm{softmax}(z)_i = e^{z_i}/\sum_j e^{z_j} \le e^{z_i}/(k\, e^{z_i-\delta}) = e^{\delta}/k$. The lower bound is the mirror argument with the roles of i and j swapped in the hypothesis ($z_j - z_i \le \delta$, i.e. $e^{z_j} \le e^{z_i+\delta}$ for every j), giving $\sum_j e^{z_j} \le k\, e^{z_i+\delta}$ and hence $\mathrm{softmax}(z)_i \ge e^{-\delta}/k$. Both proofs are the same three-step pattern – bound every summand, sum k identical bounds via `Finset.sum_const`, divide – executed twice with the inequality direction and the sign of δ flipped.

`softmax_dist_le`

```
/-- **Occurrence indistinguishability.**  The attention profile over a block of
`k` repeated spans is within `(e^{δ} - e^{-δ})/k` of uniform.  For fixed `δ`
this vanishes as `1/k`: attention weight carries no information about *which*
repetition is being attended to. -/
theorem softmax_dist_le [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ)
    (i j : Fin k) :
    |softmax z i - softmax z j| ≤ (Real.exp δ - Real.exp (-δ)) / k := by
  have hsplit : (Real.exp δ - Real.exp (-δ)) / (k : ℝ)
      = Real.exp δ / (k : ℝ) - Real.exp (-δ) / (k : ℝ) := by ring
  have hi_up := softmax_le z hz i
  have hj_up := softmax_le z hz j
  have hi_lo := le_softmax z hz i
  have hj_lo := le_softmax z hz j
  rw [abs_sub_le_iff]
```

```
  constructor <;> linarith
```

A direct corollary: both $\mathrm{softmax}(z)_i$ and $\mathrm{softmax}(z)_j$ lie in the interval $[e^{-\delta}/k,\, e^{\delta}/k]$, an interval of width $(e^{\delta} - e^{-\delta})/k$, so their difference cannot exceed that width in either direction (`abs_sub_le_iff` splits $|x-y| \le c$ into the two one-sided inequalities $x - y \le c$ and $y - x \le c$, both closed by `linarith` from the four bounds already in scope). For fixed δ this bound is $\Theta(1/k)$: as the block grows, no two occurrences' attention weights can differ by more than an amount vanishing in k – the formal content of "attention weight carries vanishing information about which occurrence is being rendered."

`entropy_ge`

```
/-- **Attention entropy grows like `log k`.**  With `p i = softmax z i`, the
Shannon entropy of the profile over the repeated block satisfies
`H(p) ≥ log k - δ`: near-uniform attention over `k` identical spans is maximally
uninformative, so the decoder's effective conditioning becomes the block mean and
loses occurrence identity. -/
theorem entropy_ge [NeZero k] (z : Fin k → ℝ) {δ : ℝ} (hz : ∀ a b, z a - z b ≤ δ) :
    Real.log k - δ ≤ ∑ i, softmax z i * (-Real.log (softmax z i)) := by
  have hk : (0 : ℝ) < k := Nat.cast_pos.mpr (Nat.pos_of_ne_zero (NeZero.ne k))
  have key : ∀ i : Fin k, (Real.log k - δ) * softmax z i
      ≤ softmax z i * (-Real.log (softmax z i)) := by
    intro i
    have hup := softmax_le z hz i
    have hlo : 0 < softmax z i := by
      have h1 : Real.exp (-δ) / (k : ℝ) ≤ softmax z i := le_softmax z hz i
      have h2 : (0 : ℝ) < Real.exp (-δ) / (k : ℝ) := by positivity
      linarith
    have hlog : Real.log (softmax z i) ≤ δ - Real.log k := by
      have h1 : Real.log (softmax z i) ≤ Real.log (Real.exp δ / k) := Real.log_le_log hlo hup
      rwa [Real.log_div (Real.exp_pos δ).ne' hk.ne', Real.log_exp] at h1
    have hneg : Real.log k - δ ≤ -Real.log (softmax z i) := by linarith
    calc (Real.log k - δ) * softmax z i
        ≤ (-Real.log (softmax z i)) * softmax z i :=
          mul_le_mul_of_nonneg_right hneg hlo.le
      _ = softmax z i * (-Real.log (softmax z i)) := mul_comm _ _
  calc Real.log k - δ
      = (Real.log k - δ) * ∑ i, softmax z i := by rw [sum_softmax z]; ring
    _ = ∑ i, (Real.log k - δ) * softmax z i := by rw [Finset.mul_sum]
    _ ≤ ∑ i, softmax z i * (-Real.log (softmax z i)) := Finset.sum_le_sum (fun i _ => key i)
```

The entropy floor. Write $H(p) = \sum_i p_i(-\log p_i)$ for the Shannon entropy of the attention profile $p_i = \mathrm{softmax}(z)_i$. The proof first shows a pointwise inequality (`key`): for every occurrence i, $(\log k - \delta)\, p_i \le p_i(-\log p_i)$. This follows because `softmax_le` gives $p_i \le e^{\delta}/k$, hence (taking logs, valid since $p_i > 0$ by `le_softmax`) $\log p_i \le \delta - \log k$, i.e. $-\log p_i \ge \log k - \delta$; multiplying both sides of that by the nonnegative p_i preserves the inequality. Summing the pointwise bound over all k occurrences and using $\sum_i p_i = 1$ (`sum_softmax`) turns $\sum_i (\log k - \delta)\, p_i$ into exactly $\log k - \delta$, giving $H(p) \ge \log k - \delta$. As $k \to \infty$ for fixed δ, the minimum possible entropy of the attention profile over the repeated block grows without bound like $\log k$: attention over a large repeated block cannot be concentrated, no matter how the model's underlying logits are computed, subject only to their spread being bounded by δ.

1.3 The axiom audit

`scripts/check_lean.sh` is the verification gate. It performs three checks in sequence: (1) `lake build` succeeds, i.e. the project type-checks against the pinned mathlib revision; (2) `grep -rn "sorry"` over `SpectralTTS/` and `SpectralTTS.lean` finds no occurrence, i.e. no proof obligation anywhere in the source was left unproved with the `sorry` placeholder; and (3) an axiom audit, reproduced verbatim below, that prints the transitive axiom dependency of eleven exported theorems and fails the build if the string `sorryAx` appears anywhere in that output.

```
== axiom audit
cat > /tmp/spectraltts_axioms.lean <<'EOF'
import SpectralTTS
open SpectralTTS
#print axioms SpectralTTS.dist_iterate_fixedPoint_le
#print axioms SpectralTTS.dist_iterate_iterate_le
#print axioms SpectralTTS.exists_indistinguishable
#print axioms SpectralTTS.readout_gap_le
#print axioms SpectralTTS.counting_margin_le
#print axioms SpectralTTS.counting_horizon
#print axioms SpectralTTS.softmax_le
#print axioms SpectralTTS.le_softmax
#print axioms SpectralTTS.softmax_dist_le
#print axioms SpectralTTS.entropy_ge
#print axioms SpectralTTS.sum_softmax
EOF
OUT=$(lake env lean /tmp/spectraltts_axioms.lean 2>&1)
echo "$OUT"
if echo "$OUT" | grep -q "sorryAx"; then
  echo "FAIL: sorryAx in dependency closure"; exit 1
fi
```

What the three admitted axioms mean. Lean's kernel tracks, for every proof term, exactly which axioms it ultimately rests on beyond pure type-theoretic reduction. Three axioms are considered standard and non-suspicious throughout ordinary (i.e. classical) formalised mathematics, and mathlib is built expecting all three:

- **`propext`** (propositional extensionality): two propositions that are provably logically equivalent are equal as terms of type `Prop`. This licenses rewriting one true statement for another and is used pervasively any time an equality between propositions, rather than merely an if-and-only-if, is needed internally.

- **`Classical.choice`**: the axiom of choice, in the form that every nonempty type has a term. Mathlib is a classical (not constructive) library and uses this axiom routinely – for instance, to decide the truth of an arbitrary proposition about real numbers (`Real.log`'s case splits, order comparisons, and the existence statement in `ContractingWith.fixedPoint` all rest on classical reasoning of this kind).

- **`Quot.sound`**: quotient soundness. Lean 4 builds quotient types (used, transitively, to construct $\mathbb{R}$ as a quotient of Cauchy sequences, `NNReal` as a subtype of that, and `Finset` as a quotient of `Multiset` which is itself a quotient of `List`) and this axiom is what makes the definitional behaviour of quotients sound at the kernel level.

Essentially every nontrivial mathlib theorem depends on some subset of these three; a formal result is treated as unconditionally established in this ecosystem exactly when its axiom dependency is a subset of {`propext`, `Classical.choice`, `Quot.sound`}.

What `sorryAx` would mean. Whenever a proof anywhere in a term's transitive dependency closure contains an unproved hole – written `sorry`, or produced by an incomplete tactic block – Lean's elaborator inserts a synthetic axiom called `sorryAx` standing in for "trust me." A theorem whose dependency closure contains `sorryAx` is not actually proved: it is, formally, an assumption. Because `sorryAx` propagates through every downstream use of the offending lemma, a single unproved hole anywhere upstream – even inside a helper three calls deep, even hypothetically inside a buggy mathlib lemma – would surface in the audit of every theorem that transitively depends on it. This is why the `grep -rn "sorry"` textual scan (step 2) and the semantic `#print axioms` scan (step 3) are both run: the grep catches `sorry` written directly in this project's own two files, while the axiom audit is the only check that would also catch a hole hidden inside a project-local tactic-generated term that the textual scan cannot see, or (in principle) an inconsistency imported from mathlib itself. Passing both certifies that all eleven audited theorems reduce, through Lean's kernel type-checker, to a proof term whose only axioms are the three standard ones above.

Coverage note. The audit list contains eleven theorems: the seven from `CountingCollapse.lean` covered in Section 1.1 except `le_log_div_log_of_le_pow`, plus the four listed and named theorems from `AttentionDilution.lean` (`softmax_le`, `le_softmax`, `softmax_dist_le`, `entropy_ge`) and `sum_softmax`. `le_log_div_log_of_le_pow` is not itself named in the `#print axioms` list, but this is not a coverage gap: it is a direct dependency of `counting_horizon`, so Lean's kernel necessarily re-checks it (and everything *it* depends on) while type-checking `counting_horizon`, and any axiom it introduced would appear in `counting_horizon`'s own reported axiom set. Auditing `counting_horizon` therefore already certifies `le_log_div_log_of_le_pow` transitively. The same reasoning covers the unlisted helper lemmas `orbitRadius_nonneg`, `sum_exp_pos`, and `softmax_nonneg`: each is a dependency of an audited theorem.

What is and is not proved (restated plainly). The Lean artifact certifies an implication, not a fact about trained transformers. Concretely: *if* a decoder's per-repetition update map F is a genuine q-contraction ($q < 1$, Lipschitz) on its hidden-state space, *then* Theorem A(i)–(iv) follow with no further assumptions, machine-checked down to three standard axioms. Whether any given trained decoder's F actually is a contraction is not something proved here or provable by this kind of argument at all – it is an empirical property of a specific trained network, measured per model as $\hat{q}$ in the main paper's Section 4 and Section 4 below. Symmetrically, Lemma B (attention dilution) is unconditional – it holds for any bounded-spread logits, which is a directly measurable, not assumed, property of a forward pass – but its role is only to license the informal step from "attention is diluted" to "the per-repetition map is approximately the same map at every boundary" (Assumption 1 of the main paper), and that step is itself not formalised; see Section 2.

2 What is deliberately not formalized

The main paper's Assumption 1 (periodic conditioning: there is a single map F with $s_{m+1} = F(s_m)$ for every boundary m) is presented as "what Lemma B buys us." This is true informally and is stated as such in both the paper and the `AttentionDilution.lean` header: *"The link is quantitative but*

informal (an $O(\delta + 1/k)$ perturbation of the conditioning yields an $O(\delta + 1/k)$ perturbation of F); the paper states it as a remark, not as a formalized theorem." This section says precisely what that remark asserts, why it was not formalised, and what a full formal treatment would need to add. Nothing here is proved in Lean; this section is pen and paper by design, and we are explicit about that so the boundary between the machine-checked and the informal parts of the argument is never ambiguous to a reader.

2.1 What the remark actually claims

Lemma B, as formalised, is a statement about a single decoder step's attention profile over the k text spans occupied by k repetitions: if those spans' logits lie within spread δ, the resulting softmax weights are within $(e^{\delta} - e^{-\delta})/k$ of uniform (`softmax_dist_le`) and the profile's entropy is at least $\log k - \delta$ (`entropy_ge`). Neither of these is yet a statement about F, the map *between repetition boundaries* that Theorem A's contraction hypothesis is about. The step from one to the other requires two things Lemma B does not supply:

1. A model of *what the decoder update depends on.* F, as used in `CountingCollapse.lean`, is an autonomous map $S \to S$ with no explicit dependence on anything besides the current state s. In reality the per-repetition update is a function of the current state *and* the conditioning context the decoder attends to at that step – write this as $F_c : S \to S$ for a context c drawn from some space of possible conditioning contexts (e.g. the vector of attended text values, not merely the attention weights Lemma B bounds). Autonomy of F is only recovered *if* the sequence of contexts $c_0, c_1, \ldots$ actually visited at successive repetition boundaries is (nearly) constant.

2. A translation of Lemma B's bound on *attention weights* into a bound on the resulting *context vector*. Lemma B bounds $|\mathrm{softmax}(z)_i - \mathrm{softmax}(z)_j|$, a statement purely about the weights. The context the decoder actually receives is a weight-averaged combination of *value* vectors, and Lemma B says nothing about those values – it only constrains how uniformly the weights spread across positions whose values are assumed, but not shown, to be similar because the k repeated spans carry near-identical text.

Granting both informally – flat attention over near-identical spans yields a near-constant context vector, to an error the paper writes as $O(\delta + 1/k)$ – the remark's claim is: a family of maps $\{F_c\}$ that is uniformly Lipschitz in c (i.e. nearby contexts produce nearby updates, $d(F_c(s), F_{c'}(s)) \leq \kappa\, d_C(c, c')$ for some constant κ and every s), fed a sequence of contexts $c_0, c_1, \ldots$ that are all within $O(\delta + 1/k)$ of some reference context c^*, behaves *approximately* like the single autonomous map F_{c^*}, with an error that itself scales as $O(\delta + 1/k)$ per step. This is the sense in which Assumption 1 is "bought" by Lemma B: not literally, but up to a perturbation that shrinks as the repeated block grows.

2.2 Why this was not formalised

Formalising the remark above is a strictly larger undertaking than formalising Theorem A and Lemma B as stated, for reasons that are worth being concrete about rather than hand-waving as "future work":

1. **A new object.** The current development has no type for "conditioning context" or for a context-indexed family of maps $\{F_c\}_{c \in C}$; this would need to be introduced, along with a

metric d_C on contexts and a joint-Lipschitz hypothesis relating d_C to the induced variation in F_c. None of this exists in either Lean file today.

2. **A quantitative re-derivation in the context metric, not the weight metric.** `softmax_dist_le` bounds a difference of scalar attention weights. Bounding the resulting difference in *context vectors* $d_C(c_i, c_j)$ requires an additional Lipschitz-type assumption on the value vectors being attended to (that repeated spans carry not just diluted attention but near-identical *values*), which is a second, currently unstated and unmeasured premise, and then a short but genuine argument (a weighted-average version of the triangle inequality) to convert a bound on weight differences plus a bound on value differences into a bound on the resulting weighted sum.

3. **A perturbed-contraction (shadowing) lemma.** Even granting (1) and (2), Theorem A's proof chain (Section 1) is stated for a single autonomous F. Propagating a per-step drift of size $\rho = O(\delta + 1/k)$ through it requires a genuinely different fixed-point argument: something in the spirit of "a sequence of q-contractions $F_{c_0}, F_{c_1}, \ldots$ whose contexts all lie within ρ of a common reference point produces an orbit that shadows the autonomous orbit of F_{c^*} up to an additive error that itself stabilises geometrically, to a limiting value on the order of $\kappa\rho/(1-q)$." This is a standard *shape* of result in the perturbation theory of dynamical systems (a discrete analogue of structural stability / shadowing for contractions), but it is not a lemma mathlib currently provides in a directly reusable form for this setting, and instantiating or proving it was out of scope for this artifact.

4. **Propagating the correction through the horizon.** Granting (3), every inequality in Theorem A(i)–(iv) would pick up an additive term of order $\kappa\rho/(1-q)$ on top of the geometric Cq^m decay, and the horizon of Equation (1) would gain a k-dependent correction. Because $\rho = O(\delta + 1/k)$ shrinks precisely as k (and hence m, the quantity the horizon bounds) grows, the correction is plausibly a lower-order effect in the regime the paper cares about – but showing this rigorously is a two-scale argument (the perturbation shrinking in the same variable the main bound is stated over) that requires genuinely new mathematical content, not routine bookkeeping, and was not attempted.

2.3 What a full treatment would need

Concretely, extending the formal artifact to close this gap would require, in order: (a) a formal definition of a conditioning-context space C with a metric d_C and a context-indexed family $F : C \to S \to S$; (b) a hypothesis package bundling (i) κ-Lipschitz continuity of F in its context argument, uniformly in the state argument, and (ii) a quantitative bound, in d_C, on the distance between the contexts realised at successive repetition boundaries – the latter to be derived from a strengthened version of Lemma B that bounds the attended *value* vectors' contribution, not only the attention weights; (c) a perturbed-orbit / shadowing lemma for metric-space contractions under a bounded per-step context drift, proved or sourced from mathlib; and (d) a re-derivation of Theorem A(i)–(iv) with the additive $O(\kappa\rho/(1-q))$ correction carried through explicitly, together with an argument (formal or at least made quantitatively precise) that this correction is dominated by the main geometric term in the k-range where the horizon is informative. None of (a)–(d) is present in the current artifact; the current Lean development proves Theorem A and Lemma B exactly as stated in Section 1, and the bridge between them remains, honestly, a remark.

3 The benchmark in full

The benchmark is built by `data/stimuli/make_stimuli.py` and written to `data/stimuli/stimuli.jsonl`, one JSON object per line. It contains 180 items across five families, confirmed by direct count against the generated file:

Table 1: Stimulus families.

Family	Count	What it tests
`word_rep`	60	a single word repeated k times in a fixed carrier
`control_word`	54	the same carrier, k *distinct* filler words (no repetition)
`sentence_rep`	32	a whole short sentence repeated k times
`twisters`	24	a tongue-twister sentence (with internal repetition) repeated k times
`numbers`	10	English number phrases with intrinsic digit-word repetition
Total	180	

Every item carries an explicit `item_id`, `family`, `template`, `text`, `target_unit`, `k`, and `expected_count` – a ground-truth number of occurrences of the target unit, so a generation can be scored by counting rather than by string match against a reference transcript. `control_word` items additionally carry `control_of` (the `item_id` of the repeated item they are matched to) and `control_units` (the k distinct fillers actually substituted).

3.1 The k-ladder

`word_rep` and `control_word` items are built at

$$k \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32\}$$

(`K_LADDER` in `make_stimuli.py`). The ladder is dense at the low end and log-spaced above; the source comment explains why directly: the collapse point k^* is expected in the 4–16 range, so that is where resolution is spent, without paying the instrumentation and generation cost of testing every integer k up to 32. `sentence_rep` uses a truncated ladder $k \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ (`SENTENCE_K`), since a full sentence repeated 32 times would be prohibitively long to generate and transcribe. `twisters` use only $k \in \{1, 2, 4\}$, since a twister sentence already contains internal repetition (Table 5) and the family exists to probe alliterative/phonological stress rather than long-range counting. `numbers` items are not built on a k-ladder at all: each number phrase is a single fixed string with its own intrinsic repetition count (Table 3).

3.2 Word-repetition carrier templates

Table 2 reproduces all six carrier templates (`WORD_TEMPLATES`) verbatim: the fixed prefix and suffix, the target word repeated k times between them, and the pool of eight distinct fillers used to build the matched control at the same k. An item's text is `prefix + " " + (target repeated k times) + " " + suffix`; a control's text substitutes `prefix + " " + (k distinct fillers, cycled through the pool) + " " + suffix`, guaranteeing the target word itself never appears in its own control.

Table 2: Word-repetition carrier templates (`WORD_TEMPLATES`, `data/stimuli/make_stimuli.py`).

ID	Prefix	Target	Suffix	Filler pool (for the control)
t1	The dog was	very	big and it ran across the field.	quite, really, truly, fairly, rather, somewhat, extremely, notably
t2	She said	no	to the offer and then left the room.	yes, maybe, sure, fine, okay, well, right, hmm
t3	He walked	far	into the forest before turning back.	deep, fast, long, wide, high, low, near, past
t4	The light was	blue	before the storm arrived.	green, bright, pale, dim, warm, cold, sharp, soft
t5	The runner moved	quick	along the narrow path.	swift, smooth, steady, light, sharp, loose, tight, clean
t6	They were	really	tired after the long journey.	truly, quite, very, rather, deeply, clearly, plainly, surely

Why these six target words. The source comment states the selection criterion directly: targets are "chosen for ASR robustness: common, monosyllabic-or-disyllabic, no homophone confusions, and unlikely to be swallowed by Whisper's LM prior." Concretely, *very*, *no*, *far*, *blue*, *quick*, and *really* are all common, short, phonetically distinct from every filler in their own pool, and none are function words or discourse markers that an ASR language-model prior might silently elide or merge with an adjacent word – a real risk for a word repeated up to 32 times in a row, which is exactly the regime this benchmark stresses. This choice matters because the behavioural metric (Section 4) counts occurrences of the target unit in the ASR transcript; a target word prone to ASR mis-transcription independent of the TTS model's behaviour would contaminate that count with ASR noise rather than TTS behaviour.

Superseded by the audit (Section 9.1). This selection criterion assumed the risk was *lexical* – that choosing short, common, phonetically unambiguous target words could engineer around Whisper's language-model prior item by item. The concatenative audit built from exactly these six words (*very, no, far, blue, quick, really*) shows that assumption does not hold: on audio with exact ground truth, Whisper undercounts these same targets at a mean counted/true ratio of 0.27–0.65 for $k \geq 4$, while a CTC recogniser scores the identical audio at ratio 1.00 throughout (Table 14). The risk was never which word was chosen; it is that the judge's own decoder is autoregressive, which no choice of target word can fix. Every behavioural number in this document that is judged by Whisper should be read against that fact.

3.3 Number phrases

Table 3 reproduces all ten number-phrase stimuli (`NUMBER_PHRASES`) verbatim, with the target digit-word and its true occurrence count. Text is capitalised and given a trailing period at generation time (`text.capitalize() + "."`). The family's rationale, from the source comment: "English number words are the natural repetition stress test: the same digit-word recurs at several magnitudes and the model must keep an internal place-value counter that repetition-attractor dynamics would destroy." Unlike `word_rep`, the repeated unit here is not adjacent tokens but a digit-word recurring at different magnitude positions (hundreds, thousands, millions), so a correct rendering additionally requires tracking place value, not merely repeating a token a fixed number of times.

At scoring time (Section 4), a numeral rendered by Whisper as digits (e.g. `"666,666"`) is algorithmically expanded back into English number words by `expand_number_token` in

Table 3: Number-phrase stimuli (`NUMBER_PHRASES`, `data/stimuli/make_stimuli.py`).

ID	Phrase	Target	Count
n01	six hundred sixty six thousand six hundred sixty six	six	6
n02	seven hundred seventy seven thousand seven hundred seventy seven	seven	6
n03	nine hundred ninety nine thousand nine hundred ninety nine	nine	6
n04	three hundred thirty three thousand three hundred thirty three	three	6
n05	six hundred sixty six million six hundred sixty six thousand six hundred sixty six	six	9
n06	eight hundred eighty eight thousand eight hundred eighty eight	eight	6
n07	five hundred fifty five	five	3
n08	four hundred forty four	four	3
n09	two hundred twenty two million two hundred twenty two thousand two hundred twenty two	two	9
n10	one hundred eleven thousand one hundred eleven	one	5

`src/common/score_counts.py`, so a model's output can be counted in the same normalised token space as every other family; a numeral longer than 12 digits (i.e. a looping model's runaway output, which Whisper can render as an absurdly long integer) is deliberately *not* run through place-value expansion and is instead read off digit-by-digit, on the grounds that place-value names are undefined past *billion* and a digit-wise reading is the semantically correct interpretation of what a looping decoder actually spoke.

3.4 Sentence repetition and tongue twisters

`sentence_rep` repeats one of four short sentences $k \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ times (Table 4); the scored target is a single content word within the sentence. `twisters` instead repeat a whole tongue-twister sentence $k \in \{1, 2, 4\}$ times (Table 5); because several classic twisters already repeat a word or a phoneme cluster *within one copy* of the sentence, the ground-truth count is expected_count = per-copy occurrences $\times$ k, not k itself. Two twisters (`w6`, `w7`) are alliterative rather than word-repetitive – their nominal "target" is a short substring (`"s"`, `"fr"`) that occurs in many unrelated words, so the source sets their per-copy occurrence count to 0 rather than attempting a substring count that would be semantically meaningless; these two items therefore always carry `expected_count`= 0 and exercise phonological/alliterative rendering stress rather than the counting metric.

Table 4: Sentence-repetition stimuli (`SENTENCES`, `data/stimuli/make_stimuli.py`). $k \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ for every row.

ID	Sentence	Target
s1	The bell rang twice.	bell
s2	He counted the stones.	stones
s3	Rain fell on the roof.	rain
s4	The door stayed open.	door

Table 5: Tongue-twister stimuli (`TWISTERS`, `data/stimuli/make_stimuli.py`). "Per-copy" is the number of times the target occurs within a single copy of the sentence; `expected_count` = per-copy $\times k$ for $k \in \{1, 2, 4\}$.

ID	Sentence	Target	Per-copy
w1	She sells sea shells by the sea shore.	sea	2
w2	Peter Piper picked a peck of pickled peppers.	peck	1
w3	How much wood would a woodchuck chuck.	chuck	1
w4	Red lorry yellow lorry red lorry yellow lorry.	lorry	4
w5	The sixth sick sheikh's sixth sheep is sick.	sixth	2
w6	Six slick slim sycamore saplings.	s (alliteration only)	0
w7	Fresh French fried fly fritters.	fr (alliteration only)	0
w8	Truly rural truly rural truly rural.	rural	3

3.5 The matched-control construction

Every `word_rep` item at $k \geq 2$ has a corresponding `control_word` item ($k = 1$ is defined to be its own control, since there is no repetition to remove): the same carrier prefix and suffix, the same word count, with the k copies of the target replaced by k *distinct* words drawn from that template's filler pool (cycled and skipped past any accidental collision with the target itself, so the target word is guaranteed never to appear in its own control). This is the construction underlying the paper's central dissociation test (Section 3.1 of the main paper, Figure 1(a)): the control holds carrier syntax and word count fixed while removing exactly one variable, periodicity, so that a difference in outcome between the repeated item and its control cannot be explained by sequence length, generation duration, or the amount of text the model has to render.

3.6 The `boundary_units` field

`word_rep` and `sentence_rep` items carry `boundary_units`: the ordered list of the k words (all copies of the target, for a repeated item) whose text positions delimit the k repetition boundaries. `control_word` items carry the analogous list of the k distinct fillers actually substituted (duplicated under the key `control_units` for the separate purpose of scoring: was each of the k fillers rendered at all). This field exists so that the boundary-location procedure of `src/common/boundaries.py` (`unit_columns`, Section 5) is handed an explicit, model-agnostic list of search targets rather than having to special-case repeated versus control text – both item types get their boundaries located by the identical left-to-right, non-rewinding string-search procedure, over the identical field, which is what makes the fitted decay rates of repeated and control items directly comparable and not confounded by using two different localisation methods for the two arms of the comparison. `numbers` and `twisters` items carry no `boundary_units` field at all: numbers have no single repeating word-level unit (the repetition is a digit-word recurring at different magnitude positions), and twisters repeat a whole sentence rather than a word, so neither family has a boundary structure at the granularity this field is designed for.

3.7 The instrumented subset

An item is marked `instrumented=true` (and therefore receives the teacher-forced hidden-state/attention capture pass described in Section 6) exactly when its family is one of {`word_rep`, `sentence_rep`, `control_word`} and $k \geq 2$. The source comment gives the rationale directly: "Twisters have no single repeating unit at $k = 1$, numbers are short; both are behavioural-only."

$k \geq 2$ is required because at $k = 1$ there is by definition no repetition boundary to instrument. This yields 54 (`word_rep`, $k \geq 2$)+54 (`control_word`, all $k \geq 2$ by construction)+28 (`sentence_rep`, $k \geq 2$) = 136 instrumented items, matching the count written by `make_stimuli.py` at generation time and the `NInstrumented` macro emitted by `analysis/make_numbers.py`.

4 Measurement protocol in full

Superseded as the paper's headline instrument (Section 9). Everything in this section documents the measurement pipeline as it stood through Sections 1–8: Count A is Whisper large-v3, and correctness is decided by exact match against the requested count. A dedicated audit, described in full in Section 9.1, later established that Whisper's decoder is itself autoregressive with a language-model prior and systematically undercounts genuinely correct periodic speech – by construction the worst possible property for an instrument scoring a study of repetition-suppression dynamics. The main paper's headline behavioural metric is accordingly not what this section describes: it is a CTC recogniser with no decoder and no language model, scored by *relative count error* rather than exact match (Section 9.3). This section nonetheless remains the accurate reference for everything in it that does not depend on which recogniser produced the transcript: Count B's duration-fitting procedure, the outcome taxonomy's logic (all eight labels below apply verbatim under either judge, since `classify` consumes a generic count and never inspects which recogniser produced it), the audio-level degeneracy detectors, and the template bootstrap are unchanged in their machinery by the judge swap – though Count B's fitted constants are recomputed separately per judge, since the fit is calibrated on whichever judge's low-k points are being scored (exactly where the audit finds both recognisers reliable, so the two fits agree closely in practice). What does not survive unchanged is any specific number computed from Count A as Whisper, or any `correct` label built by exact match against it; both recur through Section 8 and are flagged individually where they do. `score_counts.py`'s `--judge {whisper,ctc}` flag switches only the transcript source, reusing this section's audio-level degeneracy flags across both judges rather than recomputing them (Section 9.3).

4.1 The conjunctive correctness criterion

The central behavioural question – given text asking for k repetitions, how many did the model actually produce – is answered by reconciling two independent, differently-biased estimators, implemented in `src/common/score_counts.py`.

Count A (transcript-based). The generated audio is transcribed with Whisper large-v3 (`src/common/asr_transcribe.py`), the transcript is normalised (lower-cased, punctuation stripped except commas needed for numeral parsing, numerals expanded back to number words via `expand_number_token`), and `count_occurrences` counts exact whole-token matches of the target unit (or, for multi-word targets, greedy non-overlapping subsequence matches). Count A is accurate when Whisper faithfully renders every repetition, which holds reliably at low k and *unreliably at high k* – Whisper is itself an autoregressive model and is prone to de-duplicating or collapsing long runs of near-identical speech, exactly the failure mode under study, so it cannot be assumed neutral at the k values that matter most.

Count B (duration-based). Per (model, family, template) cell, a linear fit duration $= a + b \cdot k$ is estimated on the *trusted low-k regime*, defined *before* looking at any high-k behaviour so the calibration cannot be contaminated by the effect being measured: items with $k \leq 3$, Count A exactly equal to the requested k, and audio duration > 0.2 s (`fit_duration_models`). The fit falls back from (model, family, template) to (model, family) to (model) when a cell has too few trusted

points, and is discarded if the fitted slope b is not meaningfully positive. Inverting the fit at a given observed duration gives Count B, a count that is *immune* to ASR de-duplication (it never looks at the transcript) but is correspondingly *blind* to a model that babbles for approximately the right length without actually saying the right thing.

Because the two estimators fail in different, largely uncorrelated directions, **correctness is decided conjunctively**, not by picking one estimator or averaging them: an item is labelled `correct` only if Count A exactly equals the expected count *and* the observed duration falls within a tolerance band of the duration the calibrated fit predicts for that expected count (duration_ratio $\in [0.70, 1.45]$, constants `DUR_LO`, `DUR_HI`). The band is wide enough to absorb ordinary prosodic speech-rate variation (a model reading the same words slightly faster or slower is not a counting failure) while still catching the two directions of genuine failure: a transcript that Whisper collapsed but whose audio ran much longer than it should have (loop), and a transcript that is short and whose audio is also short (truncation). Neither estimator is ever used alone to *assert* a count in a disagreement case – the reconciliation only ever *certifies* correctness when both streams of evidence agree; a disagreement produces an `agree=False` audit flag ($|\text{Count A} - \text{Count B}| > \max(1, 0.15 \cdot \text{expected\_count})$) rather than a synthesised number, so that a wrong count can never silently enter the behavioural CSV disguised as ground truth.

4.2 The outcome taxonomy

`classify` in `score_counts.py` assigns exactly one of eight outcome labels, checked in the following fixed order (audio-level degeneracy first, since noise or silence of coincidentally the right duration would otherwise slip through the duration test):

1. `empty` – duration < 0.25 s or RMS $< 10^{-3}$: essentially no audio was produced.
2. `degenerate` – spectral flatness > 0.35 (noise/babble rather than speech) or an empty ASR transcript.
3. `correct` – Count A equals the expected count *and* the duration ratio is in-band (the conjunctive criterion above).
4. `loop` – generation hit the model's token cap, *or* the duration ratio exceeds 1.6: the model kept going well past where it should have stopped.
5. `overcount` – Count A exceeds the expected count (and the item was not already classified as a loop).
6. `truncation` – the duration ratio is below `DUR_LO`: the model stopped early.
7. `undercount` – Count A is below the expected count (and the item was not already classified as a truncation).
8. `miscount` – the residual fallback: none of the above applied, e.g. the transcript count matches but the duration ratio is out of band without meeting either the loop or the truncation threshold ("right count, implausible pacing").

The main paper's Section 2 describes this taxonomy at the six-way level a figure caption can carry – {`correct`, `undercount`, `truncation`, `overcount`, `loop`, `degenerate`} – which folds `empty` into `degenerate` conceptually (both are audio-level failures rather than counting failures) and omits the low-frequency `miscount` residual bucket; the eight-way taxonomy above is the one actually implemented and emitted into `data/results/behavioural.csv`'s `outcome` column.

4.3 Audio degeneracy detectors

Computed by `audio_flags` in `src/common/asr_transcribe.py`, independent of the ASR transcript, because a TTS failure does not always manifest as wrong *words* – it can produce noise, a stuck vowel, or silence, none of which a transcript-only metric would catch:

- **RMS** – root-mean-square amplitude of the waveform; near-zero flags near-silence.
- **Voiced fraction / trailing silence** – a smoothed energy envelope (frame length sr/50) is thresholded at 2% of its peak to flag voiced frames; `trailing_silence_s` is the time from the last voiced frame to the end of the clip, distinguishing an abrupt truncation (long trailing silence relative to content) from a clean stop.
- **Spectral flatness** – `librosa`'s spectral flatness measure, near 1 for noise-like spectra and near 0 for tonal/harmonic (speech-like) spectra; used as the `degenerate` threshold above.
- **Loop autocorrelation** – the log-mel envelope (32 mel bands, hop length 256, mean over frequency) is autocorrelated with itself; a strong peak (`loop_ac_peak`) at a lag beyond 0.4 s (`min_lag`) indicates the model is re-rendering materially the same acoustic content periodically – an exact-loop signature distinct from, and complementary to, the duration-ratio-based loop detection in `classify`.

Whisper transcription itself is run with `condition_on_prev_tokens=False`. This is deliberate, not a default left in place: Whisper's own decoder is autoregressive and prone to the same class of repetition lock-in under study, and allowing it to condition on its own previously decoded text would risk the ASR judge's loops contaminating the measurement of the TTS model's loops. Word-level timestamps (`return_timestamps="word"`) are requested for a specific downstream purpose beyond transcription: mapping an audio-domain repetition boundary back to a decoder step index via step $= t_{\text{word}} \times$ token_rate_hz, where `token_rate_hz` is each model's nominal acoustic-token rate from the panel registry (`src/common/registry.py`: 50.0 Hz for the Llasa family, 21.53 Hz for XTTS-v2, 12.5 Hz for Qwen3-TTS). The registry's own docstring notes that nominal rates are cross-checked against the measured n_{tokens}/duration at smoke-test time, and it is the measured value that downstream analysis actually uses.

4.4 Effective rank, spectral slope, and Kirchhoff index

Computed by `spectral_proxies` in `analysis/state_dynamics.py` on a trajectory $h \in \mathbb{R}^{T\times d}$ of hidden states at one probe layer (mean-centred over time before the decomposition; trajectories longer than `MAX_SVD_STEPS`= 512 steps are uniformly subsampled first, since these are properties of the *shape* of the spectrum, which uniform subsampling preserves, and the alternative is an $O(Td^2)$ SVD per layer per item; trajectories shorter than 24 steps are skipped). Let $\sigma_1 \geq \sigma_2 \geq \cdots$ be the singular values of the centred trajectory matrix and $p_i = \sigma_i / \sum_j \sigma_j$.

- **Effective rank** $\mathcal{N}_{\text{eff}} = \exp\big(-\sum_i p_i \log p_i\big)$: the exponentiated Shannon entropy of the normalised singular-value spectrum, i.e. a participation-ratio-style count of how many singular directions are actually being used, smoothly interpolating between 1 (all variance in one direction) and the nominal matrix rank (perfectly flat spectrum). This is the statistic behind the capacity-gain measurement of Section 8.3.

- **Spectral slope** α: the negative slope of $\log \sigma_i$ regressed against $\log(i+1)$ over the mid-tail index window $i \in [2, \max(6, 0.8 \cdot \text{rank})]$ – excluding the first couple of dominant components and the extreme tail – i.e. the exponent of the spectrum's power-law decay.
- **Kirchhoff index** $\log_{10} Kf = \log_{10}\big(\sum_i 1/\lambda_i\big)$ with $\lambda_i = \sigma_i^2$, floored at $\lambda_i > 10^{-10}\lambda_{\max}$ for numerical stability: the standard graph-theoretic Kirchhoff index (sum of inverse Laplacian eigenvalues, a resistance-distance-based measure of global connectivity) applied to the covariance spectrum of the trajectory, carried over unchanged from the companion Whisper study [**?**] so its Table 2 and the present study's Table 2 are directly comparable.

4.5 The template bootstrap

Confidence intervals throughout `analysis/capacity.py` are resampled over *stimulus templates*, not over items, and the rationale is stated directly in the source: items sharing a template (e.g. every k-value built from carrier `t1`, "The dog was … big and it ran across the field.") are not independent draws, since they share carrier syntax and length structure; bootstrapping over raw items would understate the true variance. Two statistics are computed this way.

Capacity gain (`gain`): for a given family (repeated or control) and model, $\mathrm{d}\mathcal{N}_{\text{eff}}/\mathrm{d}\log k$ is the ordinary-least-squares slope of $\mathcal{N}_{\text{eff}}$ against $\log k$ at deep probe layers (`layer_frac`> 0.6), fit on items with $k \geq 2$, requiring at least four distinct k values and eight points to attempt a fit at all. The 95% interval is built by resampling the set of templates with replacement 2000 times, refitting the slope on the union of each resample's items every time. A "saturation" value is also reported: the median $\mathcal{N}_{\text{eff}}$ over items in the top half of the observed k range, as a plateau estimate.

Paired gap (`paired_gap`): for every (template, k) cell with both a repeated-item median and a control-item median available, the paired difference (control minus repeated) is computed; because the stimuli were built as matched pairs, this differencing cancels template identity and k exactly, making it the estimator with the least nuisance variance among those available. The 95% interval bootstraps the *mean* of the vector of paired differences (2000 resamples with replacement), and `frac_pos` reports the fraction of paired differences that are positive, i.e. the fraction of (template, k) cells where the control gained strictly more distinguishable states than its matched repeated twin.

5 The negative result, in detail

This section expands the main paper's Section 4, which measures the theorem's premise and rejects it. We consider it a real contribution of the project, not a discarded attempt, and document it at the level of detail a reviewer wanting to check the diagnosis would need. The relevant code is `src/common/boundaries.py` (explicitly marked, in its own module docstring, as *"SUPERSEDED as the paper's primary measurement"*) and `src/common/dispersion.py` (the estimator that replaced it, marked *"the primary state measurement"*).

5.1 What the boundary-difference estimator was

Theorem A is stated about the sequence of *repetition-boundary states* $s_0, s_1, s_2, \ldots$ – the decoder's hidden state exactly at the moment each successive copy of the repeated phrase has been rendered. The most direct way to test the contraction premise is therefore to locate those states in a recorded generation trajectory and difference them. The estimator built to do this, in order:

1. **Locate the k ground-truth text columns.** For a repeated item with target unit u (or a control with k distinct fillers), `unit_columns` walks the model's own tokenized, transformed prompt text left to right with a non-rewinding regular-expression search (`\bword\b`, case-insensitive), using the model-specific `OffsetTokenizer` (`src/common/offset_tok.py`) so that each family's own string transform is respected – notably XTTS-v2, which lower-cases and replaces every space with a literal `[SPACE]` token before tokenizing, so column indices must be (and are) computed on the transformed string, not the raw one. This gives k exact text-span column indices, one per occurrence, by construction in the same left-to-right order for a repeated item and its matched control.

2. **Track an attention centroid.** At the deepest instrumented probe layer, `read_head` computes, for every generation step t, the row-normalised text-attention centroid $c_t = \sum_j j\, p(t, j)$ (a 5-tap moving average is applied for smoothing). A centroid rather than an argmax is used deliberately: the source comment notes that when the k spans are near-identical, their attention columns are near-identical too, so a per-column argmax collapses to the same step for every occurrence, whereas a centroid degrades gracefully and still moves even when no single column dominates.

3. **Match columns to steps monotonically.** `boundary_steps_from_attention` rescales the k target columns onto the empirical 2nd–98th percentile range the centroid trace actually traverses (a centroid, being a weighted mean, is pulled toward the middle of the attended mass and so compresses relative to the raw column range), then walks forward through the centroid trace once, for each rescaled target in turn finding the first step at which the trace first reaches or exceeds it. This is monotone by construction – it can only ever move forward through the generation.

4. **Difference and fit.** The resulting k boundary steps index into the recorded hidden-state trajectory, giving $d_m = \|h(t_{m+1}) - h(t_m)\|$ (`boundary_distances`), and `fit_geometric` regresses $\log d_m = a + m \log q$ by least squares to recover $\hat{q}_{\text{bnd}}$ and an R^2.

5.2 Why it was the natural first choice

This construction is fully self-contained: it uses only the model's own recorded attention and the tokenizer's exact character offsets, with no external ASR pass, no separate alignment model, and no assumption that speech is rendered at a constant tempo. Unlike the estimator that replaced it (Section 5.5), it operationalises Theorem A's boundary states s_m *literally*, at the specific generation step where occurrence m is actually believed to be rendered, rather than approximating the theorem's claim through a proxy quantity. If the model's attention tracked position the way a monotone alignment mechanism would, this would be the right tool, and it is kept in the codebase (rather than deleted) precisely because it remains the natural estimator for any future model that does have a genuinely monotone text read-head.

5.3 The measurement that killed it

The estimator's soundness rests on one load-bearing assumption: that the attention centroid c_t moves *monotonically forward* through the text as generation proceeds, i.e. that it is tracking position, not merely fluctuating around some fixed point of mass. This is directly checkable by computing, over consecutive generation steps, the fraction of steps at which the centroid actually advances ($c_{t+1} > c_t$). A genuine left-to-right sweep would produce a fraction close to 1; a process with no net directional drift – a random walk – would produce a fraction close to 1/2.

The measured value, reported in the project's running notes (`findings.md`) and in the docstrings of both `dispersion.py` and `boundaries.py`, is that **the deep-layer attention centroid advances on approximately** 51% **of generation steps** – statistically indistinguishable from an unbiased random walk, and nowhere near the sweeping behaviour the estimator needs. (We note for precision that this figure is recorded as an empirical summary in the project's notes and source comments rather than reproduced by a single standalone script in the current snapshot of `analysis/`; it is the stated empirical justification, cited three times independently across the codebase, for abandoning the estimator, and it is corroborated by the concrete symptom described next, which *is* directly reproducible from committed data.)

That concrete symptom is visible in `data/results/pooled_q.csv` (produced by `analysis/pooled_q.py`, which pools boundary-distance curves across items within a (model, family) cell and regresses $\log(d_m/d_0)$ on m through the origin, bootstrapping the item set for a confidence interval). Under the boundary/attention method, the pooled fit for Llasa-1B recovers $\hat{q}$ statistically indistinguishable from 1 in every instrumented family – `control_word`: $\hat{q} \approx 1.00$ (95% CI $[0.99, 1.02]$); `sentence_rep`: $\hat{q} \approx 1.03$ (95% CI $[1.00, 1.12]$); `word_rep`: $\hat{q} \approx 0.98$ (95% CI $[0.94, 1.02]$) – i.e. *no measurable contraction at all*, for a model whose boundary-free dispersion estimate (Section 5.5) recovers $\hat{q}$ clearly below 1 for the same generations. This is a snapshot from the instrumented panel available at the time of writing and individual decimal values will move as more of the panel finishes running; the qualitative pattern – boundary-method $\hat{q} \approx 1$ versus dispersion-method $\hat{q} < 1$, on the identical underlying trajectories – is the diagnosis, not the specific digits.

5.4 Why the failure is self-defeating by construction

The failure is not a matter of insufficient smoothing or a fixable implementation bug; it is a direct structural corollary of Lemma B (Section 1.2). `softmax_dist_le` proves that the per-occurrence attention-weight differences shrink as $O(1/k)$, and `entropy_ge` proves the profile's entropy floor grows as $\log k - \delta$: as the repeated block grows, the attention distribution the estimator's centroid is computed from becomes provably closer to uniform. A weighted-mean centroid computed over an increasingly uniform distribution carries, by the same argument, increasingly little information about which position is truly being rendered – in the limit its expected value converges to the midpoint of the attended span regardless of which occurrence is actually current. The estimator therefore degrades precisely as k grows, which is exactly the regime in which a clean boundary estimate would matter most for testing the theorem. This is why the failure could not have been patched by using more attention heads, denoising the centroid trace, or any comparable engineering fix: the very effect the estimator was built to exploit (identifiable, position-informative attention) is the effect Lemma B proves vanishes as the block it is trying to measure grows.

5.5 The replacement: boundary-free dispersion

Theorem A(ii) (`exists_indistinguishable`, Section 1) licenses a different, boundary-free operationalisation: it says the states visited late in a repeated generation become mutually indistinguishable as a *set*, which is a statement about the *spread* of the visited states and needs no per-occurrence boundary labels at all. `src/common/dispersion.py` implements this directly: the trajectory is tiled into `N_WINDOWS`= 8 equal-length windows (each requiring at least `MIN_WINDOW`= 8 steps to support an estimate); within each window, states are ℓ_2-normalised (direction, not residual-stream magnitude, is what is compared – magnitude is close to constant along these trajectories in any case, roughly 130 for Llasa-1B, so normalising changes little but makes the resulting quantity comparable

across models with different activation scales) and the mean pairwise chordal distance $\sqrt{2-2\cos\theta}$ among normalised states in the window is computed; $\log\sigma(\text{window})$ is then fit linearly against window index, and the fitted per-window rate is converted to a per-repetition $\hat{q}$ by the ratio of window length to average repetition length (n_{windows}/k). This sidesteps the localisation problem entirely, at the cost of measuring a slightly different (but, per Theorem A(ii), equally licensed) quantity: the spread of the states visited within a window of the trajectory, rather than the literal distance between two specific, individually-identified boundary states. `analysis/state_dynamics.py` computes both estimators side by side for every instrumented item (columns `q_hat` for dispersion and `q_bnd` for the boundary method in `data/results/state.csv`) and records which localisation method, `attention` or the uniform-tempo fallback `uniform`, produced each row's boundaries, precisely so this substitution is auditable rather than silent.

5.6 A third attempt: the exact Jacobian, gated before it touches a model

Neither of the two estimators above measures the theorem's contraction factor directly, and `analysis/jacobian_q.py` (the source of the main paper's Section 4 numbers) is not a retry of either: it differentiates the decoder's own boundary-to-boundary map exactly, rather than inferring it from attention or approximating it from within-window spread.

What is differentiated. In a KV-cached autoregressive transformer, the theorem's state s_m has no single-layer realisation: information reaches a later position only by rising through depth, so the map from depth ℓ at position t to depth ℓ at $t' > t$ is *exactly* zero, not small. The only well-posed object is the whole per-repetition sub-stack, $S_m = (h_\ell(t) : \ell = 0..L-1,\, t \in W_m)$, the residual stream entering every layer at every token position of repetition m. Teacher-forcing the generated tokens fixes everything outside $W_m \cup W_{m+1}$, making $S_{m+1} = F(S_m)$ a deterministic, differentiable endomorphism. Because the restriction to depths $\geq \ell$ is exactly closed under the same argument, $q_\ell := \sigma_{\max}$ of F restricted to that sub-stack is itself a legitimate contraction factor, non-increasing in ℓ; q_0 (whole stack) is the headline and the sweep over ℓ is reported alongside. Perturbations are measured in units of each layer's own RMS activation (matching the RMSNorm that K, V are actually computed from), so F stays an endomorphism of a fixed metric space across layers of very different activation scale. Because the speech-token span of each repetition is not directly observed, boundaries are *constructed* – a uniform partition of the rendered trajectory into k equal spans – rather than located by an attention read-head, and the conclusion is required to survive a sweep over which width counts as "one repetition" (lags `half_tau`, `tau`, `two_tau`, `tau_rendered`, the last keyed to the CTC-judged rendered count) rather than resting on the partition being exactly right.

Two biases, declared before the model was run. Teacher-forcing deletes the token channel – in real generation, perturbing S_m changes which tokens S_{m+1} renders from, an additional path this measurement cannot see – so the reported q is a *lower* bound on the true per-repetition Lipschitz constant, biased *toward* the contraction premise: a measured $q \geq 1$ is decisive against it, and a measured $q < 1$ would be weaker evidence for it than it looks. Working against that, perturbing each layer's cache entries independently admits a larger set of directions than the network's own forward pass can reach at a boundary, which over-states $\sigma_{\max}$ and biases *against* the premise. The two are not claimed to cancel.

Gate 0, run identically on every checkpoint before it was pointed at that checkpoint's real generations. S1 is a synthetic positive control – the identical power-iteration code path applied to $y = A\tanh(x) + b$, whose Jacobian at $x = 0$ is A – and is independent of which model is being measured, so it is bit-identical across all five runs, as shown. S2 is causality: the same estimator run backwards in time (state at $W_{m+1} \to$ state at W_m) must return exactly 0, since a causal transformer cannot move information back in time; any nonzero value means the hooks

leak. S3 checks exact JVPs against finite differences over the step range where the map is linear ($h \in \{0.3, 1.0, 3.0\}$; smaller steps are float32 roundoff divided by h and are reported, not judged).

checkpoint	S1 rel. err.	S2 σ_{fwd}	S2 σ_{bwd}	S3 max rel. err. (judged)
Llasa-1B	3.7×10^{-4}	43.89	0.000000	5.8×10^{-4}
Llasa-3B	3.7×10^{-4}	85.33	0.000000	1.5×10^{-3}
Llasa-8B	3.7×10^{-4}	101.10	0.000000	5.9×10^{-4}
Qwen3-TTS-0.6B	3.7×10^{-4}	40.14	0.000000	8.9×10^{-4}
Qwen3-TTS-1.7B	3.7×10^{-4}	32.99	0.000000	8.5×10^{-4}

All five pass S1–S3 well inside their gates (5% for S1 and S3, exactly 0 for S2). σ_{fwd} is not itself a report of q – it is the S2 scale reference that first showed a value far above 1 and motivated adding the typical-direction gain as a descriptive companion statistic (Section 5.11).

Qwen needed a fourth gate. Qwen3-TTS's talker never materialises a flat token stream to teacher-force: each decode step's input is a sum of sixteen RVQ-codebook embeddings plus a text term, and the codec ids themselves are never saved by generation. The inputs the model actually used are instead captured by a forward pre-hook at generation time (`analysis/qwen_codec_dump.py`) and required to reproduce incremental generation to cosine $\geq$ 0.999 before any Jacobian is computed from them (S4, `extra_selftests` in `analysis/jacobian_q.py`).

checkpoint	n items	cosine (min / median)	rel. err. (max)
Qwen3-TTS-0.6B	88	0.99920 / 0.99980	0.0399
Qwen3-TTS-1.7B	90	0.99963 / 0.99987	0.0274

Both pass ($\text{cosine}_{\min} > 0.999$ and rel. $\text{err.}_{\max} < 0.05$). The dump script additionally counts, per item, whether its own re-sampled `n_speech_tokens` agrees with the generation ledger's; per the project's committed record this agreement is 88/88 on Qwen3-TTS-0.6B and 90/90 on Qwen3-TTS-1.7B. That specific pairwise count is not itself a field in `jacobian_q_qwen06b.json` / `jacobian_q_qwen17b.json` – only the cosine and relative-error summary derived from it is – so, in the same spirit as the attention-drift figure in Section 5.5, it is flagged here as sourced from the dump-time verification and the commit record rather than reproduced by a script in the current results snapshot.

5.7 The five-checkpoint panel

Having passed its gates, the estimator is run at its headline setting (lag τ, whole stack, RMS metric) on every checkpoint. `jacobian_q.json` (Llasa-1B) and its four siblings (`jacobian_q_{llasa3b,llasa8b,qwen06b,qwen17b}.json`) together cover 346 items (repeated plus length-matched control) and 15,243 attempted measurements, of which 14,796 are valid and reported (the 447 excluded are addressed in Section 5.10, not silently dropped). `jacobian_q_panel.json` is the cross-checkpoint summary.

checkpoint	layers	n pairs	q repeated [95% CI]	q control [95% CI]	paired p
Llasa-1B	16	29	38.1 [32.9, 51.4]	31.8 [27.3, 39.8]	0.017
Llasa-3B	28	26	118.8 [101.9, 156.5]	86.4 [77.2, 137.3]	0.269
Llasa-8B	32	29	347.7 [258.1, 393.5]	185.4 [142.1, 236.5]	0.256
Qwen3-TTS-0.6B	28	44	21.0 [19.7, 24.2]	21.9 [20.6, 23.2]	0.931
Qwen3-TTS-1.7B	28	45	24.7 [23.1, 25.7]	23.6 [21.4, 25.1]	0.177

$q < 1$ in 0 of the 346 items on either arm – `frac_items_q_below_1` is exactly 0.0 for both repeated and control on every checkpoint in `jacobian_q_panel.json`. Pooling across checkpoints, q runs 21.0–347.7. The single most favourable cell in the entire panel (the smallest bootstrap lower confidence bound over all lags, sub-stacks and both arms of any checkpoint) is Qwen3-TTS-0.6B's 3.35, rounding to the 3.3 the main paper reports as the floor. `verdict` is `"PREMISE DEAD"` on every one of the five, and `jacobian_q_panel.json`'s own `answer` field states the pooled conclusion in one sentence: "the refutation generalises: every checkpoint measured, across 2 architecture families, has q far above 1 on BOTH the repeated and the length-matched control arm, so the contraction premise was never a live description of any of these decoders."

5.8 Both arms are expansive, in all sixteen cells, on all five checkpoints

The paired p column above is the repeated-versus-control contrast at the headline cell only. The stronger and more direct fact is per-arm: at every one of the sixteen (lag × sub-stack) cells the estimator visits, on every checkpoint, on *both* arms, the bootstrap CI for q lies entirely at or above 1 (`expansive_in_every_cell: true`, `contracting_in_some_cell: false` in every checkpoint's `summary.premise_by_arm`), and the fraction of items with $q < 1$ is 0.0 in every cell of every checkpoint on both arms.

checkpoint	repeated: range over 16 cells	control: range over 16 cells
Llasa-1B	[3.90, 78.10]	[3.50, 46.61]
Llasa-3B	[9.33, 182.80]	[7.60, 148.28]
Llasa-8B	[11.43, 499.13]	[12.28, 329.33]
Qwen3-TTS-0.6B	[3.35, 29.47]	[3.41, 28.23]
Qwen3-TTS-1.7B	[3.45, 37.83]	[3.35, 31.18]

The repeated-versus-control contrast at the headline cell is significant in the *wrong* direction on Llasa-1B ($p = 0.017$: repetition makes the map *more* expansive than its own length-matched control, not less) and null on the other four ($p = 0.269$, 0.256, 0.931, 0.177 – the range the main paper quotes as 0.18–0.93). `premise_by_arm.statement` in every one of the five JSONs states the honest reading in identical language: "the premise fails on BOTH arms: the length-matched control, which this decoder counts correctly, is also expansive in every cell. Contraction was never a live description of this decoder for any input – periodicity does not break it, it was never there to break." This is a claim about measurement, not about prose: it is what `expansive_in_every_cell` and `contracting_in_some_cell` record for every arm of every checkpoint, not an inference drawn on top of the numbers.

5.9 Two facts that sharpen the refutation past replication

q grows with scale inside Llasa. The three checkpoints of one architecture family, same estimator, same headline cell:

checkpoint	q (headline)
Llasa-1B	38.1
Llasa-3B	118.8
Llasa-8B	347.7

A "contraction mechanism specific to the larger checkpoints" is excluded in the direction that would have rescued the theory: the bigger the checkpoint, the further from contraction, not the closer.

Qwen3-TTS-1.7B counts correctly and has $q = 24.7$. It is the one checkpoint in this panel with zero median count error (the main paper's `ErrBestModel` result), and its per-repetition map is exactly as expansive as everyone else's – 24.7 $[23.1, 25.7]$, indistinguishable in kind from Llasa-1B's 38.1 or Qwen3-TTS-0.6B's 21.0, both of which count imperfectly. A decoder that renders the count perfectly has an equally expansive map to decoders that do not: whatever separates counting from miscounting in this panel, it is not q.

5.10 Reported against interest

Two things about how these five runs were produced would flatter the result if left unstated, so they are stated.

Llasa-8B skipped 22% of its measurements. Of 1,677 attempted measurements, 365 (21.8%, rounding to the 22% the main paper reports) were not taken:

checkpoint	read window (tok.)	attempted	skipped	reasons
Llasa-1B	2000	2736	12 (0.4%)	exceeds MAX_TOKENS (8), OOM (4)
Llasa-3B	1100	2286	70 (3.1%)	exceeds MAX_TOKENS (70)
Llasa-8B	700	1677	365 (21.8%)	exceeds MAX_TOKENS (236), OOM (129)
Qwen3-TTS-0.6B	2000	4224	0 (0.0%)	—
Qwen3-TTS-1.7B	2000	4320	0 (0.0%)	—

Llasa-8B ran at `MAX_TOKENS=700` (a smaller read window than the other four, on the card it happened to be scheduled on) plus a share of out-of-memory skips at the same card; between them these account for all 365 skipped anchors, individually recorded (`"skipped": "exceeds MAX_TOKENS"` or `"skipped": "OOM"`) in `jacobian_q_llasa8b .json`'s `records`, not aggregated away. Truncating at the read window is exact on a causal model – a shorter window changes which anchors can be formed, not the value the estimator returns for the anchors it does form – so every surviving $\sigma_{\max}$ is unchanged and only the anchor set shrinks. The main paper's framing ("truncating at the read window is exact... so surviving values are unchanged and only the anchor set shrinks") covers the majority reason (`exceeds MAX_TOKENS`, 236 of 365); the remaining 129 are plain out-of-memory skips on the same card, a capacity limit rather than a windowing choice, and are reported here as a distinct reason rather than folded into the read-window story.

Llasa-3B ran before a `MIN_TAU` patch. The narrowest window the estimator will call "one repetition" was originally 8 decoder frames, a value that is a rate heuristic, not a principle: at Llasa's 50 Hz codec that is $8/50 = 0.16$ s, but at Qwen3-TTS's 12.5 Hz codec the same 8 frames is $8/12.5 = 0.64$ s (≈ 0.6 s) – long enough to have silently discarded nearly every Qwen item, the checkpoint the whole extension from one to five checkpoints was about, for a reason that is about sample rate and not about the decoder. The floor was lowered to `MIN_TAU=3` (still a window, not a single position) before the Qwen runs. Llasa-3B was measured under the old floor, and the patch changes only its `half_tau` lag, and only where $\lfloor \tau_k/2 \rfloor < 8$: exactly two items, `wr_quick_t5_k32` at seeds 0 and 1 ($\tau_k = 15$, so `half_tau` $= 7$ under the current floor, would have been clamped to 8 under the old one). No other checkpoint, lag or item is affected. The bias of the patch runs *toward* the contraction premise, not against it: $\sigma_{\max}$ grows with window width (measured directly on Llasa-1B: 16.2 at a 1-frame window against 39.4 at a 77-frame window, per `analysis/jacobian_q.py`'s own constants), so the narrower floor makes the two affected estimates *smaller*, i.e. more favourable to contraction, not less – and the premise still fails on them.

5.11 Two corroborating measurements

Typical-direction gain. $\sigma_{\max}$ is what the theorem's uniform Lipschitz bound requires and what decides the verdict above; $\mathbb{E}\|Jv\|/\|v\|$ over random unit directions v (a Frobenius-norm probe) is a purely descriptive companion, reported because "$\sigma_{\max} > 1$ while the average direction contracts hard" and "the map expands in every direction" are different mechanistic pictures. At the headline cell:

checkpoint	repeated (median)	control (median)
Llasa-1B	0.152	0.129
Llasa-3B	0.238	0.187
Llasa-8B	0.408	0.285
Qwen3-TTS-0.6B	0.283	0.231
Qwen3-TTS-1.7B	0.229	0.191

Typical-direction gain runs 0.15–0.41 at the headline cell, below 1 in 100% of items on both arms of every checkpoint and every cell (`frac_below_1`= 1.0 throughout `summary.by_cell_typical_gain`), and repeated exceeds control at the headline cell on all five checkpoints. So the typical direction genuinely contracts everywhere it is measured; it is the worst direction, the one the uniform bound is about, that does not, and the verdict above is decided by $\sigma_{\max}$, never by this statistic.

Boundary-distance decay. `summary.boundary_distance_decay` fits $\log(d_m/d_0)$ against m on the same uniformly-partitioned boundaries, independently of the Jacobian estimator:

checkpoint	$\hat{q}$ [95% CI] (word_rep)		R^2	n items
Llasa-1B	0.981	[0.970, 0.989]	0.020	29
Llasa-3B	0.991	[0.986, 0.994]	−0.015	26
Llasa-8B	0.991	[0.981, 0.998]	0.011	29
Qwen3-TTS-0.6B	0.986	[0.982, 0.988]	−0.308	44
Qwen3-TTS-1.7B	0.995	[0.993, 0.996]	−0.123	45

$\hat{q}$ sits in a narrow 0.98–0.99 band on every checkpoint – statistically indistinguishable from 1, the same qualitative finding Section 5 reports for the boundary-based estimator on Llasa-1B – and R^2 runs from −0.31 to +0.02: at no checkpoint does distance from a repetition boundary decay geometrically at all, let alone at a rate matching the Jacobian estimate. Every row is marked `"usable": false` in the source JSON for exactly this reason – not fit for computing a horizon – which is why Section 5.7's headline uses the Jacobian estimator, not this one.

6 Per-model implementation notes

Table 6 reproduces the model panel (`src/common/registry.py`) in full; the notes below detail how instrumentation was obtained for each family and, for Qwen3-TTS, precisely what could not be. The panel is exactly these six checkpoints. The registry also carries a seventh entry, `xtts2norp` – XTTS-v2 with its shipped repetition penalty disabled – which is an ablation of the `xtts2` row, not an independent seventh checkpoint, and is deliberately omitted from Table 6 for that reason; every panel-pooled statistic in the main paper and in this document excludes it by construction (`ABLATIONS = {"xtts2norp"}` in `analysis/make_numbers.py`, `analysis/capacity.py`, `analysis/capacity_confound.py`, and `analysis/unit_invariance.py`). The ablation is documented on its own terms in Section 8.5.

Table 6: Model panel (`PANEL`, `src/common/registry.py`).

Key	HF id	Fam.	Par.	Token rate	Probes	Notes
llasa1b	HKUSTAudio/Llasa-1B	llasa	1B	50.0 Hz	all	Llama-3.2-1B / X-codec2
llasa3b	HKUSTAudio/Llasa-3B	llasa	3B	50.0 Hz	every 2	Llama-3.2-3B, same tokenizer/codec
llasa8b	HKUSTAudio/Llasa-8B	llasa	8B	50.0 Hz	every 4	Llama-3.1-8B, top of scale ladder
xtts2	coqui/XTTS-v2	xtts	0.4B	21.53 Hz	every 3	GPT2-style AR + HiFiGAN
qwen06b	Qwen/Qwen3-TTS-12Hz-0.6B-Base	qwen	0.6B	12.5 Hz	every 3	dual-track multi-codebook LM
qwen17b	Qwen/Qwen3-TTS-12Hz-1.7B-Base	qwen	1.7B	12.5 Hz	every 3	same family as 0.6B

`probe_layer_indices` always force-includes the final layer of whichever density setting is used, since that is the layer that actually feeds the model's own readout (stop-token logit or codec head);

the "every n" densities on the larger checkpoints exist to bound per-item activation storage as depth grows.

6.1 Llasa (1B / 3B / 8B)

`src/models/llasa_gen.py` runs every Llasa checkpoint through the same two-pass design. **Pass 1 (generate)** samples speech tokens autoregressively via `model.generate` with the attention backend set to SDPA (`do_sample=True`, temperature 0.8, top-p 1.0, `max_new_tokens`= 2048 by default); the prompt is built from the model's own chat template with sentinel tokens (`<|TEXT_UNDERSTANDING_START/END|>`, `<|SPEECH_GENERATION_START/END|>`) and `continue_final_message=True` to prime the assistant turn on the speech-start token; generated speech-token ids are extracted from the `<|s_NNN|>` vocabulary pattern and saved as integer arrays, with waveform synthesis deferred to a separate offline pass (`src/models/xcodec2_decode.py`) run in the isolated `xcodec2` environment (Section 7). A `hit_cap` flag records whether generation ran to the token ceiling; per the project's running notes, 21.5% of Llasa-1B generations do so, treated in this study as a signal (runaway generation is common enough to be informative) rather than noise to be tuned away by simply raising the cap.

Pass 2 (instrument) runs only when the seed equals `instrument_seed` (default 0), the stimulus item is marked `instrumented`, and at least one speech token was produced. The *entire realised sequence* (prompt followed by the tokens actually sampled in Pass 1, EOS stripped) is fed through a single teacher-forced forward call with the attention backend switched to `eager` – the only backend that materialises per-layer attention-weight tensors – and `output_hidden_states=True`. Because the model is causal, position t of this forward pass carries exactly the hidden state that *produced* generated token t during the actual sampling pass, so this single clean forward recovers the whole instrumented trajectory without altering the realised token sequence in any way; it is a pure read-out of Pass 1's output, not a re-sampling. `TextAttentionRecorder` attaches forward hooks to a chosen subset of `self_attn` layers (up to four of $\{0, \lfloor L/3 \rfloor, \lfloor 2L/3 \rfloor, L-1\}$ for L total layers); each hook slices the returned `(attn_output, attn_weights)` tuple's weights down to `[B, H, Q, text_lo:text_hi]`, head-averages, and moves to CPU `float16` *inside the hook*, specifically to avoid ever materialising the full $O(T^2 \cdot \text{heads} \cdot \text{layers})$ attention tensor in memory. Entropy and top-1 probability of the next-token distribution are recorded from the same forward pass's logits at no extra cost. `llasa_gen.py`'s own comment states the reason SDPA is used for sampling and eager only for the instrumented pass directly: Llasa under eager attention is roughly 5× slower to sample.

6.2 XTTS-v2

`src/models/xtts_gen.py` generates via `model.gpt.generate` (the GPT2-style autoregressive mel/codec-index decoder), using the sampling defaults shipped in the checkpoint's `config.json` unless overridden on the command line. Those shipped defaults include **`repetition_penalty`**= 5.0 – confirmed directly in the downloaded checkpoint's `config.json` – an aggressive built-in anti-repetition mechanism active during the very sampling pass this study measures. Any residual counting failure XTTS-v2 exhibits is therefore occurring *despite* a penalty actively working against it, which is why the main paper's Discussion (Limitations) frames XTTS-v2 as a *conservative*, not a neutral, member of the panel: if anything, the theory's prediction is being tested on this model under a handicap. The AR decoder's built-in generation ceiling (`model.gpt.max_gen_mel_tokens`) is derived from the shipped config's `model_args.gpt_max_audio_tokens`= 605 minus a small fixed number of reserved special-token positions, giving an effective cap of approximately 602 mel frames

per item; the script's own `--max-new-tokens` override can only shrink this further via `min()`, never raise it.

Instrumentation avoids a second forward pass through the high-level API by hand-reconstructing XTTS's own internal embedding recipe: conditioning latents, text embeddings (token + positional), and mel-token embeddings (token + positional) are concatenated into a single `inputs_embeds` tensor and fed through the *raw* underlying `transformers.GPT2Model` (`model.gpt.gpt`, whose own `wte`/`wpe` embedding tables are deleted by XTTS since embeddings are computed externally). This bypasses `Xtts.inference()`/`synthesize()` specifically because those high-level entry points do not expose `output_hidden_states=True` and additionally run a sentence-splitting/chunking stage that could silently fragment a long repeated-word stimulus into several independent `generate()` calls, defeating the "one contiguous generation" premise the state-dynamics measurement depends on. Attention is forced to `eager` once at load time (`model.gpt.gpt.config._attn_implementation = "eager"`), because SDPA (the library default) silently returns `None` attention weights even when `output_attentions=True` is requested – a failure mode the script's header comment documents explicitly, verified against the installed `coqui-tts` 0.27.5 and `transformers` 4.57.1 sources by file and function name. The same memory-avoidance trick as Llasa is used (`output_attentions=False` at the top-level call, so `GPT2Model` never collects the full attention stack across all layers, while per-layer forward hooks on the probed blocks' `.attn` submodules see the real per-layer weights regardless, since `transformers`' eager attention path computes them unconditionally internally). Vocoding runs in the same process and script as generation (unlike Llasa, which vocodes offline in a separate environment), since XTTS ships its own HiFiGAN decoder with no conflicting pinned dependency.

6.3 Qwen3-TTS (0.6B / 1.7B)

`src/models/qwen_gen.py` instruments a model that is architecturally very different from the other two families. Qwen3-TTS is a dual-track model: an autoregressive "talker" transformer (`Qwen3TTSTalkerForConditionalGeneration`) emits one 12.5 Hz acoustic frame per decode step, where each frame packs 16 RVQ codebook ids – the first sampled directly by the talker's own `codec_head` inside the standard HF generation loop, the remaining 15 filled in afterward by a separate, smaller "code predictor" sub-model conditioned on that step's talker hidden state. Only the first-codebook / talker level is instrumented, which is also the level at which Theorem A's boundary-state argument is naturally stated.

No second forward pass is needed. The mid-level library API, `Qwen3TTSForConditionalGeneration.generate()`, already unconditionally calls the talker's own `.generate(..., output_hidden_states=True, return_dict_in_generate=True)` internally on every call and then discards every hidden-state layer but the last. `TalkerGenerateCapture` monkeypatches the bound `talker.generate` method to stash the full raw `GenerateDecoderOnlyOutput` (and the `trailing_text_hidden` keyword argument) before forwarding the call through completely unchanged – a read-only interception, so sampling and behaviour are byte-for-byte identical to what the public `generate_voice_clone()` API would have produced with or without instrumentation. Per-frame, per-probe-layer hidden states are then pulled out of the captured object after the fact (`extract_trajectory`), with frame j aligned to the decode call whose forward pass predicted it, matching the library's own internal step/frame convention.

Text attention is not obtainable, and this is stated in the code as an architectural fact, not a shortfall of instrumentation effort. In the model's default streaming mode, the embedding fed into the talker at each decode step is the *elementwise sum* of a codec-token

embedding and, for a short prefix of steps, a text-token embedding (`trailing_text_hidden`, added inside the talker's own `forward`). There is consequently no column of the self-attention matrix that corresponds to "the text" the way there is for Llasa's or XTTS's concatenated token streams: attention weight over a fused position cannot be attributed to its text component versus its codec component after the sum, because the sum is not invertible. A `non_streaming_mode=True` flag exists in the library that would give text its own contiguous block of positions and could in principle be probed, but it is not the model's default or tested code path, and using it risks degrading the audio quality the project treats as the non-negotiable baseline requirement, so it is deliberately not used. `text_lo`/`text_hi` are therefore written as literal `null` in this model's metadata records – an explicit "not applicable," never a fabricated or uniform placeholder span standing in for real boundaries – alongside a `trailing_text_len` field recording how many initial decode steps did carry a distinguishable text contribution, offered as a temporal (not spatial) proxy for anyone who wants one. A direct consequence, made explicit here because it is easy to miss: because Qwen3-TTS has no obtainable text attention, it is absent from the boundary-based estimator of Section 5 *by construction* (there is no attention tensor to feed `unit_columns`/`read_head`), and its state-space measurements in this study are dispersion-only.

6.4 The count probe: layers, ridge penalty, and layer selection

The main paper's Method section states that "probe layers, ridge penalties and the layer-selection rule are listed per checkpoint in S6"; this subsection is that listing. The probe itself — ridge regression predicting $\log_2 k$ from mean-pooled hidden states, scored by leave-one-template-out cross-validation — is specified in `analysis/probe_count.py` and tested against a competing account of the deficit in Section 8.4; what follows is the configuration, not a repeat of that test.

Probe layers. The layers a checkpoint's states are captured at are fixed *before* probing, by `probe_layer_indices` in `src/common/registry.py`, from the density code in each checkpoint's `probe_layers` field (Table 6's "Probes" column): `all` keeps every decoder layer, `every`n keeps every n-th, and the final layer is always force-included regardless of n since that is the layer whose state actually feeds the model's own readout. Table 7 gives the resulting layer count P per checkpoint, read directly from the number of per-layer entries in `data/results/probe.json` (the count the probe sweeps over is exactly the count that was captured; there is no separate subsampling at probing time).

Table 7: Probe-layer count per checkpoint. Density code from `src/common/registry.py` (`PANEL`); P (layers actually swept by the probe) from `data/results/probe.json`'s per-layer arrays.

Checkpoint	Density code	P, layers probed
`llasa1b`	`all`	16
`llasa3b`	`every2`	15
`llasa8b`	`every4`	9
`xtts2`	`every3`	11
`qwen06b`	`every3`	10
`qwen17b`	`every3`	10

Ridge penalty. The ridge regression's ℓ_2 penalty is $\alpha = 1.0$ (`ridge_loto`'s `alpha` parameter, `analysis/probe_count.py`), applied identically to the centred features at every layer, for every stimulus family, for every checkpoint. Read literally, the main paper's phrasing — "ridge penalties … listed per checkpoint" — could suggest the penalty itself varies by checkpoint. It does not: $\alpha = 1.0$ is a single fixed constant, never swept or tuned, and there is no per-checkpoint value to list

beyond that constant repeated six times. What genuinely varies per checkpoint is which layers the penalty is applied at (Table 7) and which of those layers the layer-selection rule below picks out.

Layer-selection rule. No layer is fixed in advance across checkpoints or chosen by architectural analogy. For each checkpoint, each stimulus family (`word_rep`, `control_word`) and each window (*early*: mean state over the trajectory's first third; *late*: mean state over its final third), `probe_count.py` fits the ridge probe independently at every one of that checkpoint's P probed layers and reports the layer that maximises leave-one-template-out cross-validated R^2: `best_layer_idx = argmax`$_p$ `r2`(p). The rule is applied separately to the four (family × window) combinations per checkpoint, so the selected layer can and does differ between them — there is no single "the probe layer" per checkpoint, only a best layer per checkpoint per condition. Table 8 lists every selected layer index and its cross-validated R^2, from `data/results/probe.json`; these are the same fits that produce the retention ratios of Table 13, here reported before they are divided into one another.

Table 8: Selected probe layer (`best_layer_idx`, 0-indexed into the P probed layers of Table 7) and its leave-one-template-out R^2, by checkpoint, stimulus family and window. `data/results/probe.json`. n=54 items per cell throughout.

	`word_rep`		`control_word`	
Checkpoint	early	late	early	late
`llasa1b`	L9 (R^2=0.64)	L13 (R^2=0.62)	L7 (R^2=0.65)	L15 (R^2=0.71)
`llasa3b`	L4 (R^2=0.48)	L12 (R^2=0.41)	L5 (R^2=0.68)	L11 (R^2=0.67)
`llasa8b`	L3 (R^2=0.55)	L4 (R^2=0.52)	L4 (R^2=0.51)	L7 (R^2=0.56)
`xtts2`	L6 (R^2=0.25)	L6 (R^2=0.49)	L5 (R^2=0.22)	L0 (R^2=0.62)
`qwen06b`	L2 (R^2=0.32)	L3 (R^2=0.47)	L4 (R^2=0.18)	L4 (R^2=0.31)
`qwen17b`	L1 (R^2=0.62)	L9 (R^2=0.47)	L1 (R^2=0.37)	L9 (R^2=0.41)

The selected layer moves around within a checkpoint (e.g. `xtts2`'s control-word late-window best layer is L0, its earliest probed layer, against L6 for the same checkpoint's repeated-word late window) as well as across checkpoints, which is what a per-condition argmax over a noisy R^2 surface with $n = 54$ items should be expected to do; Section 8.4 reports the retention statistic this table feeds and states plainly where the resulting numbers are, and are not, strong evidence.

7 Reproducibility

7.1 The four conda environments

`env/setup_envs.sh` builds four environments because, per its own header comment, "the dependency constraints are mutually unsatisfiable." Table 9 lists every version pin found in the script and the concrete failure it exists to prevent, drawn from the script's own comments and the README's "Things that will bite you" section.

The `qwen` and `coqui` environments both install torch and torchaudio from the PyTorch `cu130` wheel index (`--index-url https://download.pytorch.org/whl/cu130`) *before* the TTS package itself, per the script's own top-level comment: "install cu130 torch BEFORE the TTS package in each env, else pip silently pulls a default CUDA-12 wheel and the driver/toolkit combo breaks."

Table 9: Conda environments (`env/setup_envs.sh`) and their pins.

Env	Python / core	Pin and the failure it prevents
`base`	py3.13, torch 2.11+cu130	`transformers==4.57.3` (additive install onto the existing base torch); runs the Llasa LMs, Whisper ASR, and all analysis scripts.
`qwen`	py3.12	`qwen-tts` package; torch/torchaudio installed from the `cu130` index *before* `qwen-tts` – installing in the other order lets pip silently pull a default CUDA-12 wheel, breaking the driver/toolkit match.
`coqui`	py3.12	`coqui-tts`; same cu130-before-package ordering. `coqui-tts` breaks on `transformers` 5.x (`isin_mps_friendly` was removed from that version), so this environment is kept on `transformers` 4.57.x.
`xcodec2`	py3.11, torch 2.5 (hard-pinned)	`xcodec2==0.1.5` pins torch 2.5 but leaves `transformers`/`torchao` unpinned, and both have since moved past compatible versions: `transformers` 5.x → `Could not import module 'PreTrainedModel'`; `torchao` $\geq$0.7 → `module 'torch' has no attribute 'int1'`. The working combination, pinned explicitly after the `xcodec2` install, is `transformers==4.46.3`, `tokenizers<0.21`, `torchao==0.6.1`, plus `soundfile`.

7.2 Other pitfalls documented in the README

- **`src/common/tokenizers.py` must never exist.** Running a script from inside `src/common/` puts that directory first on `sys.path`, so a file with that name would shadow the real `tokenizers` package and break `transformers` with a confusing `ImportError`; this is why the project's own tokenizer interface is named `offset_tok.py` instead.
- **The HF `automatic-speech-recognition` pipeline routes audio through `torchcodec`**, which does not load against the installed FFmpeg in this environment; `asr_transcribe.py` drives `WhisperProcessor` and `WhisperForConditionalGeneration` directly instead, avoiding that code path entirely.
- **XTTS replaces spaces with literal `[SPACE]` tokens and lower-cases before tokenizing**, so any text-column index computed for XTTS must be computed on the transformed string (`offset_tok._xtts_prepare`), never on the raw stimulus text.
- **Llasa under eager attention is roughly 5× slower to sample**, which is why `llasa_gen.py` samples under SDPA and switches to eager only for the single instrumented teacher-forced pass.
- **Whisper de-duplicates repeated speech**, so a transcript-only repetition count systematically understates loops; this is the direct motivation for the conjunctive correctness criterion of Section 4. Section 9.1 quantifies this bias directly against ground truth, which is why the paper's headline judge is no longer Whisper (Section 9).

7.3 Exact commands

```
# 0. Environments (once)
bash env/setup_envs.sh all       # base, qwen, coqui, xcodec2
bash scripts/setup_lean.sh       # elan + mathlib cache (CPU, ~15 min)
bash scripts/download_models.sh  # 8 checkpoints, ~74 GB

# 1. Verify the formal artifact
bash scripts/check_lean.sh

# 2. Generate (one model per GPU; resumable by (item_id, seed))
python src/models/llasa_gen.py --model llasa1b --gpu 0 --seeds 0 1 2
# xtts2 needs the `coqui` env and COQUI_TOS_AGREED=1
# qwen06b / qwen17b need the `qwen` env

# 3. Vocode, transcribe, score
bash scripts/run_pipeline.sh <asr_gpu> llasa1b xtts2 ...

# 4. Analyse and build the paper
python analysis/state_dynamics.py --models <models> --out data/results/state.csv
python analysis/capacity.py                        # the central measurement
python analysis/horizon_fit.py                      # behavioural collapse points
python analysis/figures.py
bash paper/build.sh                                 # regenerates numbers, compiles
```

`paper/build.sh` regenerates `numbers.tex` from the result CSVs before every compile (`analysis/make_numbers.py`), so a number in the compiled PDF cannot drift from the data it was computed from; every generation script is resumable, skipping any `(item_id, seed)` pair already present in its metadata file, so re-running after an interruption is always safe. Because generation and analysis for this panel were still in progress at the time this document was written, several of the per-model quantities in the main paper's Table 1 and Table 2 are recomputed automatically the next time `paper/build.sh` runs against a more complete `data/results/` directory; this document has deliberately favoured describing method over quoting point-in-time numbers for exactly that reason. Section 8 inherits the same discipline, and in one case (Section 8.2) reports a concrete instance of exactly this kind of point-in-time drift, caught by re-running an unmodified analysis script against the current `data/results/` directory.

7.4 The verification gate

`scripts/check_lean.sh` is reproduced in full in Section 1.3. Its three checks – successful `lake build`, a textual `sorry` scan, and the `#print axioms` audit for `sorryAx` – are run as a single command, `bash scripts/check_lean.sh`, and constitute the project's gate on the formal artifact: no result in the main paper or in this document should be read as resting on a machine-checked guarantee unless this script has been run against the exact commit being cited and has printed `PASS: formal artifact verified`.

8 Answering the alternatives

Two rounds of review produced five distinct, falsifiable alternative explanations for the main paper's results – not stylistic objections, but specific mechanisms that, if true, would mean the paper is measuring something other than what it claims. Each is answered here with a new measurement rather than a rebuttal in prose, following the same discipline as the rest of this document: state the objection the way a reviewer stated it, state precisely what would have to be true in the data

for the alternative to explain the main result, describe the test and why it is the right test for that alternative specifically, give the numbers as they appear in the committed result file, and say plainly where the evidence is strong and where it is not. Section 8.6 then collects the statistical machinery used throughout – Wilson intervals, the template bootstrap, and an explicit, uncorrected count of the comparisons made – in one place.

8.1 Naturalness: is the dissociation just improbability?

The objection. Section 3 constructs every repeated item's control by replacing k copies of one word with k distinct fillers from the same template's pool. A skeptical reading of the main paper's Figure 1(b) dissociation does not need periodicity at all: twelve identical copies of *very* is a far more out-of-distribution string, under any reasonable model of English, than twelve of *quite, really, truly, fairly, …* strung together in the same carrier. If a decoder's failure tracks how unlikely its input is rather than how periodic it is, the paper's sharpest test would be confounding the two variables it claims to have separated.

What would have to be true. For naturalness to explain the gap, the repeated item in a matched pair would have to be the *more* improbable member – lower likelihood, higher per-token surprisal – under a competent, independent model of English, and this would have to hold broadly across the 54 matched pairs the dissociation is built from, not just in a handful of cases.

Test design. `analysis/text_perplexity.py` scores every stimulus's raw text under a causal LM's per-token cross-entropy (`model(ids, labels=ids).loss`, which Hugging Face already averages over tokens) and compares each `control_word` item against the `word_rep` item named in its `control_of` field – the identical 54 matched pairs used throughout Section 3. Its first run used `HKUSTAudio/Llasa-1B` as the scorer, on the pragmatic grounds that the checkpoint was already local; a reviewer correctly pointed out that this is not a neutral referee, since Llasa-1B's backbone (Llama-3.2) is literally one of the panel's own architectures, and a naturalness check that answers "is this text unlike what a member of the tested family expects" cannot rule out the panel's own inductive bias leaking into the measurement. `analysis/naturalness_independent.py` reruns the identical per-token-NLL procedure under `microsoft/phi-2` (2.7B) as the primary scorer – a different lab, a different training recipe, and, critically, a different architecture family from all three panel backbones (Llama-3.2 for Llasa, the GPT-2-style AR decoder for XTTS-v2, Qwen3 for Qwen3-TTS), so no panel member's failure can be attributed to sharing an inductive bias with the referee. `gpt2-large` is reported alongside it as a secondary cross-check only, because XTTS-v2's own AR decoder is itself a GPT-2-style architecture; a GPT-2-family LM is therefore not fully independent for that one panel member, and the paper's naturalness claim never rests on it alone. Confidence intervals on the mean paired gap are built by resampling the six stimulus *templates* with replacement, not the 54 pairs, for the reason given in Section 4 and restated in this script's own docstring: the 54 pairs are 6 templates $\times$ 9 values of k, so pairs sharing a template are correlated draws and treating all 54 as independent would understate the true sampling variance.

Numbers. Table 10 reports all three scorers. Under the panel-backbone scorer (`data/results/text_nll.json`), the control is the less probable member of the pair in 89% of the 54 pairs (mean per-token cross-entropy 16.05 for controls vs. 14.85 for repeated items, gap 1.20 nats). Under the independent primary scorer, phi-2 (`data/results/text_nll_independent.json`), the control is still the less probable member in 87% of the 54 pairs (4.89 vs. 3.69, gap 1.20 nats, 95% template-bootstrap CI $[0.58, 1.67]$). The secondary cross-check, gpt2-large, agrees even more strongly: the control is less probable in every one of the 54 pairs (gap 1.98 nats, CI $[1.80, 2.17]$) – a Wilson interval on that 54/54 proportion is $[93.4, 100.0]$% (Section 8.6), which is the cleanest illustration in this project of why Wilson rather

than the normal approximation is used: a normal interval around $p = 1$ is degenerate and would have to be truncated by hand, while Wilson returns a legitimate bounded interval directly.

Table 10: Naturalness check, three scorers, 54 matched pairs. "Ctl. higher" is the fraction of pairs in which the control has the higher (less probable) per-token cross-entropy.

Scorer	Role	Rep. mean	Ctl. mean	Gap	Ctl. higher
Llasa-1B	panel backbone	14.85	16.05	1.20	89%
phi-2	independent, primary	3.69	4.89	1.20	87%
gpt2-large	secondary cross-check	3.36	5.34	1.98	100%

The disagreement, stated honestly. All three scorers agree on the headline direction and on the pooled magnitude of the gap ($\approx$ 1.2–2.0 nats, control less probable in 87–100% of pairs). They do *not* agree on the shape of the gap across k, and the main paper's text is deliberately worded to claim only what both the panel-backbone and the independent scorer support. Under Llasa-1B, the per-k gap widens monotonically across the entire ladder, from 0.37 nats at k=2 to 1.95 nats at k=32 (`by_k` in `text_nll.json`). Under phi-2, the gap widens similarly at first – 0.34 nats at k=2, rising to a peak of 2.62 nats at k=8 – but then *narrows*: 2.12 at k=12, 1.45 at k=16, 0.24 at k=24, and by k=32 it has reversed sign to -0.62 nats, with the fraction of pairs in which the control is less probable falling from 100% at k=3–12 to 33% at k=32 (`by_k_frac_ctl_higher` in `text_nll_independent.json`). At the very top of the ladder, in other words, phi-2 finds 32 verbatim copies of one word *easier* to predict than the length-matched control, which is plausible on its own terms – a small, locally-conditioned LM's induction-head copying becomes close to trivial once a token has already repeated a dozen times, whereas the control at k=32 is 32 fillers cycled four times through an eight-word pool, which is repetitive in a different, less exploitable way. This reversal sits entirely past every panel checkpoint's own k^* (1–8, Table 1 of the main paper): by the k range where counting collapse actually happens, both scorers agree without qualification. The paper's claim is accordingly the conjunction, not the union, of what the two scorers show: the control is the less probable member of the pair in the large majority of cases and across the range where the phenomenon occurs, and naturalness therefore runs opposite to, not in support of, the alternative explanation. A claim that the gap *widens monotonically with* k, which only the non-independent scorer supports, is not made.

Conclusion. Naturalness does not explain the dissociation. If anything, the harder-to-predict member of each pair – the control, under every scorer used – is the one the panel renders correctly, which is the reverse of what the alternative needs. The evidence for the headline direction is strong (three scorers, two of them mutually independent, agree); the evidence for any particular trend of the gap across k is weak past the point where collapse has already occurred, and the paper does not lean on it.

8.2 Unit invariance: repetitions, or acoustic tokens?

The objection. A second alternative locates the attractor on the acoustic side entirely, with text playing no causal role: perhaps a decoder simply loses its place once it has emitted enough near-identical codec frames in a row, regardless of why those frames are near-identical. Under this account, "periodicity" is doing no work beyond producing repetitive audio, and the same collapse

should occur for any input that yields enough near-duplicate acoustic tokens, text-side repetition structure notwithstanding.

What would have to be true. A whole repeated sentence costs several times more acoustic tokens per repetition than a single repeated word. An acoustic-token-budget account therefore predicts that sentence repetition should collapse at a substantially *smaller* k than word repetition (fewer repetitions are needed to accumulate the same token budget), and that the *token count* at the collapse point should be roughly equal across the two families. Theorem A predicts the opposite on both counts: collapse at a comparable number of repetitions regardless of unit, and a token count at collapse that differs in proportion to the unit's cost.

Test design. `analysis/unit_invariance.py` computes, per model, k^* separately for `word_rep` and `sentence_rep` using the same chance-adjusted threshold as the main paper's Table 1 (largest k with accuracy > 0.5), together with the median number of realised acoustic tokens (`n_speech_tokens`) at that k. Comparing k^* directly tests the repetition-count account; comparing the token count at k^* tests the token-budget account; running both on the same models from the same behavioural table lets the two predictions be checked against each other rather than against different data.

Numbers. Table 11 reproduces `data/results/unit_invariance.json` as committed: mean $|k^*_{\text{word}} - k^*_{\text{sent}}| = 1.7$ repetitions across 6 models, against a mean token-count ratio at collapse of $1.34\times$ – repetitions differ by less than two units while tokens differ by a third, which is the pattern Theorem A predicts and the token-budget account does not.

Table 11: Unit invariance, as committed in `data/results/unit_invariance.json`. Token counts are medians at the reported k^*; `n/a` where no k cleared the 0.5 threshold.

Model	k^*_{word}	tok. (word)	k^*_{sent}	tok. (sent)
Llasa-1B	0	n/a	3	351
Llasa-3B	1	209	3	457
Qwen3-TTS-0.6B	4	53	4	80
Qwen3-TTS-1.7B	8	80	4	86
XTTS-v2	4	89	4	128
XTTS-v2 (no penalty, ablation)	2	63	1	29

A data-integrity caveat, found while verifying this section. The committed `unit_invariance.json`'s six-model set is {`llasa1b`, `llasa3b`, `qwen06b`, `qwen17b`, `xtts2`, `xtts2norp`} – it includes the `xtts2norp` ablation and omits Llasa-8B entirely, in tension with the rule stated in Section 6 and enforced everywhere else in this project's pipeline that the ablation is never pooled as if it were an independent checkpoint. The cause, verified directly: at the time this file was written, `data/results/behavioural.csv` carried only 319 of 540 expected rows for Llasa-8B (59%), and for at least one of its two families (`word_rep` or `sentence_rep`) accuracy had not yet cleared the 0.5 threshold at any k, so `unit_invariance.py`'s own finiteness check dropped it from the output, leaving `xtts2norp` as the sixth entry by coincidence of ordering rather than by design – the script itself (re-inspected for this section) does correctly exclude `ABLATIONS` by construction, so this is a snapshot-timing issue in the committed artifact, not a logic error in the analysis code. Re-running the identical, unmodified script against the current `data/results/behavioural.csv` – performed here for verification, and not written back to the repository, since this document's brief is to report on committed results rather than to regenerate them – gives the true six-panel-member set with Llasa-8B in place of the ablation: k^*_{word}=4 (median 328 tokens), k^*_{sent}=3 (median 424 tokens) for Llasa-8B, and an across-panel mean $|k^*_{\text{word}} - k^*_{\text{sent}}|$ of 1.7 repetitions (unchanged to

one decimal) against a mean token ratio of 1.50× (vs. the committed file's 1.34×). The qualitative conclusion is unaffected by the correction – repetitions, not tokens, is still the unit that predicts the collapse point – but the committed file, and the 6-model statistic quoted from it in the main paper, should be regenerated once Llasa-8B's behavioural generation completes, and the corrected number should read as a true six-of-six-panel statistic rather than a six-of-seven-including-the-ablation one.

Conclusion. The horizon is counted in repetitions, not accumulated acoustic tokens: k^* moves by under two repetitions between a unit that costs one token and one that costs several times more, while the token count at collapse tracks the unit's cost almost exactly. The evidence is qualitatively robust to the data-integrity issue just described; the point estimate of the token-count ratio is not, and should be treated as provisional until the artifact is regenerated. One further per-cell caveat: Llasa-1B's word-repetition accuracy never exceeds 0.5 even at k=1 (Table 11, `n/a`), so no token count at collapse is defined for that cell; the mean token-count ratio is accordingly computed over five models for the word side of the comparison, not six, a distinction the committed JSON's flat `mean_tok_ratio` field does not surface on its own.

Superseded instrument (Section 9). Both k^* and the per-k accuracy this subsection thresholds ("largest k with accuracy > 0.5" against `data/results/behavioural.csv`) are computed from the Whisper-judged, exact-match pipeline of Section 4, which Section 9.1 shows is biased specifically against the periodic material this comparison is built from. The repetitions-vs-tokens conclusion above does not depend on the exact resolution used to locate the collapse point – the $|k^*_{\text{word}} - k^*_{\text{sent}}|$ argument only needs a threshold applied *consistently* to both families, which it is, and an undercounting bias that hits both families' transcripts in the same direction does not selectively favour either side of the comparison – but the absolute k^* values in Table 11 should not be read as calibrated against the paper's current CTC-judged relative-count-error metric, which has no single-threshold crossing point by construction (Section 9.3).

8.3 Capacity confound: is effective-rank decline tautological?

The objection. A model that correctly renders *very* sixteen times in a row produces audio that genuinely is more acoustically monotonous than a control rendering sixteen distinct words. A shrinking effective rank on repeated-text trajectories could then be measuring nothing but this surface fact about the stimulus – the audio the prompt asked for – rather than any loss of the decoder's capacity to represent *how many* repetitions have occurred. Under this reading, the capacity-gain ratio (Section 8.3) would be a restatement of the stimulus design, not a discovery about the decoder.

What would have to be true. For the tautology objection to fully account for the ratio, the repeated-vs-control gap in effective-rank growth would have to close once (i) the trajectory's own realised output diversity is held constant by regression, so "more repetitions requested" is separated from "less varied audio produced," and (ii) the comparison is restricted to items the model rendered *correctly*, where the audio is by construction exactly what was asked for and no degenerate-output confound can apply.

Test design. `analysis/capacity_confound.py` implements both checks on data already collected, per model and per family (`word_rep`, `control_word`). The *adjusted* test regresses $\mathcal{N}_{\text{eff}}$ on $\log k$ together with three z-scored covariates of the emitted acoustic-token sequence: the type/token ratio, the immediate-repeat rate (the fraction of tokens equal to their immediate predecessor, a direct index of local acoustic monotony), and the raw token count; the reported quantity is the $\log k$ coefficient with the covariates included, i.e. the capacity gain that remains after realised output diversity is partialled out. These covariates are only available for checkpoints whose acoustic-token ids are persisted to disk, which in this panel is the Llasa family only – XTTS-v2 and Qwen3-TTS

contribute a token count but not the distinct-token identities needed for type/token ratio or repeat rate, so their adjusted cells are inestimable rather than zero. The *correct-only* test refits the same unadjusted regression restricted to items labelled `correct` at seed 0 (Section 4); coverage thins sharply at high k precisely because correct renderings become rare there, which is the phenomenon under study rather than a defect of the test, so per-model n is reported alongside every ratio rather than suppressed.

Numbers. Table 12 reproduces `data/results/capacity_confound.json`'s per-model ratios (repeated slope over control slope; smaller means the repeated-text trajectory gains fewer distinguishable states per doubling of k than its control). The panel median is 0.50 unadjusted (n=6 checkpoints), 0.50 after controlling for output diversity (n=3, the Llasa sub-family only, for the reason above), and 0.47 restricted to correct-only renderings (n=5).

Table 12: Capacity-confound ratios (repeated/control slope of $\mathcal{N}_{\text{eff}}$ against $\log k$), from `data/results/capacity_confound.json`. n pairs are (word_rep, control_word) item-layer rows entering each regression; `---` marks an inestimable cell (fewer than 12 usable rows, or fewer than 4 distinct k values).

Model	Raw	+diversity	correct-only
Llasa-1B	0.42	0.50	0.01 (n=14/32)
Llasa-3B	0.28	0.34	0.80 (n=24/22)
Llasa-8B	0.43	0.75	— (n=0/0)
Qwen3-TTS-0.6B	0.57	—	0.31 (n=42/84)
Qwen3-TTS-1.7B	0.78	—	0.47 (n=64/82)
XTTS-v2	0.60	—	0.56 (n=22/41)
Panel median	**0.50**	**0.50**	**0.47**
XTTS-v2 (no penalty, abl., not pooled)	0.89	—	— (n=9/22)

The three medians agree to within two hundredths of each other, which is the central result: neither control moves the ratio meaningfully away from the unadjusted panel figure. Two per-model cells deserve a direct flag rather than being folded silently into the median. Llasa-1B's correct-only ratio, 0.01, looks like the tautology objection winning outright for one checkpoint, but it rests on 14 repeated-side and 32 control-side items – a small-sample cell in which the repeated-side slope is itself measured at 0.47, i.e. essentially flat, and a single noisy near-zero slope in the denominator's counterpart is enough to send the ratio close to zero without implying the gap has actually closed; read this cell as unstable, not as evidence the effect vanishes. Llasa-8B's correct-only cell is empty (n=0): none of its seed-0, $k \geq 2$ `word_rep` items were labelled `correct`, consistent with the low k^* reported for that checkpoint elsewhere in this document (Section 8.2) rather than with any failure of the confound test itself.

Conclusion. The tautology objection is answered, with two honest qualifications. Holding realised output diversity fixed and restricting to demonstrably correct renderings both leave the panel-median ratio within rounding of the raw figure (0.50 raw vs. 0.50 adjusted vs. 0.47 correct-only), so the capacity-gain gap is not an artifact of repeated audio being acoustically monotonous by construction. The qualification: the diversity-adjusted control is well-powered for only three of the six panel checkpoints, because acoustic-token identities are persisted for the Llasa family alone, so 0.50 is a narrower claim than the raw, six-checkpoint 0.50; and the correct-only control has two unstable or empty per-model cells at the extremes of the panel's capability range, so it should be read as directionally corroborating the raw result rather than as an independently decisive test in its own right.

Superseded correctness label (Section 9). The *correct-only* variant restricts to items whose `outcome` equals `correct` under the Whisper-judged, exact-match criterion of Section 4 – exactly the label Section 9.1 shows is biased toward *under*-crediting genuinely correct repeated renderings at moderate-to-high k. That bias only ever moves an item out of the repeated side's `correct` pool (a genuinely correct rendering that Whisper undercounts is relabelled `undercount`, never manufactured as `correct` by a transcript that happens to agree by coincidence with a wrong recording), so the correct-only cells reported here are, if anything, an under-powered rather than an inflated sample of repeated-side successes – the same asymmetry the gap-plausibility bound of Section 9.1 uses to bound the Whisper-judged accuracy gap. The raw and diversity-adjusted variants of this test do not use the `correct` label at all and are unaffected.

8.4 The competing account: does the count survive in the representation?

The objection. Venkatesh [**?**] reports that in text-only autoregressive LMs performing repeated-token counting, a linear probe on the residual stream decodes the true count near-perfectly at every layer, including the exact layer where the model's own output is already wrong: the count is not erased, only unused by the output pathway. A trained linear probe is precisely the L-Lipschitz readout class Theorem A(iii)/(iv) (main paper, Section 4) constrains, so if the same dissociation holds for TTS decoders, the effective-rank account of Section 8.3 would be measuring the wrong thing: a small, count-specific subspace could persist and remain linearly readable inside a trajectory whose *aggregate* rank still looks low, since a whole-trajectory participation-ratio statistic like $\mathcal{N}_{\text{eff}}$ has no reason to be sensitive to a low-dimensional signal buried inside it.

What would have to be true. For the competing account to hold here, a linear probe would have to recover k about as well from the *late* part of a repeated-text trajectory as from the *early* part – no within-trajectory degradation – and about as well on repeated text as on the length-matched control. Representational degradation, by contrast, predicts within-trajectory loss specific to periodic conditioning: better decodability early (when the boundary between "count so far" and "count remaining" is still information the decoder has just read off the text) than late (once Theorem A(ii)'s indistinguishability has had repetitions to bite), and no comparable loss on the non-repetitive control.

Test design. `analysis/probe_count.py` fits a ridge regression predicting $\log_2 k$ from the mean hidden state over, separately, the first third and the last third of each instrumented trajectory, at every probe layer, scored by leave-one-template-out cross-validation so a probe is never evaluated on a carrier sentence it was fitted on – solved in the dual ($n \approx 54$ items against up to 4096 features per layer) purely for tractability, with identical predictions to the primal. This is not an analogy to Theorem A(iii); a linear, cross-validated probe with a reported R^2 is exactly the readout class the theorem constrains, so the test evaluates the theorem's own claim directly. *Retention* is defined per model and family as the best-layer late-window R^2 divided by the best-layer early-window R^2: because the same probe class, the same items, and independently the best layer for each window are used at both ends, retention cancels probe capacity and item-difficulty confounds, leaving only the within-trajectory change. The matched control makes the comparison interpretable: the identical probe, the identical model, the identical layer-selection procedure, decoding the identical quantity from non-repetitive text of equal length.

Numbers. Table 13 reports retention for every model and condition in `data/results/probe.json`. Across the six true panel checkpoints, median retention on repeated text is 0.97 against 1.11 on controls, and repeated-text retention is lower than its own control in all six of six. Best-layer R^2 decoding $\log_2 k$ is modest even at its best: the panel median late-window best-layer R^2 on `word_rep` is ≈ 0.48 – well short of the near-perfect decodability [**?**]

reports for text LMs – so the count is only ever coarsely recoverable here, at either end of the trajectory.

Table 13: Ridge-probe retention (late-window best-layer R^2 / early-window best-layer R^2), from `data/results/probe.json`. All entries n=54 items.

Model	`word_rep` retention	`control_word` retention
Llasa-1B	0.97	1.09
Llasa-3B	0.87	0.99
Llasa-8B	0.96	1.09
Qwen3-TTS-0.6B	1.46	1.74
Qwen3-TTS-1.7B	0.77	1.12
XTTS-v2	1.92	2.85
Panel median	**0.97**	**1.11**
XTTS-v2 (no penalty, abl., not pooled)	1.37	1.40

A second pooling caveat, found while verifying this section. The main paper's probe statistics (0.97 repeated, 1.12 control, "repeated lower in seven of 7 variants") are computed by `analysis/make_numbers.py` over every entry in `probe.json`, including `xtts2norp`. Unlike that same script's behavioural-dissociation aggregation, which explicitly filters `beh[~beh.model.isin(ABLATIONS)]` (Section 8.5), its probe-aggregation loop iterates `pr.values()` with no corresponding filter, so the reported "7" variants are the six true panel checkpoints plus the ablation counted as a nominal seventh – exactly the double-counting Section 6 and `findings.md`'s own "Lessons and Constraints" warn against elsewhere in this project. Recomputed over the six true panel checkpoints alone (Table 13), the medians are 0.97 repeated and 1.11 control – materially identical to the pooled-with-ablation figures – and repeated retention is still lower in six of six. The ablation's inclusion therefore does not manufacture this particular result; the correct fix is that `make_numbers.py`'s probe section should apply the same `ABLATIONS` filter its behavioural section already applies, and the main paper's "seven of 7" should read "six of 6," with `xtts2norp` reported as a consistent seventh, separate data point rather than folded into the panel statistic. This document does not modify that script; it reports the discrepancy and gives the corrected, panel-only numbers above so that the supplementary record is self-consistent even where the generating script currently is not.

Conclusion. The representational-degradation account is favoured over the output-policy account, but only on qualified terms that this document states plainly rather than rounds up. On the affirmative side: retention is below 1 – within-trajectory loss – on repeated text in every panel checkpoint, and at or above 1 – flat or improving – on the matched control in five of six (Llasa-3B's control retention, 0.99, is essentially flat rather than clearly improving). On the qualified side: best-layer R^2 never approaches the near-perfect decodability the competing account reports for text LMs, so even early in generation the count is only coarsely present; and retention below 1 is degradation, not erasure – the probe still recovers a non-trivial signal late in most trajectories, so this test cannot rule out that some of the count remains linearly readable somewhere the aggregate effective-rank statistic of Section 8.3 would not detect. As the main paper's Discussion already states, the two accounts are not mutually exclusive, and this probe result is evidence for representational degradation over pure output-policy failure, not proof against the latter.

8.5 The XTTS repetition-penalty ablation

The objection. XTTS-v2 ships `repetition_penalty`= 5.0 on its acoustic-token decoding (Section 6), an aggressive built-in anti-repetition mechanism active during every sampling pass this study measures. A skeptical reading: XTTS-v2's comparatively high k^* among the panel is an artifact of this decoding-time crutch rather than evidence bearing on Theorem A at all, and any argument built on contrasting XTTS-v2 against the Llasa scale ladder – e.g. that capacity alone does not order k^* – is confounded by an uncontrolled decoding-rule difference between the two families.

What would have to be true. If the penalty, not the mechanism the paper argues for, were responsible for XTTS-v2's collapse-resistance, then disabling it should make the periodicity-vs-length *dissociation* disappear or shrink toward a decoding-rule artifact – the model should stop showing a repeated-vs-control gap once the crutch is removed, because there would be no underlying mechanism left for the gap to reflect.

Test design. The registry (Section 6, `src/common/registry.py`) carries `xtts2norp` as a seventh entry: the identical XTTS-v2 checkpoint, generated by `src/models/xtts_gen.py --repetition-penalty 1.0` (penalty disabled), scored and instrumented through the identical pipeline as every panel member, under a separate registry key so its outputs never mix with `xtts2`'s on disk. This is the right design for the objection specifically because it manipulates exactly one decoding-time knob while holding the checkpoint weights, the stimuli, and every other setting fixed – and because it is a within-model manipulation rather than a new independent system, it is excluded by construction from every panel-pooled statistic in this project (`ABLATIONS = {"xtts2norp"}`, enforced in `analysis/make_numbers.py`, `capacity.py`, `capacity_confound.py`, and `unit_invariance.py`, modulo the one gap in the probe aggregation noted in Section 8.4). Pooling it would silently inflate the panel size and double-count XTTS-v2's architecture – precisely the failure `findings.md`'s "Lessons and Constraints" records was caught and fixed once already in this project.

Numbers. With the penalty (`xtts2`): $k^* = 4$, accuracy at $k \geq 6$ is 13.9% repeated vs. 59.3% control. Without it (`xtts2norp`): $k^* = 2$, 1.9% repeated vs. 14.8% control. Disabling the penalty makes the model strictly worse at counting – k^* falls from 4 to 2 and both accuracies drop – while the relative dissociation not only survives but widens: the control-to-repeated accuracy ratio is $\approx 4.3\times$ with the penalty and $\approx 7.8\times$ without it. On the capacity side (`data/results/capacity.json`), disabling the penalty compresses the entire dynamic range of the measurement: repeated-text gain falls from 21.6 $[14.1, 28.7]$ to 7.7 $[4.6, 10.8]$ and control gain falls from 36.1 $[31.2, 40.9]$ to 8.7 $[5.1, 12.4]$, so the ratio moves from 0.60 to 0.89 and the paired-gap confidence interval, which excludes zero with the penalty on ($[6.0, 20.8]$), crosses it with the penalty off ($[-1.8, 6.3]$, `ci_separated: false`). This is the single checkpoint-level cell in the entire panel-plus-ablation set where the capacity contrast does not reach significance, and it is reported here as such rather than omitted.

Conclusion. The penalty is masking the collapse, not producing it: the model counts worse without it, and the periodicity-vs-length behavioural gap persists – indeed strengthens in relative terms – once the crutch is removed, which is the opposite of what the objection needs. XTTS-v2's place in the panel *with* the penalty is accordingly a conservative test of the theory, not an inflated one. The qualification is on the capacity side specifically: removing the penalty shrinks both conditions' dynamic range so much that the effective-rank contrast, though still pointed the same direction (ratio $0.89 < 1$), is no longer statistically resolved for this one ablated checkpoint. That instability is itself informative – it is consistent with the penalty's role being to inject additional acoustic-token diversity into *every* generation, repeated or not, which is exactly the kind of output-diversity confound Section 8.3 controls for directly on the unablated panel – but it means the

ablation corroborates the behavioural dissociation more strongly than it corroborates the capacity measurement, and the two should not be conflated.

Superseded numbers, and a withdrawn reading (Section 9). The k^* and accuracy figures above, and the reading built on them – that disabling the penalty makes the dissociation "persist, indeed strengthen" – are Whisper-judged and exact-match, both superseded by Section 9. Rerun under the CTC judge with the relative-count-error metric (`data/results/count_error.json`, Section 9.3), the picture is different: `xtts2` (penalty on) renders repeated material at a median relative count error of -15.6% against an exact 0.0% control at $k \geq 6$, matching the qualitative story above, but `xtts2norp` (penalty off) degrades on *both* arms – -58.3% repeated, -66.7% control – close enough in absolute terms that the ablation no longer isolates periodicity from a general breakdown of the model's decoding once its one built-in anti-repetition mechanism is removed. The specific reading that the penalty selectively masks the collapse, rather than merely holding the whole model together, is accordingly withdrawn, matching the project's own running notes (`findings.md`). The subsection's structural conclusion is otherwise unaffected: the ablation remains uninformative about the dissociation rather than confirming it, and XTTS-v2 with the penalty on remains the panel's conservative member, since the penalty acts against the behaviour under study – that conclusion just no longer rests on the Whisper-judged "strengthens" comparison given above.

8.6 Statistical treatment

Wilson intervals for proportions. Every proportion reported as a percentage with an interval in the main paper and in this document – counting accuracy, the fraction of matched pairs with the control less probable, the fraction of repeated-vs-control paired differences that are positive – uses the Wilson score interval (`wilson` in `analysis/make_numbers.py`), not the normal approximation. The reason is not a stylistic preference: several of this project's cells sit at or near 0% or 100% (word-repetition accuracy at high k for the weaker checkpoints; the gpt2-large naturalness check at exactly 54/54, Section 8.1), where the normal interval's half-width $z\sqrt{p(1-p)/n}$ degenerates toward zero or produces bounds outside $[0, 1]$ that then have to be clipped by hand. Wilson's interval, derived by inverting the normal test for p rather than plugging $\hat{p}$ into a symmetric approximation around it, stays inside $[0, 1]$ by construction and remains informative at the boundary: the gpt2-large cell, 54/54, has an undefined normal interval (zero estimated variance) but a Wilson interval of $[93.4, 100.0]\%$, which is the number actually reported.

Bootstrap over templates, not items. Every confidence interval on a *mean* or *slope* in this project – the capacity-gain intervals of Section 4, the naturalness-gap interval of Section 8.1 – resamples the finite set of stimulus *templates* with replacement, never the larger set of items or pairs built from them. The reason is structural, not a robustness nicety: the 54 naturalness pairs are 6 templates $\times$ 9 values of k, and the panel's capacity-gain fits are built from a handful of carrier sentences crossed with many k and (for the instrumented pass) one seed; items sharing a template share carrier syntax, word count, and, for word-repetition items, seven of eight possible filler choices, so they are correlated draws rather than independent ones. Resampling items directly would treat, e.g., the 54 naturalness pairs as 54 independent observations when the design in fact contains six independent units; the resulting interval would be too narrow, understating genuine uncertainty in exactly the comparisons this section exists to get right.

Per-cell sample sizes. This document reports n alongside every point estimate in Sections 8.1–8.5 rather than only in a methods paragraph, because several cells are close to the floor at which an effect can be resolved at all: the diversity-adjusted capacity-confound test is estimable for only 3 of 6 panel checkpoints (Table 12); its correct-only variant has one cell at n=0 (Llasa-8B) and

one at n=14 (Llasa-1B, repeated side) that produces a ratio (0.01) this document explicitly flags as unstable rather than load-bearing; the linear-probe retention statistic is computed from n=54 instrumented items per model and family throughout, the same instrumented subset documented in Section 3. Reporting these n's is what makes it possible to tell an unstable small-sample cell (Llasa-1B's correct-only ratio) from a stable one (the panel medians, which agree to within two hundredths across three different controls) without re-deriving the analysis from source.

No multiple-comparison correction, stated so a reader can apply their own. Nowhere in this project's analysis pipeline – confirmed by reading every script cited in this section – is a Bonferroni, Holm, or Benjamini–Hochberg correction applied across the per-model tests that make up the panel-level claims. The count a reader wanting a family-wise guarantee would need depends on which family of tests they consider the relevant unit. The primary capacity-gain contrast of Section 8.3 is six per-model disjoint-interval tests, one per panel checkpoint (`ci_separated` in `capacity.json`). The confound checks of Section 8.3 add up to two further ratio estimates per model (diversity-adjusted, correct-only) on top of the raw one, for as many as eighteen nominal model-level cells across the three variants, of which fourteen are actually estimable (6 raw + 3 adjusted + 5 correct-only, Table 12). The probe-retention comparison of Section 8.4 is six per-model directional tests (repeated retention below control retention). The naturalness by-k breakdown of Section 8.1 is nine k-values reported under two mutually scrutinised scorers. None of these is adjusted for having been looked at together. A reader who wants a Bonferroni-style family-wise error rate should divide the nominal $\alpha = 0.05$ by six for the primary capacity contrast alone, or by the low twenties if every per-model cell across Sections 8.3 and 8.4 is treated as one family; this document supplies the counts rather than picking a family on the reader's behalf, because the right grouping depends on which claim in the main paper the reader is trying to stress-test.

9 The judge is part of the experiment

This section documents the project's single most consequential methodological correction, made after Sections 1–8 were written: the behavioural judge that produced every Whisper-scored, exact-match number in this document was itself found to be biased against exactly the phenomenon the paper studies, and was replaced. We give the audit that found the bias, the validation of its replacement, the metric change the replacement made possible, an independent ASR-free attempt that did not replicate the result (and why that failure is informative rather than damning), and the implication for measurement practice beyond this paper. Sections 1–8 are not rewritten; the numbers in them are marked at each point they recur, because the contrast between what the old instrument said and what the new one says is itself part of the evidence this section presents.

9.1 The audit that disqualified Whisper

Why an autoregressive recogniser is the wrong instrument, in principle, before any measurement is even taken. Every behavioural number reported through Section 8 is built on Count A: occurrences of the target word in a Whisper large-v3 transcript (Section 4). Whisper's decoder is itself an autoregressive sequence model with a learned language-model prior over plausible continuations, generating each transcript token conditioned on its own previously decoded tokens. Confronted with audio that genuinely repeats a phrase many times, that prior does what a language model's prior does to any low-marginal-probability continuation: it discounts it. A decoder transcribing "the dog was very big …" eight times in a row has, by the third or fourth repetition, strong internal evidence – from its own prior over what sentences look like – that the correct continuation is *not* an implausible ninth verbatim repeat, and it is prone to closing the

sentence and stopping. That is the same architectural pattern Theorem A (main paper, Section 4) describes for a TTS *decoder* under periodic conditioning, applied here to a different modality by the instrument doing the scoring. Using such a recogniser to grade a TTS decoder's repetition count is accordingly not a noisy measurement, in the way an imperfect segmentation or a slightly-too-permissive threshold would be; it is a category error, because the instrument is subject to the very failure mode it is being asked to certify the presence or absence of in something else. No amount of decoding-parameter tuning fixes this, since the flaw is architectural, not a setting – which is exactly what the decoding-consistency test below was designed to check.

`analysis/asr_reliability.py` (`data/results/asr_reliability.json`) turns this into three measurements that do not require ground-truth transcripts of the real generations, none of which existed at project scale.

(a) Decoding-consistency. If Whisper is asked to transcribe the same audio twice – once with the production greedy configuration (`condition_on_prev_tokens=False`, the setting used to build every number in Section 4), once with beam search (`num_beams=5`, otherwise unchanged) – a recogniser measuring the audio rather than its own decoding path should agree with itself almost always. On a stratified 114-item subsample spanning `word_rep` and `control_word` items across four checkpoints from all three panel architectures, it does not agree evenly. On `control_word` items (distinct fillers, no repetition) the two decoding strategies produce the same occurrence count 96.3% of the time (n=54, mean absolute count difference 0.91); on `word_rep` items (the repeated material) they agree only 83.3% of the time (n=60), with a mean absolute disagreement of 33.8 occurrences – tens of repetitions apart, on average, between two transcriptions of the identical audio. By k=24 the two decoding strategies agree on only 17% of repeated items, against 100% agreement on the matched controls at the same k. Whatever Count A measures on repeated material at moderate-to-high k, it is measuring Whisper's own decoding path at least as much as it is measuring the TTS output.

(b) Bias direction on synthetic ground truth. Decoding-consistency shows Whisper is unreliable on repeated material; it does not by itself show which direction the bias runs, since two decoding strategies could disagree without either being systematically low. To settle direction and magnitude against real ground truth, six single, individually verified k=1 renderings (one per carrier template, all from the same `xtts2` voice, each independently confirmed to contain the target word exactly once with duration matching a single occurrence) were concatenated $k \in \{2, 4, 8, 16, 32\}$ times with a short (0.25 s) silence between copies. Because the sequence is built by concatenation of a verified atom, the true occurrence count is exact by construction – there is no annotation step to get wrong.

That construction is also this audit's limitation, and it is worth stating plainly rather than leaving for a reader to notice. Concatenated audio is not the audio under study: it has clean boundaries and inserted silences where generated repeated speech has co-articulation, drifting prosody and copies that run together. A judge calibrated on the spliced version may be more accurate there than on the real thing, which would make the numbers here an optimistic bound on judge reliability rather than a measurement of it. Exact ground truth and realistic audio cannot both be had without human annotation, and we chose exactness.

What covers the other side is S14's noise floor, which needs no ground truth: it measures how far two independent recognisers disagree on the *generated* audio, and finds the deficit larger than that disagreement by a factor of 1.86, with our judge reporting the *higher* count in 11 of 13 disagreements — the direction that makes the deficit harder to see, not easier. Neither audit alone settles the judge; the pair bracket it, exact ground truth on unrealistic audio and a floor on realistic audio. Table 14 gives the mean counted/true ratio at every k, alongside the identical construction run with k *distinct* words from the same voice and carrier family instead of one word repeated (three

cyclic rotations per k, same silence structure), which isolates whether the undercount is specific to periodicity or a generic artefact of long concatenated audio.

Table 14: Whisper large-v3, mean counted/true ratio on synthetic ground truth (`data/results/asr_reliability.json`, `synthetic_ground_truth` and `synthetic_ground_truth_control`). n=6 per cell for the repeated-word condition, n=3 rotations per cell for the distinct-word control.

Condition	k=2	k=4	k=8	k=16	k=32
Repeated word	0.92	0.50	0.27	0.65	0.53
Distinct words (control)	1.00	1.00	1.00	1.00	1.00

The distinct-word control transcribes exactly, at every k up to 32: Whisper has no trouble with long concatenated audio *per se*. The repeated-word condition falls to a ratio of 0.27–0.65 for $k \geq 4$, non-monotonically (the dip at k=8 and partial recovery at k=16 track *how* Whisper collapses – typically emitting one or two copies of the sentence rather than a count that degrades smoothly with length – not a smooth function of duration). The gap between the two rows is the direct measurement decoding-consistency could only imply: an undercount specific to identical repeated material, of exactly the sign that would inflate the paper's repeated-vs-control accuracy gap.

Gap-plausibility bound. Two independent bounds ask how much of the paper's headline Whisper-judged gap this bias could account for, without trusting either the real behavioural data or the synthetic trials alone. The *behavioural-CSV bound* credits every repeated item Whisper's exact-match criterion labelled `undercount` – the one outcome bucket structurally reachable by an undercounting bias – back to `correct`, which is deliberately generous to the confound (some of those items are certainly genuine TTS failures, not ASR artefacts, so this is an upper bound, not a point estimate). At $k \geq 6$ the measured Whisper-judged accuracy is 12.3% repeated (n=612) against 42.8% control (n=612), a $\approx 3.5\times$ gap; crediting every undercount back raises repeated to at most 36.3%, still $\approx 1.2\times$ below the control rate. The bias can inflate the reported gap; it cannot manufacture it. The independent *instrument-ceiling bound* uses only the synthetic trials: the fraction of ground-truth-known periodic sequences at $k \geq 4$ where Whisper's transcript count exactly equalled the true count – the best "correct" rate the exact-match criterion could *ever* award to a model that always renders a perfect k-repetition – is 12.5% overall (n=24 trials: 33.3% at k=4, 16.7% at k=8, 0% at k=16 and at k=32). The exact-match criterion throws away real successes at these k, a genuine measurement-validity problem that neither reverses the paper's effect nor is fully responsible for its size.

Conclusion. The audit converges from three independent angles – self-inconsistency under a changed decoding strategy, systematic undercount on ground truth with an exact known answer, and a matched control showing the undercount is periodicity-specific rather than a length artefact – on the same verdict: Whisper large-v3 cannot be trusted to count repeated speech, precisely because it is repeated, and precisely because Whisper's decoder is the same kind of machine as the one under study.

9.2 Validating the replacement

Disqualifying one instrument does not license trusting the next one on faith; adopting a replacement without putting it through the identical test would repeat the very error just documented. `analysis/ctc_validation.py` therefore reruns the audit's construction – a verified k=1 rendering concatenated N times with a short silence, exact ground truth by construction, no splicing inside

a word – on *both* recognisers side by side, on identical audio, at $k \in \{2, 4, 8, 16, 32\}$, for the same six word templates plus a matched set of six control items built the same way: each is a single verified `control_word` utterance (already containing two distinct filler words, no repetition within it) concatenated N times, exactly as the periodic arm concatenates a single verified word rendering N times. That construction repeats a *sentence*, not a word – the resulting audio is periodic at the utterance grain, only at a coarser grain than the periodic arm above it – so it is labelled below for what it is (*repeated-sentence*) rather than as a distinct-word condition. It is consistent with the periodicity account, not a counterexample to it, but it does not supply the genuinely aperiodic comparison a reader might expect from a "distinct" label; that comparison, for Whisper specifically, is Table 14 above, built by cycling real distinct donor words together rather than repeating one utterance.

The replacement is `facebook/wav2vec2-large-960h-lv60-self`: a connectionist temporal classification (CTC) recogniser. It has no autoregressive decoder and no internal language model. It emits a per-frame distribution over characters and collapses repeated frames and blank symbols into a final transcript; the number of times a word appears in the output is governed by how many times the acoustic pattern for that word actually appears in the input, not by what a decoder expects to come next, because there is no next-token expectation to have. `src/common/asr_ctc.py` chunks long audio into 25 s windows with a 0.25 s overlap before transcribing (repetition items run up to 40 s and the model's conv+attention stack is more stable processed in pieces; the overlap exists so a word straddling a chunk boundary is not cut in half), then joins the per-chunk transcripts with spaces – a purely local, non-autoregressive operation with no scope for the kind of global self-suppression Whisper exhibits in Section 9.1. It is a substantially worse recogniser in absolute terms: higher word error rate, no punctuation, no casing, upper-case-only output. That is the right trade for this purpose, because its errors are not correlated with repetition.

Table 15: Head-to-head validation on identical synthetic ground truth (`data/results/ctc_validation.json`). Median counted/true ratio, $k \geq 4$ pooled (`_kge4` in the committed summary). The "repeated-sentence" arm is a single verified control utterance concatenated N times – periodic at the sentence grain, not a genuinely aperiodic control (see text).

Recogniser	Material	$k \geq 4$ ratio	Notes
CTC	periodic (repeated word)	1.00	22/24 exact; 2 off by 1 (k=16, 32)
CTC	repeated sentence (control)	1.00	23/24 exact; 1 off by 1 (k=32)
Whisper large-v3	periodic (repeated word)	0.19	no transcript at k=16, 32
Whisper large-v3	repeated sentence (control)	0.25	no transcript at k=16, 32

The CTC recogniser's *median* ratio is 1.00 at every k from 2 to 32, on both arms, but the median is not every trial: of the 24 $k \geq 4$ periodic trials, 2 fall short (ratio 0.938 at one k=16 donor, 0.969 at one k=32 donor, each off by exactly one occurrence), and of the 24 repeated-sentence trials, 1 falls short (ratio 0.969 at one k=32 donor). The mean tells the same story more bluntly: 0.996 on the periodic arm (95% donor-level CI [0.988, 1.000], `data/results/judge_audit_ci.json`) and 0.999 on the repeated-sentence arm. "Exact at every k" therefore overstates what the data show; the accurate claim is that the CTC judge is correct on all but 3 of these 48 trials, and every miss is off by a single occurrence, never a collapsed or fabricated count. Whisper reproduces the same undercount pattern this validation exists to check for (0.25 at k=4, 0.125 at k=8 on both arms), and surfaces something the shorter sequences of the earlier audit did not: every one of its transcription calls at k=16 and k=32 – both arms, all twelve word templates – raised a `ValueError` and returned

no transcript at all, on audio between roughly 40 and 100 seconds long. The $k \geq 4$ ratios in Table 15 for Whisper are therefore computed from only the k=4 and k=8 trials that returned anything; the true failure mode at $k \geq 16$ is not "low ratio," it is *no measurement*, an operational fragility layered on top of the counting bias itself.

This is the same test that disqualified Whisper, applied without exception to its replacement. The asymmetry in the result – one recogniser correct on all but a handful of trials, each off by a single occurrence, the other biased and, at the top of the ladder, unable to produce output at all – is the paper's warrant for treating the CTC judge as right for this purpose, not merely as a different choice made for convenience.

9.3 The metric: from exact match to relative count error

Adopting the CTC judge changes what Count A measures; it also exposes a second problem that a better recogniser alone does not fix. Sections 4 and 8 score correctness by exact match: an item is `correct` only if the transcript count equals the requested k precisely. Exact match is the wrong resolution for this measurement regardless of which recogniser supplies the count. A model that renders 30 of 32 requested repetitions and one that renders 3 of 32 are doing categorically different things – one is a near-miss, the other a near-total failure – but exact match scores both identically `wrong`, discarding exactly the information the paper's ladder design (k up to 32, dense at low k) was built to resolve. Worse, exact match is not even robust on its own terms: a recogniser slip of ± 1 – one missed or duplicated word, well within ordinary ASR noise even for a good recogniser – flips the label from `correct` to `undercount` or `overcount`, so the binary label carries less signal than the continuous count it is thresholded from.

`analysis/count_error.py` replaces exact-match correctness with the *relative count error*, $(\hat{c} - k)/k$, where $\hat{c}$ is the CTC occurrence count. This is signed, so premature stopping (undercount, negative error) is distinguishable from looping (overcount, positive error) rather than both being folded into a single `wrong` bucket; it is scale-free, so a deficit of one repetition reads as a small error at k=32 and a large one at k=4, matching how noticeable the failure actually is rather than counting absolute misses; and it degrades gracefully under residual recogniser noise, since a ± 1 slip moves the statistic by $1/k$ rather than flipping a label outright. Degenerate audio (silence, spectral flatness above threshold) is excluded and reported as its own rate rather than folded in as an extreme negative error, the same discipline Section 4's outcome taxonomy already applies. The judge switch itself is a single flag, `--judge {whisper,ctc}`, in `src/common/score_counts.py`: audio-level degeneracy flags (RMS, spectral flatness, trailing silence, loop autocorrelation) are computed once from the audio during the Whisper pass and are judge-independent properties of the waveform, so they are borrowed unchanged when scoring under the CTC judge rather than recomputed – the only thing that changes between `behavioural.csv` and `behavioural_ctc.csv` is which transcript supplies Count A.

Five of six panel checkpoints show a repeated-item error interval disjoint from their own control's, which sits at exactly 0.0% for every model at every k up to 32 – the matched-control result of the main paper's Section 3.1 reproduced under the new judge with the new metric. Qwen3-TTS-1.7B is the exception, at 0.0% on both arms: not a null result but a second regime, as the main paper's Discussion frames it – a checkpoint for which Theorem A's contraction premise may simply not hold, which the theorem itself is silent about rather than refuted by. At k=8 specifically, the five affected checkpoints' median relative error ranges from -12.5% to -18.8% (`rep_by_k` in the committed JSON), i.e. a deficit of roughly one to one-and-a-half repetitions out of eight – the figure Section 9.4 uses to explain why a duration-only criterion cannot see this effect. The ablation row, `xtts2norp`, is excluded from every pooled panel statistic for the reason given throughout this

Table 16: CTC-judge relative count error, median [95% bootstrap CI] at $k \geq 6$ (`data/results/count_error.json`). "Sep." marks a model whose repeated-item interval is disjoint from its own control's.

Model	Repeated (%)	Control (%)	Sep.
Llasa-1B	-16.1 $[-31.3, -12.5]$, n=94	0.0 [0.0, 0.0], n=104	yes
Llasa-3B	-12.5 $[-25.0, -6.3]$, n=101	0.0 [0.0, 0.0], n=105	yes
Llasa-8B	-16.7 $[-25.0, -12.5]$, n=103	0.0 [0.0, 0.0], n=106	yes
Qwen3-TTS-0.6B	-8.3 $[-12.5, -3.6]$, n=108	0.0 [0.0, 0.0], n=108	yes
Qwen3-TTS-1.7B	0.0 [0.0, 0.0], n=108	0.0 [0.0, 0.0], n=108	no
XTTS-v2	-15.6 $[-16.7, -12.5]$, n=108	0.0 [0.0, 0.0], n=108	yes
Panel (pooled)	-8.3 $[-12.5, -8.3]$, n=622	0.0 [0.0, 0.0], n=639	5/6
XTTS-v2, no penalty (abl.)	-58.3 $[-83.3, -31.3]$, n=107	-66.7 $[-75.0, -60.4]$, n=108	no

document (Section 6, Section 8.5) and is discussed on its own terms where it recurs in Section 8.5.

9.4 An ASR-free attempt, and why it does not replicate

A recogniser-based metric, however validated, is still a transcript-based metric, and the natural next question is whether the dissociation survives when the transcript is removed from the criterion entirely. `analysis/duration_only.py` asks exactly this: an item is `duration-ok` when its duration ratio falls in $[0.70, 1.45]$ (the same acceptance band `score_counts.py` uses, $\pm 30\%/45\%$ around the calibrated expected duration for that k) and it is not flagged degenerate by the audio-level detectors – neither of which consults any transcript. This is deliberately a *weaker* criterion than the conjunctive one: a model that babbles for approximately the right length without saying anything coherent passes it. That weakness is the point – duration cannot be inflated by an ASR bias it never looks at, so if the repeated-vs-control gap survived here, it could not be attributed to the judge at all.

Table 17: Fraction `duration-ok` at $k \geq 6$, Wilson 95% CI (`data/results/duration_only.json`).

Model	Repeated (%)	Control (%)
Llasa-1B	41.7 [32.8, 51.1]	40.7 [31.9, 50.2]
Llasa-3B	39.8 [31.1, 49.2]	21.3 [14.6, 29.9]
Llasa-8B	36.1 [26.0, 47.6]	22.2 [14.2, 33.1]
Qwen3-TTS-0.6B	80.6 [72.1, 86.9]	69.4 [60.2, 77.3]
Qwen3-TTS-1.7B	89.8 [82.7, 94.2]	74.1 [65.1, 81.4]
XTTS-v2	69.4 [60.2, 77.3]	72.2 [63.1, 79.8]
Pooled panel	60.9 [57.0, 64.7], n=612	51.6 [47.7, 55.6], n=612

It does not replicate. Pooled, repeated items pass the duration criterion *more* often than controls (60.9% vs. 51.6%, the wrong direction for the paper's claim), the two intervals are not disjoint, and only one of six checkpoints (XTTS-v2, 69.4% vs. 72.2%, itself not a resolved difference) shows

the gap pointing the expected way. We report this plainly as a negative result and explain, rather than explain away, why: the duration band is far too coarse to resolve the failure this paper measures. At `DUR_LO`= 0.70, `DUR_HI`= 1.45, the acceptance window at k=8 spans approximately 5.6 to 11.6 repetitions' worth of expected duration – roughly 2.4 fewer or 3.6 more repetitions than the true count, built deliberately wide to absorb ordinary speech-rate variation (Section 4). The CTC-judged deficit this paper reports at k=8 specifically is a median relative error of −12.5% to −18.8% across the panel's five affected checkpoints (Section 9.3), i.e. roughly a one-to-one-and-a-half-repetition miscount – comfortably inside a band with two to three-and-a-half repetitions of slack in either direction. A model that drops one or two repetitions out of eight sounds, in total duration, indistinguishable from one that renders all eight at a slightly different pace; duration was never going to see an effect this small at this magnitude, independent of anything about the judge. We therefore report `duration_only.py` as a *bound*, not a replication: it establishes that a purely acoustic, transcript-free signal lacks the resolution to confirm *or* refute the dissociation at the scale of miscount involved, which is precisely why a recogniser that can count – validated, in Section 9.2, to do so exactly – is necessary in the first place, rather than a convenience.

9.5 Beyond this paper

The mechanism behind Section 9.1's finding is not particular to this project's stimuli: an autoregressive recogniser with a language-model prior is, structurally, a poor instrument for counting repeated speech, because it is subject to a version of the same repetition-suppression dynamics whenever the material it is asked to transcribe is itself periodic. That instrument choice is not idiosyncratic to this project, either. Whisper is, at the time of writing, one of the most widely used off-the-shelf recognisers for evaluating text-to-speech output in general, including in work on the AR-TTS stability and hallucination failures this paper's own introduction cites as motivation [**?**, **?**, **?**]: transcribe-then-score against a reference is standard practice for measuring word error rate, and wherever an evaluation extends that same pipeline to counting or flagging repetition-type failures specifically, it inherits the risk this section quantifies. We have not audited any other paper's evaluation code or re-run its pipeline, and we make no claim that a specific published repetition or looping number is wrong – only Whisper large-v3, on our stimuli, under our decoding configuration, is measured here. What generalises is the mechanism, not a verdict on any other result: any repetition or looping metric built by transcribing with an autoregressive, language-model-equipped recogniser and thresholding or exact-matching the transcript is exposed, in principle, to the bias Section 9.1 measures, in proportion to how periodic the scored material actually is. The appropriate response is not to distrust every such number on priors, but to check it directly – the concatenative, exact-ground-truth construction used throughout this section needs no new data collection, only one verified rendering and a splice, and is cheap enough to run as a standard validity check before treating any transcript-based repetition count as neutral with respect to the phenomenon it is being used to measure.

9.6 The judge on real generated audio

The audit in Section 9.1 used concatenative audio with known ground truth, which establishes what each recogniser does to periodic speech but not what it does to *our models'* speech. This section validates the CTC judge on the generated audio the paper actually scores. Four checks, run by `analysis/ctc_field_validation.py`; the raw results are in `data/results/ctc_field_validation.json`.

Whisper's failure on real audio is worse than de-duplication. Comparing the two judges where models are reliable (control items at all k, repeated items at $k \leq 3$) gives 83.0% exact agreement over $n = 1729$. Divergence grows with k on repeated items only, reaching a mean absolute difference of roughly 180 words — far too large to be a counting disagreement. A ground-truth-free plausibility check settles what is happening. Dividing transcript length by audio duration gives an implied speaking rate, and human speech does not exceed about four words per second:

k	n	CTC median wps	Whisper median wps
1	296	2.07	2.07
2	347	2.15	2.01
3	168	2.31	2.23
4	381	2.08	1.91
6	202	2.23	2.08
8	168	2.26	2.16
12	168	2.27	2.24
16	167	2.31	2.57
24	102	2.34	12.30
32	102	2.07	10.71

CTC stays flat and physically plausible across the whole ladder. Whisper tracks it to $k \leq 16$ and then rises to 10.7–12.3 words per second, which is not a transcription of anything. Inspecting those transcripts shows fluent fabricated text — the clean $k = 1$ template returned for garbled audio, "Thank you." for near-silence. The original audit characterised Whisper's bias as de-duplication, i.e. undercounting; on real audio it also overcounts by hallucinating. Either way it is the wrong instrument, and the decision to replace it is if anything better supported than when it was made.

Perturbation stability. Re-transcribing 72 items under mild noise (30 dB SNR) and under a 16k→15.5k→16k resampling round trip moves the count by a mean of 0.12 and 0.03 counts respectively on repeated items, with 88% unchanged under noise. A re-implementation of the transcription path agreed with the stored production transcripts on 72/72 items. The judge's count is not noise-fragile.

A second, architecturally independent recogniser. HuBERT-large-ls960-ft shares the CTC output structure but not the pretraining objective (masked cluster prediction rather than contrastive), so its errors are not expected to correlate with the primary judge's. An early n=60 stratified check reported 78.3% exact agreement, mean absolute difference 0.47 counts, and the primary judge higher in 11 of 13 disagreements. **That n=60 figure is superseded**: §14.1 re-runs the same comparison, plus two further recognisers, over the full panel population (n=1559 paired items rather than 60) and finds a narrower margin — the primary judge is still the more generous of the two, but in roughly two-thirds rather than 11 of 13 (85%) of disagreements. See §14.1 for the full numbers and for why the narrower margin does not change the paper's conclusion.

The limitation this surfaced, which the synthetic audit could not. Two of those disagreements expose a real mechanism. HuBERT rendered `wr_very_t1_k12` — twelve copies of *very* — as `VERYVERY VERRYYVERYVERY`, and `wr_no_t2_k06` as `NA NA NA NAU NA`. CTC emits a blank between repeated symbols to keep them apart; when a model produces identical words back-to-back with

little or no gap, that mechanism can fail and adjacent repeats merge. The synthetic audit could not have found this, because it inserted 120 ms of silence between every repeat.

This matters for how the paper's numbers should be read, and it cuts in our favour, which is precisely why it belongs here. Blank-collapse depresses counts on *same-word* runs and not on distinct-word controls — the perturbation check found controls essentially perfectly stable — so it acts along the same axis as the effect we report. It cannot manufacture the repeated-versus-control gap out of nothing, since the control side is unaffected and its exact-rate is 94.3%, but it can inflate the gap's magnitude. We therefore treat the *direction* and *existence* of the deficit as established and the precise magnitude at $k \geq 16$ as approximate, and no claim in the paper rests on that magnitude. Carrier-word WER over a stratified sample is 0.272, most of it traceable to genuinely degraded audio that CTC renders as garbage and Whisper renders as fluent invention.

10 The shape of the deficit, and what it does not show

This section documents the analysis that changed the paper's central claim, and the exclusion rules that the numbers in it depend on. It was written after a reviewer observed that a constant *relative* deficit is not what a fixed horizon predicts. They were right, and the check is now part of the paper rather than a response to it.

10.1 Saturating versus proportional

Theorem 1(iv) bounds the largest separable repetition count by a constant N^*. Past that horizon no Lipschitz readout distinguishes counts, so a decoder driven beyond it should render roughly N^* repetitions no matter how many were requested: the rendered count *saturates*. A fixed relative loss is a different claim — it says the decoder still tracks k — and the two are distinguishable with the data we already have.

`analysis/horizon_shape.py` fits the median rendered count above $k = 6$ against three forms: a constant a (the theorem's signature), a proportional bk, and the interpolating $N^*(1 - e^{-k/N^*})$, which rises linearly and then saturates, so it is free to place a horizon wherever the data want one.

model	sat. SSE	prop. SSE	b	soft N^*	best
`llasa1b`	477.5	35.0	0.855	85	proportional
`llasa3b`	464.2	79.3	0.913	119	soft horizon
`llasa8b`	353.5	2.0	0.830	71	proportional
`qwen06b`	499.3	1.4	0.953	338	proportional
`qwen17b`	668.8	5.1	1.107	640	proportional
`xtts2`	378.0	0.8	0.860	90	proportional
pooled	485.3	2.6	0.950	278	proportional

The proportional form wins in 5 of 6 checkpoints and pooled, by two orders of magnitude in residual. The soft-horizon fit, given every opportunity to find a plateau, places N^* at or past the top of the ladder in all six. Pooled median rendered count over requested, by k: 6:0.83 8:0.88 12:0.92 16:0.94 24:1.00 32:0.94. There is no counting horizon within $k \leq 32$.

We draw the conservative conclusion. This does not refute Theorem 1, which is proved and whose premise we never established for these decoders; it establishes that the ladder does not reach the regime the theorem describes. Reporting the proportional deficit as though it were the theorem's signature would have been the error, and the main text now reports the consequence as not observed. The extension ladder in Section 10.3 is the direct test.

10.2 Which rows the paper is allowed to report

Four exclusions, applied by `src/common/population.py` so that every analysis shares one definition. They had previously drifted: the count-error analysis listed one ablation arm where the capacity analysis listed four, so re-scoring the full model list would have folded three repetition-penalty arms into the panel without anything failing.

1. **Ablation arms are not panel members.** `xtts2norp` and the `xtts2rp*` arms are one checkpoint under altered decoding; pooling them would count XTTS-v2 five times.
2. **Degenerate and empty audio has no count**, and is reported as its own rate rather than folded in as a large negative error.
3. **Templates whose vocabulary the judge cannot transcribe.** Our CTC judge renders `okay` as "o k" in 106 of 106 items and `hmm` in 89 of 89; template t2 therefore scored the recogniser's orthography. The criterion is near-total absence, which no decoder behaviour can produce — six checkpoints do not fail on one filler every single time while rendering its neighbours — so it does not require assuming which items the models got right. It is applied to both families, so it cannot favour the control side.
4. **Items cut off by our own token budget.** `hit_cap` marks generations that ran into the harness limit rather than stopping on their own. Their median relative error is -0.44 against -0.08 for the rest: that is our budget truncating audio, not a model failing to count. They were included in earlier drafts; the pooled median is unchanged by their removal, but per-model figures are not.

A fifth correction is a scoring fix rather than an exclusion. `count_units` stopped scanning at the first unit it could not find, which is correct for repeated items — all units are the same word, so failing to find one from a position means failing to find the rest — but wrong for controls, where one mis-transcribed filler voided credit for every correctly rendered filler after it. It now skips. The change is verified bit-identical on repeated items and lifts the pooled control mean error from -0.150 to -0.090; the remaining gap closes under exclusion 3.

10.3 The extension ladder

Since the main ladder cannot separate a distant horizon from no horizon, we ran the same three carriers at $k \in \{48, 64, 96, 128\}$ (`data/stimuli/make_stimuli_ext.py`, `scripts/run_ext_ladder.sh`). Two departures from the main protocol are forced by the longer items and both are recorded rather than absorbed.

First, the token budget is raised from 2048 to 8192. A $k = 128$ item is roughly 55 seconds of speech, far past a 2048-token budget at X-codec2's 50 Hz, and an item cut off by our own budget would be scored as the model failing to count — exactly the artifact exclusion 4 above exists to remove. `hit_cap` is recorded regardless and cap-hit items are excluded from the fit.

Second, XTTS-v2 is not run. Its decoder carries a built-in ceiling of about 602 mel tokens, roughly 26 seconds, which every $k \geq 48$ item exceeds; its counts would be censored by architecture rather than by any horizon, and raising the ceiling would run the model outside the range it was trained on. This is a limitation of the extension, not a result about XTTS-v2.

The control items also depart from the main ladder in a way worth stating, because it corrects a flaw the main set has. Main-ladder control fillers are drawn by cycling an eight-word pool, so controls at $k > 8$ carry period-eight repetition rather than none — tolerable when the contrast is

against period-one repetition, and the dissociation in Section 3.1 of the main paper survives it, but at $k = 128$ a cycled pool of eight would be sixteen full cycles and the control would no longer be a control. The extension pool holds 146 distinct words and is never cycled; the generator asserts this and fails loudly rather than falling back.

10.4 Do the exclusions fall evenly on the two arms?

An exclusion concentrated on one arm could manufacture the contrast it is used to support, so a reviewer asked for the per-condition breakdown. Rates over the panel, before any other filter:

arm	template rule	token budget	degenerate
repeated ($n = 1080$)	16.7%	10.5%	4.4%
control ($n = 972$)	16.7%	5.6%	3.7%

The template rule is exactly balanced, which is structural rather than lucky: template t2 is one of six carriers and contributes equally to both arms, so removing it removes a sixth of each.

The budget rule is not balanced — repeated items run into it about twice as often, which is what a model that loops or over-runs on periodic text should do. The direction matters more than the imbalance: cap-hit items carry a median relative error of -0.44 against -0.08 for the rest, so excluding them removes the *worst* repeated generations and shrinks the very gap the paper reports. Keeping them would make the effect look larger, not smaller.

Degeneracy is near-balanced and small on both arms.

11 Checkpoint-level inference

Round-5 reviewers objected that the paper's evidentiary standard was a set of vote counts — "5 of 6", "4 of 6", "a positive gap in 80% of pairs" — with shifting denominators, no common test and no multiplicity control, while the pooled confidence intervals were computed over *generations*. The objection is correct, and the two answer different questions: pooling generations asks whether the effect is present in this corpus, where the paper's claim is about models of this kind, for which the checkpoint is the sampling unit.

`analysis/checkpoint_level.py` reduces each checkpoint to a single repeated-minus-control difference and does inference across the six. Signs are oriented so that positive means the effect is present.

checkpoint	exact-rate (pts)	median-error (pts)	capacity
`llasa1b`	+78.1	+12.5	+38.5
`llasa3b`	+77.2	+8.3	+46.5
`llasa8b`	+81.4	+16.7	+30.5
`qwen06b`	+92.2	+12.5	+14.0
`qwen17b`	+60.0	+0.0	+8.1
`xtts2`	+71.1	+12.5	+10.7

contrast	mean	95% CI	p	positive
exact-rate gap (pts)	76.7	[68.4, 84.4]	0.031	6/6
median-error gap (pts)	10.4	[5.6, 13.9]	0.062	5/6
capacity gap	24.7	[13.7, 36.5]	0.031	6/6

Three things about these numbers deserve stating rather than glossing.

The signed-rank test has a floor at $n = 6$. Its smallest attainable two-sided p is $2/2^6 = 0.031$, so a perfect result cannot do better and the two contrasts that reach it sit at the ceiling of what six checkpoints can show. We compute the test exactly, enumerating all 2^n sign assignments; the normal approximation is not valid at this n.

The median-error contrast does not reach even that floor. Qwen3-TTS-1.7B's gap is exactly zero, which drops the effective n and gives $p = 0.062$. An earlier draft reported this as "4 of 6 checkpoints", which reads as a weaker version of the same evidence when it is in fact a different and less favourable result. The exact-rate contrast, which the paper now leads with, is the one holding in all six.

No multiplicity correction can rescue these p-values, and we do not ask them to. Three contrasts on the same panel invite a Holm correction, which puts all three at 0.094. That is not a fact about our effects: the exact floor at $n = 6$ is 0.031, so the smallest attainable Holm-adjusted p across three contrasts is $3 \times 0.031 = 0.094$, and *no* six-checkpoint panel can produce a corrected value below 0.05 on this test however large the effect. A panel this size cannot carry significance testing, so the paper rests on effect sizes and their consistency — a 77-point exact-rate gap, present in every checkpoint, with a bootstrap interval nowhere near zero — rather than on thresholds it is structurally unable to clear. Enlarging the panel, not re-analysing it, is what would change this.

Direction generalises; magnitude does not. The between-checkpoint SD of the capacity gap is 16.0 against a mean of 24.7 — a fourfold spread. "Holds in all six" and "holds in all six by comparable amounts" are different claims that a vote count cannot distinguish. The first is supported; the second is not.

11.1 The clustering, modelled instead of counted

Everything above treats the six checkpoints as the sampling unit and either counts votes or runs an exact sign test on the six differences. That answers "does the effect hold in every checkpoint we ran", which is a weaker claim than the one the main paper's Section 3.1 now makes: what is the arm effect for architectures *of this kind*, given that the six checkpoints are really three architecture families (three Llasa scales, two Qwen3-TTS scales, one XTTS-v2), so treating them as six independent replicates overstates the independent evidence twofold. `analysis/hierarchical.py` is the script that answers this properly: a mixed-effects model with architecture as a random effect and checkpoint nested inside it, a hierarchical Bayesian model that partially pools the six checkpoints into the three families, and a specification curve over every forking choice made along the way. Before any of that runs, the script recomputes the per-checkpoint and per-family exact-rate gaps from `src.common.population.panel` and asserts they equal `checkpoint_level.json` and `family_level.json` to machine precision ($\max|\text{diff}| = 0$ on all three checks; see `data/results/hierarchical.json`, key `reproduction`) — the population is not new, only the model of it.

11.1.1 Mixed-effects and cluster-robust models

The outcome is exact/not-exact (0/1), fitted as a linear probability model (`exact_i` $\sim$ `rep_i` + $\log_2 k_i$) so the coefficient on `rep` is directly in percentage points; effects are reported flipped in sign so positive means control minus repeated, i.e. the effect the paper predicts. The primary specification is a crossed-random-effects model with architecture family, checkpoint nested in family, and stimulus template all as random intercepts, plus — the piece that actually matters for a three-family panel — a *family-varying arm slope*: the `rep` coefficient itself is allowed to differ by architecture family, which

is what forces its standard error to be paid for out of three families rather than 982 generations. Statsmodels' `MixedLM` takes one `groups` factor, so the crossed structure is supplied as four variance components (`arch`, `ckpt`, `stim`, `arch:rep`) under a single all-ones group, fit by REML.

That primary model puts the gap at **76.4 points** (mixedlm `arm_effect_pts`= 76.423). Table 18 reports every interval construction the script computes, each with the clustering unit that generates its degrees of freedom.

Table 18: Interval constructions for the arm effect (control − repeated, percentage points), `data/results/hierarchical.json`, key `mixed`. The Wald row is reported for reference and is *not* one of the six intervals the main paper's "all six" refers to, for the reason given in the note below the table.

Construction	n clusters	Estimate	95% interval	Criterion
MixedLM, crossed RE, Wald (reference only)	3 fam.	76.4	[71.9, 80.9]	z
the six constructions the main text's "all six" counts:				
MixedLM, crossed RE, $t(2)$	3 fam.	76.4	[66.6, 86.3]	$t(2)$
OLS, clustered by family	3	76.2	[68.7, 83.7]	$t(2)$
OLS, clustered by checkpoint	6	76.2	[63.2, 89.2]	$t(5)$
Family-level means, t-interval	3	75.4	[65.6, 85.2]	$t(2)$
Cluster bootstrap over families (20k)	3	76.0	[71.1, 79.0]	percentile
Cluster bootstrap over checkpoints (20k)	6	76.2	[67.2, 84.8]	percentile
check, not one of the six (different link function):				
GEE, binomial, clustered by family	3	OR ≈ 96, log-odds	[3.31, 5.83]	$t(2)$

The widest of the six is the checkpoint-clustered OLS interval, [63.2, 89.2], which is the number the main paper quotes; the narrowest is the family-clustered bootstrap, [71.1, 79.0]. Every one of the six excludes zero, and the least favourable lower bound across all six is 63.2 points (`worst_case_lower_bound_pts`). The binomial GEE, included as a check that the linear-probability link is not doing the work, agrees in direction and significance (odds ratio ≈ 96, $t(2)$ interval on the log-odds scale excluding zero) but is not counted among the six because it answers on a different scale, not because it disagrees.

Why the Wald interval is excluded, and why this is not a convenient omission. The MixedLM Wald interval, [71.9, 80.9], is visibly the narrowest interval in the table — narrower than four of the six percentage-point intervals below it, on the same point estimate. That narrowness is an artefact, not a sign of a well-estimated effect. With three families, the family-varying arm-slope variance component is barely identified: REML shrinks its estimated between-family SD to **1.4 points** (`vcomp_sd_pts.arch_x_rep`= 1.37), against an between-family SD of the exact-rate gap of **3.9 points** measured directly from the three family means (`family_t_interval.sd_pts`= 3.95) — roughly a factor of three too small. The Wald interval is built from that shrunk variance component, so it is narrower than the between-family spread actually licenses. This is exactly the situation the script's own inline note describes: "with three families the family-varying arm slope is barely identified and REML shrinks it hard, so this Wald interval is narrower than the between-family spread alone would license; the clustered, family-t and bootstrap intervals below are the widths to quote." The main paper's Section 3.1 already quotes the widest of the six, [63.2, 89.2], as its headline interval; this is why that choice, rather than the tighter-looking Wald interval, is the defensible one. None of the four variance components (`arch`, `ckpt`, `stim`, `arch:rep`) sit exactly on the zero boundary in this fit (`vcomp_on_boundary` is `false` throughout), so the shrinkage here is a matter of degree — how much the REML estimate trusts three data points to locate a variance

— not a degenerate fit.

11.1.2 A hierarchical Bayesian model

Partial pooling is the direct answer to "six non-independent checkpoints": rather than either treating them as six independent replicates or collapsing each family to one number and discarding the within-family precision, a hierarchical model is told about the nesting and returns a posterior whose width reflects it. The model is binomial on the logit scale: each checkpoint's control-arm and repeated-arm success counts ($y_{\text{ctl}} \sim \text{Binom}(n_{\text{ctl}}, \sigma(\eta_{\text{ctl}}))$, similarly for `rep`) are generated from a control-level logit $\eta_{\text{ctl}} = \alpha + \sigma_{\text{fam}}\, z_{\text{ctl}}^{\text{fam}} + \sigma_{\text{ckpt}}\, z_{\text{ctl}}^{\text{ckpt}}$ and a gap logit $\text{gap} = \Delta + \sigma_{\text{fam}}^{\Delta}\, z_{\Delta}^{\text{fam}} + \sigma_{\text{ckpt}}^{\Delta}\, z_{\Delta}^{\text{ckpt}}$, with $\eta_{\text{rep}} = \eta_{\text{ctl}} - \text{gap}$: every checkpoint's control level and its arm gap each carry a family offset and a checkpoint offset nested inside that family, non-centred for sampling. Priors: $\alpha, \Delta \sim \mathcal{N}(0, 5)$ on the logit scale (near-flat over the range the data can move them), and all four group-level scales $\sigma \sim \text{HalfNormal}(1.0)$ (`data/results/hierarchical.json`, key `bayes.priors`).

Sampled twice, independently. The primary fit is NUTS via `numpyro` (4 chains, 1500 warmup / 3000 draw); the cross-check is a hand-rolled blocked adaptive random-walk Metropolis sampler that imports nothing but `numpy`, run for 4 chains of 20000 warmup / 60000 draws thinned by 10. Diagnostics for both (split-$\hat{R}$ and bulk ESS, computed by this project's own implementation for the hand-rolled chains and by `numpyro.diagnostics.summary` for NUTS):

Engine	$\hat{R}_{\max}$	$\text{ESS}_{\min}$	Converged ($\hat{R} < 1.01$, ESS> 400)
NUTS (numpyro), primary	1.0005	3825	yes
Hand-rolled Metropolis, cross-check	1.0255	296	yes

The two engines' posterior means agree to within 0.02 log-odds, 0.12 points on the panel gap, and 0.33 points on the new-family predictive gap (`bayes.crosscheck.max_abs_diff`), well inside the tolerances the script requires (`agrees=true`) — two independent implementations of the sampler, not one trusted alone.

Table 19: Posterior arm-effect gap (percentage points, control − repeated), partially pooled, `data/results/hierarchical.json`, keys `bayes.per_family_gap_pts` and `bayes.per_checkpoint_gap_pts`. All $P(\text{gap} > 0) = 1.000$ to the precision shown.

Level	Unit	Mean	95% credible interval
Family	Llasa	79.3	[73.4, 84.4]
Family	Qwen3-TTS	74.1	[67.7, 79.9]
Family	XTTS	73.7	[63.5, 82.2]
Checkpoint	`llasa1b`	79.5	[70.5, 86.9]
Checkpoint	`llasa3b`	77.2	[67.0, 85.5]
Checkpoint	`llasa8b`	81.0	[72.7, 88.0]
Checkpoint	`qwen06b`	87.6	[80.1, 93.8]
Checkpoint	`qwen17b`	60.5	[50.4, 70.0]
Checkpoint	`xtts2`	73.7	[63.5, 82.2]

Two further pooled summaries, both derived from the same posterior draws but answering different questions. The "typical checkpoint of a typical family" — random effects held at zero — gives a *panel* gap of 74.9 points, [41.5, 89.4] (`bayes.panel_gap_pts`); its lower bound is substantially

below the family- and checkpoint-level intervals above because it marginalises over the between-family variance the panel-level posteriors condition away. The quantity the main paper actually leads with is different again and answers the question three families cannot answer directly: drawing a *new* family's offsets from the fitted population distribution (rather than conditioning on one of the three observed families) gives the posterior *predictive* gap for an architecture outside the panel, **65.3 points**, $[5.9, 93.3]$, $P(\text{gap} > 0) = 0.996$ (`bayes.new_family_gap_pts`) — wide enough to place the effect with high probability, not narrow enough to exclude a fourth architecture with no deficit at all. The four group-level scales at their posterior means, on the logit scale, are $\sigma_{\text{family}} = 0.97$, $\sigma_{\text{checkpoint}} = 0.45$, $\sigma_{\text{family},\Delta} = 0.72$, $\sigma_{\text{checkpoint},\Delta} = 0.81$ (`bayes.sigma`): the between-*family* scale on the gap (0.72) is comparable to the between-*checkpoint* scale on the gap (0.81), i.e. with only three families and six checkpoints the data do not sharply separate "families differ" from "checkpoints within a family differ," which is exactly the uncertainty the new-family predictive propagates forward.

11.1.3 Prior sensitivity, both directions

With three families, the prior on the group-level scales is doing real work, so it is reported as part of the result rather than relegated to a robustness appendix. Table 20 refits the identical model at HalfNormal(0.5) (tighter than the reported prior, favourable to a narrow interval) and HalfNormal(3.0) (much wider, unfavourable), against the reported HalfNormal(1.0).

Table 20: Prior sensitivity on the four group-level scales σ, new-family predictive gap (points), `data/results/hierarchical.json`, key `bayes.prior_sensitivity` (the reported prior's row is `new_family_gap_pts` from Table 19's text).

Prior on σ	Mean	95% CrI	$P(\text{gap} > 0)$
HalfNormal(0.5) — tighter, favourable	71.7	[26.6, 90.9]	1.000
HalfNormal(1.0) — reported	65.3	[5.9, 93.3]	0.996
HalfNormal(3.0) — much wider, unfavourable	52.3	[−1.6, 96.1]	0.956

Stated as unfavourably as the data allow: under the widest prior on the group-level scales, the 95% credible interval for the predictive gap on an architecture family outside the panel touches zero, $[-1.6, 96.1]$, with $P(\text{gap} > 0) = 0.956$ — a 95% interval that no longer excludes no effect, and a one-in-twenty-five posterior chance (by this prior's lights) that a fourth architecture family shows no deficit or reverses it. This is not a failure of the model; it is the honest consequence of having three families to locate a between-family variance with. What the data establish, on every prior tried, is a large effect in every family actually observed and a posterior probability of a positive gap that is high under every prior and drops only under the least defensible one; what they do not establish, and cannot with $n = 3$ families, is certainty that a fourth, unobserved architecture would show it too.

11.1.4 The specification curve

The final piece asks the question a single preferred number cannot: how much does the headline gap move under the forking choices made along the way. Six axes are crossed exhaustively: which exclusion rule is active (5 levels, from the panel default to every rule relaxed at once), the k threshold (5 levels), cycled versus never-cycled (aperiodic) controls, one seed versus all three, pooled/per-checkpoint/per-family aggregation, and the outcome definition (exact-rate gap versus median-relative-error gap). The full cross is $5 \times 5 \times 2 \times 2 \times 3 \times 2 = 600$ cells; 180 are

dropped as duplicate populations (the never-cycled control arm only exists for $k = 12$–32, so every $k_{\min} < 12$ cell under `control=aperiodic` selects an identical row set and would otherwise pad the curve with copies of the same specification), leaving **420 specifications**, none infeasible (`spec_curve.n_specifications`, `n_infeasible`, `n_duplicate_populations_collapsed`). The population underlying every cell that overlaps `exclusion_sensitivity.py`'s five exclusion arms and `aperiodic_controls.py`'s never-cycled re-generation is checked to reproduce those scripts exactly (`spec_curve.reproduces`, max |diff| = 0 on all six shared cells) before any of the 420 is trusted.

The two outcome definitions are on different scales and are reported separately in Table 21 rather than pooled into one range: the exact-rate gap (percentage points of exact-match rate, 210 specifications) and the median-relative-error gap (percentage points of median count error, 210 specifications, a much smaller quantity by construction — see Section 3.1 of the main paper's own median-error numbers, which sit around 10–12 points against exact-rate's ∼77).

Table 21: Specification curve, `data/results/hierarchical.json`, key `spec_curve`. n is the number of specifications under that outcome definition.

Outcome	n	Range (pts)	Median	Positive	Zero / negative
Exact-rate gap	210	[34.4, 86.4]	70.5	210/210	0 / 0
Median-error gap	210	[0.0, 19.5]	8.3	197/210	13 / 0
Both outcomes combined	420	—	—	407/420	13 / 0

Read plainly: the exact-rate gap — the outcome the main paper leads with — runs **34.4 to 86.4 points** across its 210 specifications and never changes sign. The median-error gap, a smaller and noisier quantity by construction, is positive in 197 of 210 and exactly zero (never negative) in the remaining 13 — the same zero-but-never-reversed pattern already reported above for `qwen17b`'s single median-error checkpoint. Pooling both outcome definitions into the one number the main paper states — **0 of 420 reverse the gap's sign** — is accurate for sign-preservation, and is how that count should be read; the **34.4–86.4** range that accompanies it in the main text describes the exact-rate outcome specifically, not the smaller-scale median-error one, and a reader reconstructing the claim from `hierarchical.json` alone should not average the two outcomes' ranges together.

Which fork owns the spread. Table 22 reports the median exact-rate gap at every level of every axis (`spec_curve.axis_influence_exact_rate`), and the spread within each axis — the single most informative summary of a specification curve, because the raw min–max range conflates every axis at once.

Table 22: Which forking choice moves the exact-rate gap, and by how much (median gap at each level, percentage points). Sorted by spread, descending.

Axis	Spread (pts)	Median gap by level
k threshold	22.8	$k\geq1$: 58.0, $k\geq6$: 74.7, $k\geq8$: 78.3, $k\geq12$: 80.7, $k\geq16$: 69.4
Exclusion rule	9.7	panel: 75.8, +templates: 68.8, +cap-hits: 76.7, +degenerate: 75.7, everything: 67.1
Aggregation level	2.3	pooled: 70.6, checkpoint: 71.3, family: 69.1
Control construction	2.3	cycled: 72.0, aperiodic: 69.7
Seed subset	1.1	all seeds: 69.7, seed 0 only: 70.8

The k threshold owns most of the spread by a wide margin — unsurprising, since $k_{\min} = 1$ pulls

in the sub-horizon range where the two arms have not yet dissociated (Figure 1(a) of the main paper: below $k = 6$ every model is essentially exact in both arms), which mechanically compresses the gap toward zero rather than reflecting a fragility in the effect above the horizon. The exclusion rule is a distant second; which controls are used, which aggregation level, and which seed subset move the estimate by at most a few points each. The single weakest specification in the whole curve is **34.4 points** — every exclusion rule relaxed at once (`exclusions=everything`), $k_{\min} = 1$, cycled controls, seed 0 only, family-level aggregation ($n = 684$; `spec_curve.worst_specification_exact_rate`) — the combination that simultaneously includes the sub-horizon range, keeps every degenerate and cap-hit generation, and aggregates at the coarsest level. Even there the gap is 34 points and positive.

12 Does dilution select which generations fail?

Lemma 1 is proved and its premise is measured; Assumption 2 is neither. A reviewer asked the consequent question: if attention dilution alone accounts for the behaviour, the contraction half of the theory is idle and the honest theory is the smaller one. The test came out against the lemma rather than for it.

Both dilution and the deficit grow with k, so their unconditional correlation is guaranteed and uninformative. We therefore correlate them *within* each (model, k) cell and pool the per-cell correlations by Fisher z weighted by cell size. Three flatness measures are used, fixed in advance because "flat attention" has no single operationalisation and choosing one after seeing results would be selection: the largest single occurrence's share, the block entropy, and the deviation from uniform, each signed so that higher means flatter.

The relative count error is negative when a model undercounts, so Lemma 1 — under which flattening is what destroys the decoder's ability to tell occurrences apart — predicts a *negative* correlation.

flatness measure	repeated	control
`attn_share_max`	+0.59 [+0.35, +0.75]	+0.01 [-0.34, +0.37]
`attn_block_entropy`	+0.43 [+0.16, +0.65]	+0.17 [-0.20, +0.49]
`attn_unif_dev`	+0.55 [+0.30, +0.72]	+0.05 [-0.31, +0.39]

All three run positive on repeated items with intervals excluding zero: within a cell, the items whose attention is *flattest* are counted *best*. The matched controls sit at zero on all three, which is the sanity check — a control has no repeated block over which to dilute, so an association there would have indicated something generic about item difficulty instead.

What this does and does not establish. It does not rescue Assumption 2; nothing here measures contraction. It does not falsify Lemma 1, a theorem about softmax that remains true and that continues to hold in aggregate — attention over repeated spans is near-uniform and its entropy grows with $\log k$. What it removes is the item-level reading: dilution is not what picks out which generations fail. Causation in the reverse direction is not established either. A decoder that has stalled on one occurrence would produce exactly this peaked profile as a *consequence*, and a correlational design cannot separate that from a cause. Only an intervention on the attention profile could, and that is the experiment we would run next.

12.1 Probing past the horizon

The retraction above says the probe discriminates nothing. That is true of the probe as we ran it, and the reason is a scope error worth separating from the result: Theorem 1(iv) forbids an L-Lipschitz readout from separating counts *past* N^* and says nothing below it, while every probe in this paper was fitted on the main ladder, $k \leq 32$, at or below the fitted saturation scale of about 30. It was operating where the theorem permits success.

So we ran the missing experiment. The $k = 48 \ldots 128$ items were regenerated with hidden-state capture and the same ridge probe applied, leave-one-template-out, changing only the range.

states from	probe MAE	constant-predictor MAE	R^2
$k = 2 \ldots 32$ (below N^*)	0.63	1.12	+0.62
repeated, $k = 48 \ldots 128$ (past N^*)	0.50	0.50	-0.02
control, $k = 48 \ldots 128$ (past N^*)	0.43	0.50	+0.26

Why the constant predictor is the yardstick. R^2 falls when the target varies less, and $k \in \{48, \ldots, 128\}$ spans about a third of the $\log_2$ range that $k \in \{2, \ldots, 32\}$ does, so a low R^2 past the horizon would prove nothing on its own. A predictor that ignores the states entirely and always answers the mean achieves 0.50; the trained probe achieves 0.50. It has learned nothing about the count. Below the horizon the same probe beats its own trivial baseline by a wide margin.

The range control. The argument above is still an argument, and a reviewer is entitled to answer that $\log_2$ range is not the only thing that changed. So we ran the control that settles it empirically. Twelve *control* items — aperiodic carriers, never-cycled vocabulary, matched on k and on template — were generated with the same instrumentation over exactly $k \in \{48, 64, 96, 128\}$ and probed identically. The theorem says nothing about them: no periodic conditioning, so no contraction argument, so no horizon.

The probe recovers the count from those states (MAE 0.43 against 0.50 for the constant predictor, R^2 +0.26) while failing on the repeated states over the identical range. Whatever the probe loses past $k = 48$, it is not dynamic range, because the range is the same on both arms. This is the cleanest single result in the paper: the failure tracks periodic conditioning, not sequence length and not target variance.

Had the control arm also failed, we would have reported the repeated-side null as uninterpretable; `analysis/probe_past_horizon.py` decides which of the two sentences to print from the measured MAEs rather than leaving the choice to us after the fact.

The panel doubled: three checkpoints to five. Everything above is Llasa-1B. We ran the identical pipeline — same items, same k, same probe, same folds — on Llasa-3B and Llasa-8B, and then, once the two Qwen3-TTS checkpoints had been instrumented past the horizon in the same way, on Qwen3-TTS-0.6B and Qwen3-TTS-1.7B. Every checkpoint now also has its own control arm at $k = 48 \ldots 128$; the first version of this section had one, on Llasa-1B, and generalised the range argument to the rest by analogy. It no longer has to:

checkpoint	arm	probe MAE	constant MAE	R^2
Llasa-1B	repeated $k = 2 \dots 32$	0.63	1.12	+0.62
Llasa-3B	repeated $k = 2 \dots 32$	0.86	1.12	+0.41
Llasa-8B	repeated $k = 2 \dots 32$	0.75	1.12	+0.52
Qwen3-TTS-0.6B	repeated $k = 2 \dots 32$	0.79	1.12	+0.47
Qwen3-TTS-1.7B	repeated $k = 2 \dots 32$	0.79	1.12	+0.47
Llasa-1B	repeated $k = 48 \dots 128$	0.50	0.50	−0.02
Llasa-3B	repeated $k = 48 \dots 128$	0.49	0.50	−0.02
Llasa-8B	repeated $k = 48 \dots 128$	0.43	0.50	+0.19
Qwen3-TTS-0.6B	repeated $k = 48 \dots 128$	0.37	0.50	+0.34
Qwen3-TTS-1.7B	repeated $k = 48 \dots 128$	0.45	0.50	+0.15
Llasa-1B	control $k = 48 \dots 128$	0.43	0.50	+0.26
Llasa-3B	control $k = 48 \dots 128$	0.41	0.50	+0.33
Llasa-8B	control $k = 48 \dots 128$	0.29	0.50	+0.56
Qwen3-TTS-0.6B	control $k = 48 \dots 128$	0.47	0.50	+0.10
Qwen3-TTS-1.7B	control $k = 48 \dots 128$	0.44	0.50	+0.20

All five decode the count below the horizon. Past it, Llasa-8B's probe beats the constant predictor by 0.86 of its MAE — to two decimals exactly the margin its own *control* arm achieves, so on that checkpoint periodic conditioning past the horizon does nothing to the probe at all. Both Qwen checkpoints beat their constant predictor by a wider margin still (0.74, 0.90); the paragraph after next is why that is not the good news it looks like.

A threshold should not decide this, so we report both sides of it, tie band marked. Llasa-3B's ratio against the constant predictor is 0.99: on the "keeps the count" side of the threshold by six thousandths of a $\log_2$ unit over twelve items, which is not a margin. Reading that as recovery would be as misleading as rounding it the other way, so `probe_past_horizon.py` calls anything within ±0.02 of 1 indistinguishable from the constant predictor and says which checkpoints those are.

checkpoint	MAE ratio (repeated)	reading
Llasa-1B	1.01	indistinguishable (±0.02 tie band)
Llasa-3B	0.99	indistinguishable (±0.02 tie band)
Llasa-8B	0.86	recovers the count
Qwen3-TTS-0.6B	0.74	recovers the count
Qwen3-TTS-1.7B	0.90	recovers the count

checkpoint	MAE ratio (control)
Llasa-1B	0.86
Llasa-3B	0.83
Llasa-8B	0.59
Qwen3-TTS-0.6B	0.95
Qwen3-TTS-1.7B	0.89

Every control arm sits below 1, several by a wide margin, and none inside the tie band: on its own data, every checkpoint that lost or tied the count on the repeated arm has narrowness ruled out directly, not by analogy with Llasa-1B. The range rebuttal generalises across the whole panel.

By the strict threshold the theorem's prediction holds in 1 of 5; by effect size, 2 of 5 checkpoints show nothing recoverable past the horizon and 3 clearly do. *We quote 1 of 5*, the less favourable of the two, and record the other here so a reader can weigh it rather than discover it. Widening the panel made the reading worse, not better: it was 1 of 3 with three checkpoints and stays 1 of 5 with five, so the two Qwen checkpoints did not rescue the theorem's prediction — they extended the pattern against it. That is reported at the stricter count for the same reason the previous paragraph is.

So the theorem's conclusion holds in one of the five checkpoints we could test it on. We report it that way rather than quoting Llasa-1B and calling the others future work: a prediction that survives one case in five is a weaker basis than one in three was, and we do not round it up. The range control still does its job — it says that *where* the count is lost, narrowness is not why — but it cannot make a one-of-five result into a panel one.

The checkpoints favour opposite accounts, and scale does not predict which. This is worth stating more sharply than "a failure to replicate". The rival to Theorem 1 is the dissociation account [**?**], on which the count survives in the states and only the output policy fails. That account predicts a probe *can* still read the count past the horizon; Theorem 1(iv) predicts it cannot. Llasa-1B's result is what Theorem 1 predicts and the dissociation account does not; Llasa-8B's, Qwen-0.6B's and Qwen-1.7B's are what the dissociation account predicts and Theorem 1 does not; Llasa-3B's sits between them, close enough to the constant predictor to favour neither. So the experiment is diagnostic — it is the one measurement in this paper that could have chosen between the two — and it split, more sharply against Theorem 1 than before. A slogan that looked plausible at three checkpoints does not survive the fourth and fifth: within Llasa, the ratio falls with scale (1.01, 0.99, 0.86), which reads as "a bigger model keeps a persistent counter longer"; within Qwen it *rises* with scale (0.6B 0.74, 1.7B 0.90), the smaller checkpoint being the one whose states the probe reads best. Two families point opposite ways on the same axis, so scale is not the variable doing the work, and we do not tell a scale story here.

Two further readings we cannot separate with five checkpoints. Llasa-8B and both Qwen checkpoints may have a horizon past $k = 128$, which would make this a range problem again and not a failure; or the two 1B/3B nulls may be the accident. Distinguishing them needs the ladder pushed further, which token budgets do not currently allow on any of the three.

This remains the only place any of our measurements test the theorem's conclusion rather than its consequences. It does not establish the premise — nothing here measures q — and it does not make the account causal.

The two Qwen ratios are not evidence about the representation at all. A predictor given only the log length of the generated trajectory — no hidden states, nothing the theorem is about — beats the state probe on both Qwen checkpoints, while losing on all three Llasa checkpoints:

checkpoint	length-only MAE	state-probe MAE
Llasa-1B	0.61	0.51
Llasa-3B	0.56	0.49
Llasa-8B	0.45	0.43
Qwen3-TTS-0.6B	0.30	0.37
Qwen3-TTS-1.7B	0.36	0.45

(repeated arm, $k = 48 \ldots 128$; the length-only predictor is a ridge fit on nothing but log of the number of generated tokens, same folds as the state probe.) On Llasa the state probe beats a bare length count by a comfortable margin on every checkpoint; on Qwen, length alone beats the state

probe. The cause is visible in the generation record rather than being a puzzle: the probe applies no cap-hit filter, so budget-truncated trajectories sit in every cell of the table above, and they are far more common on the repeated arm (1–6 of 12 items per checkpoint) than on the matched control (0–1 of 12). Qwen3-TTS-0.6B is the worst case — 6 of its 12 repeated items run all the way to the 8192-token budget, concentrated at the high-k end of the ladder — and a truncated trajectory's length is not independent of k: the more repetitions requested, the more likely the model is still going when the budget cuts it off, so "how long it babbled" carries k on its own, with no count recovered from the state at all. This is exactly the count/length confound the main discussion now flags as a measurement rather than a caveat. It is why the two Qwen ratios in the table two paragraphs up cannot be read as "the count survives in Qwen's states": a predictor that never looks at a hidden state does at least as well. The Llasa checkpoints are not exposed this way — their probes beat the length-only baseline on all three — so the representational reading of *their* ratios is not undermined by this confound.

Which checkpoints, and why not the rest. One limit remains, and it is unchanged. XTTS-v2 cannot be run at all: its 602-mel-token ceiling censors every $k \geq 48$ item outright, so there are no post-horizon states to probe. Every other checkpoint has now been probed past the horizon; the earlier version of this section left the Qwen3-TTS pair out only because the instrumented pass had not yet been run on them, at roughly a minute per item, and we said so plainly rather than implying a capability limit where there was none. That gap is closed here.

Limits. Five checkpoints, 12 repeated items each plus 12 controls, one seed, three cross-validation folds — and they disagree, as above. Two engineering notes for anyone repeating it: the instrumented pass uses eager attention to capture attention weights, which materialises a $T \times T$ matrix per layer and runs out of memory at these sequence lengths — the count probe needs hidden states only, so SDPA suffices and the run completes; and the extension items must be regenerated rather than reused, since the original extension sweep was run without instrumentation. Neither the original nor the widened panel applies a cap-hit filter to the probe items themselves, which is what makes the length-only baseline above informative rather than moot — excluding truncated items outright would need the ladder regenerated a third time, which we have not done.

13 The non-autoregressive baselines

The paper's scope claim is about autoregressive decoders, and round-6 reviewers observed that we had never run one that was not. Rounds 8–10 then observed that one 2021 model could not carry the claim alone. Both were right, and the second baseline is why this section no longer says what it used to.

model	repeated exact (%)	control exact (%)	gap (pts)
Llasa-1B	13.3	91.5	+78.1
Llasa-3B	16.9	94.1	+77.2
Llasa-8B	11.8	93.2	+81.4
Qwen3-TTS-0.6B	7.8	100.0	+92.2
Qwen3-TTS-1.7B	38.9	98.9	+60.0
XTTS-v2	16.7	87.8	+71.1
F5-TTS (2024)	25.6	85.6	+60.0
VITS (2021)	65.6	58.9	-6.7

The two disagree. VITS shows no dissociation; F5-TTS shows one close to the panel's. Neither

has a generation-step recurrence, so *autoregressive* is not the property that separates them, and the architectural reading of the deficit does not survive. Per k band, in points:

	VITS	F5-TTS	AR panel
$k = 2-4$	+6.7	+8.9	-1.0
$k = 6-8$	-10.0	+43.3	+60.0
$k = 12-16$	-13.3	+56.7	+85.7
$k = 24-32$	+3.3	+80.0	+83.4

F5-TTS's gap grows with k as the panel's does. Its durations scale with k (median 2.6 s at $k=1$, 11.6 s at $k=32$), so it is rendering the repetitions rather than truncating, and its controls hold above 83% throughout: the effect is specific to periodicity there too.

VITS's null is not a floor effect. At $k = 6$–8, where the AR panel's gap already reaches 60 points, VITS renders 76.7% of repeated and 66.7% of control items exactly — both far off the floor, and ordered the opposite way. A model pinned at its ceiling of competence would show two low arms, not one high arm and a reversal.

VITS's null is not its stock configuration either. VITS was run once, at defaults, while the AR panel got a repetition-penalty sweep — so its stock duration predictor could be what is immune rather than its architecture. The duration predictor is the component that would have to carry the count (output length is the sum of predicted phoneme durations), so it is the right thing to perturb. Two settings, both well away from stock, chosen to degrade it:

arm	setting	exact rep	exact ctl	gap
stock	—	67.6%	49.1%	−18.5
duration	`noise_scale_duration` 1.6	53.7%	20.4%	−33.3
rate	`speaking_rate` 1.35	61.1%	31.5%	−29.6

The gap is control minus repeated, so the AR panel's +76 means the control is counted right and the repeated item is not. Every VITS arm is negative: the repeated item is counted *better* than its own control, and the repeated-side median relative error is exactly zero in all three.

Both perturbations lower absolute accuracy on both arms, and the control side further. That is a judge effect, not a counting one — a control sentence made of many distinct words has more to lose from faster or noisier speech than a repeated one does — so it inflates the size of VITS's negative gap and says nothing about repetition. The sign, not the size, is what this arm reports.

What it rules out is the specific objection that stock defaults were load-bearing. It does not rule out the broader one that VITS is small and old, which no amount of knob-turning on VITS can address. `python analysis/vits_config.py`.

What plausibly separates them. VITS predicts a duration for each input token and expands the sequence accordingly, so "how many" is carried structurally by the input and never has to be represented. F5-TTS estimates one total duration and denoises the whole mel in parallel, so it must represent how much speech to produce — the same burden an autoregressive decoder carries in its state. On that reading the relevant property is not recurrence but whether the count has to be held internally, and both baselines are consistent with it.

We label this a hypothesis rather than a result. It was formed after seeing the two baselines, it rests on $n = 2$ systems that differ in more than their duration mechanism, and we did not test it. The experiment that would is a duration ablation within one architecture — forcing a per-token

duration model into an otherwise unchanged parallel decoder — and it is the first thing we would run next.

The VITS null still does the judge's work. Two review rounds asked whether CTC blank-collapse manufactures the repeated-versus-control gap by merging adjacent copies of a word. Auditing the recogniser against reference audio cannot settle it, because that audio is not the audio in question. VITS can: it renders the same repeated words from the same stimulus file, transcribed by the same recogniser, and shows no gap. Whatever blank-collapse costs, it does not create the dissociation.

13.1 An intervention: supplying the duration

Everything else in this paper is observational — vary the stimulus, watch the output. This is the one place we intervene on the mechanism we proposed, and it is worth reading for where it fails as much as for where it works.

The proposal was that what separates the two non-AR baselines is whether the model must represent "how many" internally. F5-TTS exposes `fix_duration`, which replaces its own total-duration estimate with a supplied one, so the proposal is testable. The duration supplied is the median of F5-TTS's own *control* renderings at the same k — items it counts correctly — so it is what k units of speech in this carrier take, obtained without looking at how the repeated item came out. A duration read off the repeated item's own output would have been circular.

k	free exact (%)	given exact (%)	free median	given median
6	46.7	80.0	-0.167	+0.000
8	40.0	73.3	-0.125	+0.000
12	33.3	40.0	-0.083	-0.083
16	20.0	26.7	-0.062	-0.062
24	6.7	0.0	-0.167	-0.125
32	6.7	0.0	-0.094	-0.125

Below $k = 12$ the intervention largely works: exact renderings rise from 43.3% to 76.7% ($n = 30$) and the median error goes to zero. At or above it the gain is 0.0 points ($n = 60$) — nothing. Handed the correct total length, the model still cannot place thirty-two repetitions inside it.

That split is the result. The mild end of the deficit is substantially a duration-estimation failure, and the severe end is not: something other than "how much speech to make" is failing once k is large, and it is the same range where the AR panel's own deficit is worst. We take the proposal as supported at low k and refuted as a complete account.

One implementation note, because it cost us a run. `fix_duration` is the length of the reference *plus* the generated speech, not of the generated speech alone. Passing the target directly asks for a total shorter than the six-second reference clip, and the model duly emits almost nothing — thirty empty and fifteen degenerate outputs, which looked briefly like a dramatic negative result. The reference length is measured and added.

Limits. One model, $n = 15$ per cell, one seed triple, and the supplied duration is a median rather than an item-specific target. This establishes a direction, not an effect size.

14 The judge's noise floor

Round-9 reviewers asked for the number that makes every other number in the paper readable, and that we had not computed: the recogniser's error on genuine model output, and hence the

resolution beneath the reported effect sizes. Without it a median count error of -8.3% floats free — a reader cannot tell whether it sits comfortably above the instrument's own uncertainty or inside it.

Two estimates of the floor, both on real generated audio and both in repetitions, which is the unit the recogniser errs in:

source of variation	mean $\lvert\Delta\rvert$ (repetitions)
two recognisers, n=60 stratified subsample (**superseded, see below**)	0.47
two recognisers, full population, all k (n=1559)	0.43
restricted to $k \geq 6$ (n=982)	0.61
restricted to $k \geq 12$ (n=649)	0.86
30 dB additive noise	0.12
resampling round trip	0.03
effect: mean repetitions missing, $k \geq 6$	0.87

The first row is the n=60 estimate this section originally reported; it is kept in the table, struck through in spirit rather than deleted, because the project reports its own corrections rather than quietly editing them. The full population (§14.1, `data/results/independent_judge.json`) gives a comparable floor at $k \geq 6$ (0.61, against the n=60 estimate of 0.47 and 0.79 at the more restrictive $k \geq 12$ the original table used) and a lower one when every k is pooled (0.43), because low-k items — where the two recognisers agree almost perfectly — now dominate the mean.

The honest reading, on the apples-to-apples comparison at $k \geq 6$, is that the per-item effect is thin: $0.87/0.61 = 1.4$ times the disagreement between two independent CTC recognisers (2.0 times if the noise floor is pooled over every k instead, which is the more flattering reading and not the one we lead with). We state it rather than leave it to be discovered, because it bounds what any single item can show.

Two things keep the claim standing, and both are properties of the design rather than assurances. **The control passes through the same judge.** Whatever noise the recogniser contributes, it contributes to both arms, and controls come back exact in 94.3% of generations. A recogniser with indiscriminate half-count noise could not produce that. The claim is a difference between two arms measured by one instrument, not an absolute count.

The disagreements run in our disfavour, though by a narrower margin than first reported. Where the two recognisers differ on the same audio, the n=60 check found ours reporting the *higher* count in 11 of 13 cases (85%); **that figure is superseded**. Over the full paired population the primary judge reports the higher count in 65.9% of all-k disagreements (114/173) and 72.2% of $k \geq 6$ disagreements (83/115), breaking out by arm as 61.2% on the repeated arm (63/103) versus 72.9% on the control arm (51/70) over all k, and 67.1% repeated (53/79) versus 83.3% control (30/36) at $k \geq 6$. Every one of these is still on the side of the primary judge under-, not over-, stating the deficit, so the qualitative conclusion is unchanged: a more accurate recogniser would, on this evidence, report a larger effect, not a smaller one. But the margin is a majority, not a near-consensus, and §14.1 is where the full four-judge replication — of which this two-judge, n=60 comparison was the pilot — is reported in full, including the argument that actually carries the weight of the objection's rebuttal.

What this does not do is rescue any individual item. An item missing one repetition is not, on its own, distinguishable from recogniser noise; the evidence is in the rate across hundreds of generations and in the contrast with the control, not in any single transcript. The released audio sample (§3) is there so that a reader can form their own view of a few of them.

14.1 Four independent judges, the full replication

Every round-9–13 objection to §9 and the paragraphs above reduces to one sentence: the CTC judge was validated on spliced concatenative audio with known ground truth, never on real generated failure audio, and CTC's blank/repeat-collapse rule — which merges adjacent identical tokens — is exactly the confound this paper's headline dissociation would produce if the judge, not the model, were doing the counting. `analysis/independent_judge.py` answers this directly: it re-scores the *same* generated audio, run through the paper's own scoring code (`src/common/score_counts.py`) and population rules (`src/common/population.py`'s `panel()`), with three further recognisers, over the full panel population rather than the n=60 subsample above. Two are CTC judges of comparable capacity and word-error rate to the primary judge but different pretraining, so a disagreement cannot be waved off as "the weak judge is noisy"; the third, Whisper large-v3, is autoregressive and has *no* blank-collapse rule of any kind, so it addresses the confound mechanism as a class rather than one recogniser's idiosyncrasy. The verdict, threshold, and analysis were all pre-committed in the script's docstring before any number existed (reproduced there in full); it resolves to **primary judge confirmed**, retaining 94% of the published gap under the decisive HuBERT arm. Full output: `data/results/independent_judge.json`.

The four judges and the checkpoint-level headline. All four are pinned to a specific Hugging Face revision, recorded in `independent_judge.json`'s `revisions` field rather than typed by hand:

Judge	Revision	Control exact	Repeated exact	Gap
wav2vec2-large-960h-lv60-self (primary)	`bdb1442`	94.3%	18.2%	+76.7
hubert-large-ls960-ft (decisive arm)	`ece5fab`	89.5%	17.9%	+72.3
wav2vec2-large-robust-ft-libri-960h	`5d28473`	82.1%	17.5%	+65.1
whisper-large-v3 (no blank-collapse)	`06f233f`	92.9%	17.1%	+75.5

Every judge's exact-rate gap (control exact rate minus repeated exact rate, $k \geq 6$, paired panel) is **positive in 6 of 6 checkpoints and 3 of 3 families** — the same pre-registered criterion the paper's own headline number is judged by. All four gaps clear the pre-committed confirmation floor of +50 points by a wide margin; the weakest, robust-ft, still retains 85% of the published gap.

Own-panel and paired-panel (the intersection of rows both judges keep after `panel()`) figures differ by only about 0.1 point (e.g. HuBERT: own panel 72.16%, paired panel 72.28%); both are computed and reported in the JSON because `panel()` drops "degenerate" output partly on an empty transcript, which is a property of *which recogniser* scored the clip, not only of the audio. Comparing each judge on its own surviving rows would silently confound a change of scorer with a change of population, so every paired statistic below runs on the intersection, and the small own/paired gap is the price of that discipline, not evidence against it.

XTTS-v2 is the one real casualty. Broken out by checkpoint and family (paired panel, points):

Checkpoint	primary	HuBERT	robust-ft	Whisper
Llasa-1B	78.1	78.6	65.2	69.9
Llasa-3B	77.2	74.8	65.9	76.0
Llasa-8B	81.4	80.3	69.7	71.7
Qwen3-TTS-0.6B	92.2	87.8	82.2	92.2
Qwen3-TTS-1.7B	60.0	57.8	50.0	71.1
XTTS-v2	71.1	**54.4**	57.8	72.2
family: Llasa	78.9	77.9	66.9	72.5
family: Qwen3-TTS	76.1	72.8	66.1	81.7
family: XTTS	71.1	**54.4**	57.8	72.2

XTTS-v2 under HuBERT is the least favourable single cell in the entire replication: 71.1 → 54.4, a **16.7-point drop**, and the only cell of 24 (4 judges × 6 checkpoints) that moves by more than 14 points. It is reported here prominently rather than folded into a mean. Two things keep it from being read as the confound surfacing: it does not reproduce under the third CTC judge (robust-ft: 71.1 → 57.8, a smaller 13.3-point move) or under Whisper (which has no blank-collapse rule at all and actually finds XTTS-v2 marginally *more* consistent, 71.1 → 72.2), and even at 54.4% the checkpoint's gap is still positive and the family mean stays positive across every judge. One checkpoint moving under one judge, in the direction a shared-encoder artifact rather than a shared counting mechanism would produce, is a limitation to state, not a result to explain away, and it is why the pre-committed criterion asks for 6/6 and 3/3 rather than a mean.

The arm-spread argument, which is the decisive one. A judge-side collapse is a claim specifically about the *repeated* arm: the scorer is supposed to be losing repetitions that are really there, while controls are comparatively unaffected. That makes a sharp, falsifiable prediction about what happens when the scorer is changed. Across all four judges the repeated-arm exact rate spans only 17.1–18.2%, a spread of 1.0 point; the control-arm exact rate spans 82.1–94.3%, a spread of 12.2 points — roughly twelve times as wide. Changing the scorer moves the arm the models get *right* and leaves the arm they get *wrong* essentially fixed. A judge-side collapse of repeated material predicts exactly the reverse: the repeated arm should be the one that moves as the scorer changes, and the control arm should be stable. The data show the opposite pattern from the one the objection requires, and this is true independent of any single checkpoint's behaviour, including XTTS-v2's above.

The disagreement asymmetry, reported honestly. The judges do not merely differ in overall accuracy; they disagree with each other 3.3× more on repeated items than on controls (mean $|\Delta|$ 0.65 vs 0.20 repetitions, primary vs. HuBERT), and that disagreement grows with k the same way the reported effect does: repeated-arm exact agreement between the two CTC judges runs 94–97% for $k \in \{4, 6, 8\}$ (97.4%, 94.6%, 94.0% respectively) and falls to 64.4% by k=32, the single worst-agreeing cell in the study. Taken alone this pattern has the effect's own periodicity and could be mistaken for its signature. It is not: the disagreement asymmetry statistic is $A = +0.27$ counts (mean signed primary-minus-independent difference on the repeated arm minus the same on the control arm, pooled over items; the per-checkpoint-averaged version is +0.33, bootstrap 95% interval $[-0.09, +0.73]$), and the confound mechanism the objection describes requires $A \leq -1.0$. The sign is positive, not merely non-negative, meaning that where the two CTC judges disagree at high k the primary judge is, if anything, the more generous one there too. Disagreement growing with k and the disagreement's *sign* arguing against a collapse are both true at once, and reporting only one of them would be reporting the convenient half.

The least favourable readings. Three further cuts of the data, each chosen to give the objection its best chance rather than the paper's: scoring an item exact whenever *either* judge certifies it (the strongest version of "both judges miss different repetitions") retains +76.2 points; restoring template t2, which is dropped from the panel because the primary judge cannot render its vocabulary and whose exclusion was accordingly derived from the primary judge's own blind spot, gives the independent judge's gap as +59.4 (primary's own template-restored gap is +63.3, reported alongside per house style); and the single weakest of the four judges, wav2vec2-large-robust-ft-libri-960h, still reports +65.1. All three clear the pre-committed confirmation floor of +50. (The intersection rule — exact only if *both* judges agree — gives +72.7; it brackets the union figure from the other side but is not the one quoted, since it is not the least favourable reading.)

Two traps, recorded rather than discovered by a reviewer. First: `data/results/behavioural_ctc.csv` (the primary judge's transcripts) was generated at 08:19:49 on the day it was last regenerated, and the commit that fixed `count_units` to *skip* an undeliverable unit rather than stop scoring at it landed 22 minutes later, at 08:41:33. Re-scoring all 5400 rows with the current, fixed code changes zero rows — so there is no drift in any number reported from that file — but this was ordering luck, not a guarantee, and the script now calls `check_count_units_skips()` before reporting anything so the behaviour is asserted rather than assumed. Second, and added *after* the first run of the script rather than pre-committed, which its docstring says explicitly: the Whisper arm's raw signed asymmetry is −53.7, which by the pre-committed rule alone would read as a screaming confound signature. It is not one. Its median signed difference is exactly 0, and the mean is set by 152 runaway-loop items overall (147 of the 745 repeated-arm items) where the two judges' transcripts diverge wildly on a looping generation (one judge reading 441 repetitions on an item where the other reads 5); both judges score every one of those items wrong, so no exact-rate gap depends on them, and the trimmed asymmetry (excluding differences above 10 counts) is −0.27 — above the −1.0 artifact threshold, unlike the raw figure. The pre-committed criterion is evaluated on the trimmed figure for exactly this reason, and the raw figure is reported alongside it rather than silently dropped, because the pre-registered rule says the raw one governs whenever raw and trimmed disagree on which side of the threshold they fall; on the Whisper arm they do disagree (raw trips it, trimmed does not), but on the decisive HuBERT arm both raw (+0.27) and trimmed (−0.03) agree that no confound signature is present, so nothing about the verdict turns on which figure is used.

Verdict. Gate, thresholds, and criteria are those in `analysis/independent_judge.py`'s docstring, fixed before any independent-judge number existed: confirm at $G \geq +50$ points positive in 6/6 checkpoints and 3/3 families with $A > -1.0$; flag a judge-side artifact at $G < +40$, fewer than 5/6 positive checkpoints, or $A \leq -1.0$. On the decisive HuBERT arm, $G = +72.3$, positive in 6/6 checkpoints and all 3 families, $A = +0.27$: **primary judge confirmed**, retaining 94% of the published +76.7-point gap. Quoted at the single least favourable judge available (robust-ft), the gap is still +65.1 points — the number to report if only one may be quoted.

15 Is the deficit the decoding-time repetition penalty?

The strongest decoding-level rival to the whole account, raised in round 8. The three families ship repetition penalties differing by an order of magnitude — XTTS-v2 5.0, Qwen3-TTS 1.05, Llasa none — and a repetition penalty is by construction a mechanism that suppresses repeated tokens. If

it produced the periodicity-specific deficit, the theorem would be explaining something the sampler already explains. We had the data to answer this and had not reported it.

arm	architecture	penalty	rep. exact (%)	ctl. exact (%)	control
`xtts2norp`	XTTS-v2	1.00	15.3	30.9	collapsed
`xtts2rp2`	XTTS-v2	2.00	15.6	92.2	intact
`xtts2rp3`	XTTS-v2	3.00	21.1	91.1	intact
`xtts2`	XTTS-v2	5.00	16.7	87.8	intact
`xtts2rp8`	XTTS-v2	8.00	15.6	84.4	intact
`qwen06brp10`	Qwen3-TTS-0.6B	1.00	10.0	100.0	intact
`qwen06b`	Qwen3-TTS-0.6B	1.05	7.8	100.0	intact
`qwen06brp15`	Qwen3-TTS-0.6B	1.50	10.0	80.0	intact
`qwen06brp30`	Qwen3-TTS-0.6B	3.00	16.7	83.3	intact

Sweeping it changes nothing that matters. On XTTS-v2 the repeated exact-rate holds between 15.6% and 21.1% across a fourfold change in the penalty; on Qwen3-TTS-0.6B, between 7.8% and 16.7% across a threefold change. The two architectures even disagree on the sign of the slope, which is what a lever that is not acting on the relevant quantity looks like.

Turning it off changes nothing either. Qwen3-TTS-0.6B with the penalty disabled outright renders 10.0% of repeated items exactly against 100.0% of controls — a 90-point gap with no repetition penalty anywhere in the decoder. This is the cleanest single arm in the argument: a model that normally ships a penalty, run without one, still shows the effect at full size.

And three checkpoints never had one. Llasa passes no repetition penalty at all, so its decoding rule cannot be suppressing anything. Across the three: 14.0% of repeated items exact against 92.9%, a gap of 78.9 points.

The asymmetry that makes the sweep readable. Disabling XTTS-v2's penalty collapses its *control* to 30.9% — without it the model cannot render thirty-two distinct words either. That arm is therefore uninformative about repetition rather than evidence about it, and slopes are fitted only over arms whose control survives. Qwen3-TTS is the useful contrast precisely because its control is unharmed at zero penalty, so the zero-penalty comparison can be made at all.

16 Chronology: what was decided when

Reviewers observed, correctly, that this study contains many analysis decisions taken after seeing data — an extension of the k ladder, a second control regeneration, a template exclusion, a change of judge, a retracted probe claim. Individually each is disclosed where it appears. Collectively they are a garden of forking paths, and a reader is entitled to ask which choices preceded the data and which followed it.

The honest answer is that only the original ladder and the matched-control design were fixed in advance. Everything below followed from something the data showed, and the order is recoverable from the repository's commit history rather than from our recollection — each row corresponds to a commit, timestamped, in a public log. We report it as a chronology rather than claim a pre-registration we do not have.

when	status	decision
before data	*pre-specified*	k ladder to 32; matched length controls; CTC-vs-AR judge comparison planned
17:09 d1	pre-specified	theorem and Lean development frozen
18:49 d1	*forced*	control scoring rewritten (the first version scored every control as a loop)
20:44 d1	*forced*	judge changed from Whisper to CTC: the AR judge undercounts exactly the material under study
20:49 d1	*forced*	metric changed from exact match to relative count error, which the new judge made meaningful
22:23 d1	post hoc	repetition-penalty sweep on XTTS-v2
08:41 d2	*forced*	horizon shape tested after a reviewer noted the reported deficit was proportional, not saturating
08:53 d2	post hoc	k ladder extended to 128 *because* $k \leq 32$ showed no saturation
09:21 d2	post hoc	checkpoint-level inference replaces vote counts
09:27 d2	post hoc	dilution dose-response tested; came out sign-reversed
10:08 d2	post hoc	non-autoregressive baseline added
11:00 d2	post hoc	controls re-generated aperiodic at $k = 12 \ldots 32$
11:07 d2	*retraction*	probe no longer cited as mechanistic support
12:03 d2	post hoc	penalty sweep extended to a second architecture
13:29 d2	post hoc	probe control arm over the same k range: rules out the narrow-range explanation
13:51 d2	*failure*	second probe checkpoint run; it did not replicate, and the result went in as 1 of 2
14:16 d2	post hoc	horizon ratio refit with two families we did not choose; the ratio became a range
14:20 d2	post hoc	exclusion sensitivity: gap recomputed with every rule

Three observations a reader should weigh.

The forced changes went against us. Replacing the judge cost the paper its original metric and every number computed before it. The horizon test was run because a reviewer said our headline did not match our theorem, and it confirmed the mismatch. The probe retraction removed the only evidence we had offered for the causal claim. A garden of forking paths that systematically favours the author does not look like this.

The post-hoc additions are adversarial, not confirmatory. The non-AR baseline, the aperiodic controls, the second penalty architecture and the noise floor were each run to give a rival explanation its best chance. Three of the four could have ended the paper and did not; the fourth (the dilution dose-response) did damage and is reported.

Which checkpoints got the probe was not a free choice. A reviewer asked whether the two checkpoints carrying the post-horizon test were selected after seeing which way they came out. They were not, and the log shows the shape of it: the second was run because one is not a result, and it contradicted the first; the third was launched with the 1-of-2 split already written into the paper, so its result could only confirm or further weaken a claim already reported at its weakest. It weakened it, to 1 of 3. At the time, XTTS-v2 could not be run at all (S12) and the Qwen checkpoints had not been reached; both were instrumented later in the same spirit as the third checkpoint — launched with the 1-of-3 split already written down, not after seeing which way they would land — and the split weakened again, to 1 of 5 (S12). XTTS-v2 still cannot be run.

The out-of-sample prediction, with its timestamps. The main text reports a predicted gap for an architecture outside the panel and then an observed gap for CosyVoice 2, and a reader is entitled to check that the prediction really did precede the observation rather than being fitted to it. The filesystem and the commit log agree on the order:

Time (UTC)	Event	Value
17:15	`hierarchical.json` written	$+65.3$ $[5.9, 93.3]$
17:28	committed as `6f34945`	—
17:50	*first* CosyVoice 2 audio file exists	—
18:47	CosyVoice 2 generation completes	—
18:58	CosyVoice 2 scored	$+64.4$

All on 2026-08-12. The predictive interval was written thirty-five minutes and committed twenty-two minutes before the first CosyVoice 2 waveform existed, so no CosyVoice 2 measurement could have informed it. Generation was already *queued* when the fit was committed, which is why this is recorded as a held-out prediction and not as a pre-registration: the architecture had been chosen, and only its result was unknown. The prediction is also a point estimate carrying an interval $[5.9, 93.3]$ nearly ninety points wide, and under the widest prior on the group-level scales that interval includes zero (S11). Landing within 0.9 points of the posterior mean is therefore a good deal more precise than the model was entitled to promise, and should be read as one architecture falling comfortably inside a wide predictive interval rather than as a sharp quantitative hit.

The extension ladder is the exception, and we flag it as such. It was run because the pre-specified range failed to show the theorem's signature, and it is the one result whose framing benefited from being sought. The main text labels it post hoc at the point of introduction, reports the checkpoint that contradicts it, and does not claim the fitted $\hat{K}$ is the theorem's N^*.

17 Is the horizon ratio a property of the data or of the curve?

The extension ladder and its saturating fit were both chosen after seeing that $k \leq 32$ was proportional. S16 dates that decision against the commit log, but a reviewer is right that dating *when* we chose the exponential form does not show the form was not chosen because it fit. Cells past $k = 48$ hold 21–26 generations. At that size the curve can move the number a long way.

A repair landed between drafts, and every number below moved. `src/models/qwen_gen.py` computed its decode-step count one token short (`n_steps = len(hidden_states) - 1`), so `hit_cap` read False on every Qwen row ever generated — including twenty-one that had saturated the token budget outright (8191 tokens against a cap of 8192; 2047 against 2048). Population rule 4 was therefore never able to exclude a Qwen generation, and budget-truncated items — whose counts are censored *downward* by the cutoff, not by the model giving up — sat inside the extension-ladder horizon fits below. The ledgers were repaired from `n_speech_tokens`, which was always recorded correctly, and every horizon analysis in this section was rerun against the repaired population. The headline soft-horizon ratio moves 3.5 → 4.2 and the three-family range moves 2.7–4.3 → 3.1–5.4: *in the direction that flatters us.* The truncated repeated-arm items were inflating the fitted saturation scale on the repeated side, so the uncorrected number had been *understating* our own effect, not overstating it. We say that in these words rather than let a correction that helps the paper pass quietly — a favourable correction is the one that most needs to be visible, not less. The tables below are the refit, post-repair; only the two Qwen checkpoints move, since the bug is specific to `qwen_gen.py` and never touched the Llasa generations.

So we refit with families we did not choose. All three are one-parameter, all three have unit slope as $k \to 0$ — so none of them fights the proportional regime the main ladder shows — and all three asymptote to $\hat{K}$. They differ only in how they bend between those two ends.

family	$\hat{c}(k)$	$\hat{K}_{\text{rep}}$	$\hat{K}_{\text{ctl}}$	ratio
soft horizon	$\hat{K}(1 - e^{-k/\hat{K}})$	24.8	104.8	4.22
hyperbolic	$\hat{K}k/(\hat{K} + k)$	33.6	181.7	5.40
tanh	$\hat{K}\tanh(k/\hat{K})$	23.7	74.6	3.14

The soft horizon is the fit reported in the paper; hyperbolic bends earliest and has the heaviest tail, tanh bends latest and has the sharpest knee, so the two bracket the exponential from either side. The control-side $\hat{K}_{\text{ctl}}$ is unchanged to the digit shown, which is consistent with the bug: it is the repeated arm's budget-truncated items, not the control's, that were biasing the fit. Per checkpoint:

checkpoint	soft horizon	hyperbolic	tanh
Llasa-1B	5.97	8.46	4.50
Llasa-8B	4.12	5.23	3.06
Qwen3-TTS-0.6B	13.13	16.33	4.90
Qwen3-TTS-1.7B	0.29	0.57	0.07

Direction is form-independent; magnitude is not. No family moves any checkpoint across 1, before or after the repair. Which checkpoints show a lowered horizon, and which one does not, is the same under all three, and the one below 1 is the exception S4.2 already reports. What the family does change is how far: the pooled ratio runs 3.1–5.4, up from the pre-repair 2.7–4.3. The

paper's headline, now 4.2 where it was 3.5, should be read as a point inside that range and not as a measured constant, which is why S4.2 quotes the range next to it.

What this does not settle. It tests the choice of saturating curve, not the choice to fit a saturating curve at all. Every family here saturates by construction, so three of them agreeing says nothing to a reader who doubts saturation itself — and the declining repeated counts past the plateau (`horizon_ext.py --report-decline`) still fit none of the three. Three agreeing curves are easy to mistake for more agreement than they carry, so we say plainly what was and was not varied.

Reproduce. `python analysis/horizon_forms.py`. The fit grid, exclusion rules and bootstrap are shared with `horizon_ext.py`; the soft-horizon row reproduces the paper's numbers exactly, which is the check that the two scripts see the same population.

18 Do our own exclusions make the gap?

About a quarter of panel generations are removed before the headline statistics are computed: a judge-unmeasurable template, generations cut off by our own token budget, and degenerate audio. Each rule was adopted for a stated reason and each is defensible alone. That is precisely the situation in which a reader should want the numbers without them, because three defensible rules can still add up to a selected sample.

population	n	exact rep	exact ctl	gap
panel (as reported)	1234	18.2%	94.3%	76.1
+ judge-unmeasurable template	1424	17.2%	79.8%	62.6
+ budget-truncated items	1362	16.3%	93.1%	76.7
+ degenerate audio (scored 0)	1242	17.9%	93.9%	76.0
nothing excluded at all	1620	14.8%	76.5%	61.7

With every rule switched off the gap is 61.7 points against the 76.1 we report, on 1620 generations rather than 1234. The effect is not manufactured by the exclusions; it is somewhat flattered by them, by about fourteen points, and the main text now gives both numbers.

Which rule does the work, and why. Only the template rule moves anything, and it moves the *control* side: restoring t2 drops control accuracy from 94.3% to 79.8% while the repeated side barely changes. That is the expected direction and the reason the rule exists. The CTC judge emits `okay` in 0 of 106 items and `hmm` in 0 of 89, so t2 rows score our recogniser's vocabulary rather than the decoder — and a control sentence made of many distinct words has more to lose from an unmeasurable word than a repeated one does. Restoring those rows therefore adds judge noise to the control arm specifically, which shrinks the gap without telling us anything about counting.

The budget-truncation rule is the one that could have flattered us, since truncated generations are censored downward and dropping them removes bad-looking repeated items. It does not: restoring them moves the gap from 76.1 to 76.7, because truncation hits both arms. We report that arm separately for exactly this reason.

Reproduce. `python analysis/exclusion_sensitivity.py`. Degenerate audio has no transcript to count, so scoring it means choosing a number; zero is the harshest reading available and the only one that cannot be accused of flattering the repeated side.

19 Exact model revisions

Checkpoint names alone do not pin a model: repositories are updated in place, and a reader who downloads `coqui/XTTS-v2` next year may not get the weights these numbers came from. These are the commit hashes actually resolved by the runs, read back from the download cache rather than typed, and they cover the judge and the vocoder as well as the synthesisers — the judge decides every count in this paper, so a change to it changes every number.

repository	role	revision
`HKUSTAudio/Llasa-1B`	panel	`cef8ce6a6eea`
`HKUSTAudio/Llasa-3B`	panel	`740b08c956fd`
`HKUSTAudio/Llasa-8B`	panel	`2a8339e38734`
`Qwen/Qwen3-TTS-12Hz-0.6B-Base`	panel	`5d83992436ea`
`Qwen/Qwen3-TTS-12Hz-1.7B-Base`	panel	`fd4b25438912`
`coqui/XTTS-v2`	panel	`6c2b0d75eae4`
`HKUSTAudio/xcodec2`	Llasa vocoder	`e412427ed30f`
`facebook/mms-tts-eng`	VITS baseline	`c71de0fe7204`
`SWivid/F5-TTS`	F5-TTS baseline	`84e5a410d9ce`
`facebook/wav2vec2-large-960h-lv60-self`	CTC judge	`bdb144224e9b`

Hashes are truncated to twelve characters here; the full values are in `data/results/model_revisions.json`, regenerated from the cache rather than maintained by hand so that they cannot drift from what was run.

20 Rivals we cannot exclude

The main text names the rivals we tested: length, aperiodicity, the decoding rule's repetition penalty, and architecture. Three more we did not, and a reader weighing the contraction account should weigh these against it.

A learned dispreference for verbatim repetition. If a model's training data rarely contains a phrase repeated six or thirty times verbatim — or if its objective actively discourages producing one, which is the degeneration literature's account of the text case — then it would undercount repetitions for reasons having nothing to do with contracting state. This predicts our central dissociation exactly: repeated text is the out-of- distribution case, its length-matched control is not. Nothing in this paper separates it from the dynamical story, and the introduction's analogy to neural text degeneration gestures at it without ever operationalising it as a rival to be ruled out.

What would separate them is a checkpoint-level correlate — the frequency of $k \geq 6$ verbatim repeats in each model's training data, if obtainable, against its measured deficit — or a controlled fine-tune on repetition-rich data. The first needs corpus access we do not have for any of the six checkpoints; the second is a study, not an ablation.

A generation-length prior. All six checkpoints saturate on *both* arms, which is why we report a ratio rather than the repeated-side scale. A learned prior over utterance length would produce that saturation, and would lower the repeated-side scale further if repeated text is also rendered faster or with less silence. The ratio divides out a shared ceiling; it does not divide out a ceiling that differs by arm.

Acoustic-token periodicity. Every panel checkpoint decodes into a discrete acoustic codebook, and the attractor, if there is one, could live in the codec's token dynamics rather than the language

model's hidden state. Our probes read decoder hidden states, so they cannot tell those apart. The non-autoregressive baselines bear on this only indirectly: VITS has no acoustic token sequence at all, and F5-TTS's is not autoregressive.

None of these is refuted here, and the paper's framing is a conditional for that reason: what is proved is an implication, what is measured is a dissociation, and the link between them is the part still missing.

21 What the brackets mean

Every bracketed pair in the paper is a 95% interval. Which kind depends on what is being estimated, and the choice is not cosmetic:

- **Rates** (exactly-correct percentages, outcome rates) use a Wilson score interval. Several cells sit at or near 0% and 100%, where the normal approximation is degenerate or runs outside $[0, 1]$.
- **Fitted quantities** (capacity gain, saturation scale $\hat{K}$, the horizon ratio) use a nonparametric bootstrap, resampling the unit named in the relevant section — stimulus templates for capacity, items for $\hat{K}$.
- **Checkpoint-level means** (the $n = 6$ panel statistics) use the range across checkpoints, not a distributional interval. With six points a percentile interval would imply more than six points can support.

Item-pooled intervals treat items as exchangeable and therefore ignore clustering within checkpoints; they are narrower than they should be, which is why the main text quotes the checkpoint-level number beside every pooled one and leans on the former.

22 Does the deficit survive greedy decoding?

Theorem 1 bounds a readout, so no property of the sampling rule enters the proof and the bound is stated as covering greedy, sampled and beam decoding alike. That claim sat next to a panel in which every generation was sampled. It is the cheapest ablation the paper could have run and, as a reviewer observed, the one most directly relevant to its own premise.

The premise is what makes it interesting rather than merely tidy. Assumption 1 posits a single time-invariant map F between repetition boundaries. Stochastic token choice is exactly the per-step perturbation that would break that autonomy: what gets sampled at repetition m changes what conditions repetition $m+1$. Greedy decoding removes the perturbation, so it is the setting in which Assumption 1 is *most* defensible — and therefore the one in which the deficit has the fewest excuses.

arm	n	exact rep	exact ctl	gap
sampled, three seeds	216	8.3%	83.3%	75.0
sampled, seed 0	72	5.6%	83.3%	77.8
greedy	72	11.1%	83.3%	72.2

Qwen3-TTS-0.6B, $k \geq 6$, `do_sample=False` with temperature and top-p omitted rather than passed and ignored. Greedy is deterministic, so one seed is the entire experiment on that arm; the sampled

arm is given both at seed 0 and pooled over its three seeds, because quoting whichever is more favourable would be the obvious way to cheat here.

The deficit survives. Greedy renders 11.1% of repeated items exactly against 83.3% of controls — the control rate is identical across all three arms — for a gap of 72.2 points against 75.0 sampled. Removing sampling noise moves the repeated side by about five points and leaves the dissociation intact. Whatever produces it is in the conditioning, not in the draw.

The truncation confound, checked rather than assumed. Greedy decoding on repetitive text runs into a generation budget more readily than sampling does, and budget-truncated counts are censored downward — precisely the direction that would manufacture a greedy deficit. The cap-hit rate is 0.0% on both Qwen arms, so nothing was excluded on either side and the comparison is not doing hidden work. (It is not zero everywhere: the Llasa checkpoints run 9–18%, which is why the rule exists.) The first launch of this run, over the full stimulus file, was abandoned for the related reason that greedy decoding hit the cap far more often on families the comparison does not use.

Limits. One checkpoint, one alternative decoding mode. It says that Qwen3-TTS-0.6B's deficit is not a sampling artifact; it does not say so for the panel, and beam search remains untested. `python analysis/greedy_decoding.py`.

23 The horizon exception is the weakest checkpoint

Qwen3-TTS-1.7B shows no repeated-side saturation within $k \leq 128$ and inverts the horizon ratio. Earlier drafts called it an unexplained exception and left it there, which a reviewer rightly read as setting an inconvenient checkpoint aside rather than accounting for it. The panel numbers do account for it.

checkpoint	exact-rate gap	median-error gap	capacity gain gap
Llasa-1B	78.1	12.5	38.5
Llasa-3B	77.2	8.3	46.5
Llasa-8B	81.4	16.7	30.5
Qwen3-TTS-0.6B	92.2	12.5	14.0
XTTS-v2	71.1	12.5	10.7
Qwen3-TTS-1.7B	**60.0**	**0.0**	**8.1**

It ranks last on all three, and it is the only checkpoint that ranks last on any of them. The exception is therefore not arbitrary: it is where the effect is smallest on every measure we have, and the horizon ratio is the measure on which smallest tips over into absent.

This is an account, not an explanation. It says the exception is consistent with the rest of the panel rather than in tension with it; it does not say *why* this checkpoint has the weakest effect. Two candidates we cannot separate: its per-repetition map may not contract, or its horizon may simply lie past $k = 128$, which for the checkpoint with the smallest deficit is exactly what one would expect. Distinguishing them needs a ladder longer than its generation budget allows.

One thing this does *not* license: reading the ranking as evidence for the theorem. A weakest checkpoint exists in any panel, and it will be the one nearest any threshold by construction. What the ranking rules out is the specific worry that the horizon result rests on discarding a checkpoint that contradicts it — the checkpoint that fails to saturate is the one with almost nothing to saturate.

24 Where every sample size comes from

A reviewer counted the n's across Sections 3–5 — 2052, 1559, 457, 525, 108, 36 — and noted that the paper never reconciles them against each other. That is a fair complaint about a paper whose argument is "trust the numbers because you can regenerate them": a reader who cannot see how 2052 becomes 457 cannot tell a defensible restriction from a convenient one.

population	n	what leaves, and why
all generations scored	5400	every model, every family, every seed
repeated and control families	3420	drops numbers, twisters, `sentence_rep` — other questions
panel checkpoints only	2052	drops 1368 ablation and baseline rows
judge-measurable templates	1710	drops 342 rows in t2, whose fillers the CTC judge never emits
stopped by the model, not our budget	1578	drops 132 budget-truncated rows, whose counts are censored downward
audible speech	1559	drops 19 empty or degenerate rows, which have no count at all
$k \geq 6$, where the deficit lives	982	below k=6 every checkpoint is essentially exact

The leaves of that cascade are the numbers the analyses quote:

analysis population	n
word_rep at $k \geq 6$	457
control_word at $k \geq 6$	525
VITS per family (word_rep)	108
VITS per family (control_word)	108
greedy per arm (word_rep)	36
greedy per arm (control_word)	36

$457 + 525 = 982$, which is the $k \geq 6$ row: the two arms exhaust it, with no third category quietly absorbing rows. `analysis/sample_sizes.py` asserts that identity rather than leaving it to the reader, so a future change to the exclusion rules that broke the reconciliation would fail rather than produce a table that no longer adds up.

Two features of the cascade are worth naming because they look like padding and are not. The drop from 3420 to 2052 removes ablation arms and non-autoregressive baselines, which are re-runs of panel members under altered settings and contrasts respectively — pooling them would count XTTS-v2 five times. The drop from 1559 to 982 is the $k \geq 6$ restriction, and it is the one place a reader should be suspicious, since k is the independent variable. Figure 1(a) shows the full ladder including $k < 6$; the restriction exists because every checkpoint is essentially exact below it, so those rows contribute no variance to a median and would dilute the effect size without changing its direction.

The generation counts differ from these because a generation that produced no audible speech was still generated. Section 3's total is generations attempted; every n here is rows that survived to be scored.

25 What we did not listen to

The released sample is 165 clips, stratified over every checkpoint, k band and outcome label, each paired with its transcript and the scoring decision that transcript produced. A reviewer asked the obvious next question: what fraction did *we* listen to, and how often did we agree with the judge?

The honest answer is that we did not run a systematic listening study. Clips were audited by ear during development — when the control-scoring bug was found, when the template exclusion was decided, when the first duration-intervention run came back silent — and each of those audits changed something and is recorded where it applies. None of them was a blind, pre-specified agreement measurement, and reporting a percentage from them after the fact would dress up debugging as validation.

So the sample is released for a reader to check us, not as evidence that we checked ourselves. What stands in for a listening study is S9's synthetic ground truth, where the true count is exact by construction, and S14's noise floor, where two independent recognisers disagree on real generated audio without needing any ground truth at all. Neither is a human ear. A blind listening study on the released sample is the obvious thing a follow-up should do, and the sample exists so that it can be done without regenerating anything.

25.1 The pipeline at work, on rows nobody chose

Three review rounds have made a sharper version of the same complaint: the phenomenon is audible, the evidence is a number an automated pipeline produced, and the paper never once shows that pipeline working on a concrete generation. That is true and it is cheap to fix. What makes a worked example worth anything is that we did not pick it, so the selection rule is fixed in code (`analysis/worked_examples.py`) and takes whatever it lands on: restrict to the reportable panel, sort by (model, item, seed), walk the outcome labels in a fixed order and the checkpoints in sorted order together so outcome i is drawn from checkpoint i mod N, and take the first row of that pair. The checkpoint cycle exists because the simpler rule — first row per outcome — puts all six examples on Llasa-1B, which sorts first; that is a property of the alphabet, not of the pipeline.

outcome	checkpoint	k	c_A	c_B	dur. ratio	RMS	transcript
correct	Llasa-1B	3	3	3	1.00	0.0028	`THE LIGHT WAS GREEN BRIGHT PALE…`
undercount	Llasa-3B	2	1	3	1.40	0.0088	`THE LIGT WAS GREEN RED AT A SPAN`
overcount	Llasa-8B	8	9	12	1.50	0.0585	`THE BELL RANG TWICE…`×9
loop	Qwen-0.6B	6	6	10	1.73	0.0635	`HE COUNTED THE STONES…`
degenerate	Llasa-1B	2	0	1	0.63	0.0015	(empty)
empty	Llasa-1B	8	0	29	3.66	0.0003	(empty)

c_A is the CTC transcript count, c_B the duration-based estimate; `classify` reads c_A and the duration ratio, and the two estimators are never merged into a count. Four of these rows are worth reading closely.

The **loop** row is the case the conjunctive rule was built for. Its transcript count is 6 against a requested 6 — a transcript-only judge would score it *correct* — but the audio ran 1.73× the length six renditions take, so the rule refuses the label. This is the failure direction S9 documents at the population level, here in a single row.

The **empty** row is the mirror image. Thirty-seven seconds of audio at RMS 0.0003: silence. A duration-only estimator reads c_B=29 off it, which would have entered the table as an enormous overcount rather than as a generation that produced no speech. Degeneracy is therefore checked *before* either count, and this row is why.

The **undercount** row is an ASR error, not a model error, and it is on a *control* item: "LIGT" for *light*, one filler word lost. It is counted against the control arm exactly as it stands. That direction matters — judge noise on controls lowers the control's exact rate and therefore *shrinks* the reported gap, so this class of error is conservative for the paper's claim rather than favourable to it.

The **correct** row is a control at k=3 scoring 3/3 with a duration ratio of exactly 1.00, which is what a pass looks like and is shown first for readers who suspect the pipeline of manufacturing failures.

Two cells fell back. The cycle assigns `degenerate` to Qwen3-TTS-1.7B and `empty` to XTTS-v2, and neither checkpoint produced a single row with that label anywhere in the panel, so both fall back to the first panel row and the artifact records that they did. That Qwen3-TTS-1.7B never degenerates is consistent with its being the one checkpoint that counts correctly at $k \leq 32$; that it still shows the full period ladder (S28) is the point.

26 A causal intervention on the count, reported uninterpretable

Every mechanistic claim this project makes about the count is correlational: a ridge probe reads $\log_2 k$ off Llasa-1B's residual stream (S12), and S24 showed that readability degrades with k. Neither result distinguishes the account this paper argues for – the count is erased from the state – from the rival it has never been able to exclude: the count survives in the representation and only the output policy that reads it fails. Separating them needs a manipulation of the readout at fixed state, not another probe. We ran one. It did not come back a hit for either account, and it did not come back a clean null either. It came back uninterpretable, and this section documents why, in the same detail S5 gives the estimator that failed on Theorem A.

The code is `analysis/causal_count.py` (generation) and `analysis/causal_count_score.py` (the pre-committed scoring rule, written into that file's docstring before any audio existed). The artifacts:

file	contents
`data/results/causal_count.json`	the scored verdict, both arms
`causal_patch1b_manifest.jsonl`	Arm A, per-generation records
`causal_steer1b_manifest.jsonl`	Arm B, per-generation records
`causal_patch1b_flags.json`	Arm A audio-degeneracy flags
`causal_steer1b_flags.json`	Arm B audio-degeneracy flags
`causal_steer_directions.json`	the two steering directions

(the last five under `data/results/`).

26.1 The design

Arm A – activation patching. A repeated `word_rep` item at receiver $k \in \{16, 24, 32\}$ (three carrier templates, two seeds) is teacher-forced through its own first P=128 generated positions, then at one layer $L \in \{3, 7, 11\}$ the residual stream at exactly those positions is overwritten with the states a *donor* item had at the same generated positions, and generation resumes with the hook removed. Because the key/value cache above L is rebuilt from the patched states, everything the model attends to from that window onward is donor-derived. Donors, per receiver:

- **cross-k** – the same carrier asking for a different k on the ladder ($\{8, 32\}$ or $\{8, 16\}$ depending on the receiver). The informative donor: if the count is causal, a higher-k donor should push the rendered count up and a lower-k donor down.
- **control** – the length-matched distinct-word control at the receiver's own k.
- **same-k, different seed** ("diffseed") – the identical item, identical k, a different sampling seed. No positional offset, no count difference: it isolates "the state moved" from "the count moved".
- **unrelated** – a repeated-*sentence* item from a different family, a different carrier, no matching adverb, and no information about the receiver's k. The disruption control: if states that cannot possibly encode the receiver's count move the rendered count as much as an informative donor does, the protocol is perturbing generation rather than transplanting a count.
- **self** – the receiver's own captured states. The no-op gate.

Every patched run is scored against a *resume reference*: the identical P-token prefix, continued with no hook, same seed – not against the receiver's own free generation, which draws from the sampler's random stream at a different offset. The readout is the CTC judge's rendered count on $\log(1 + \text{count})$, paired within receiver run against its own reference, because Llasa-1B's rendered count on repeated text is heavy-tailed enough that single-seed values of 3 and 476 both occur at k=24.

Arm B – steering. A count direction $\hat{v}$ is built at layer 13 (and, as a check, layer 7) from activations already on disk (54 `word_rep` items), two ways: the difference-in-means of terminal states between low $k \in \{2, 3, 4\}$ and high $k \in \{16, 24, 32\}$ items ("dim"), and the ridge probe's weight vector ("ridge"). $\alpha\Delta\hat{v}$ is added to the residual stream at every generated position, $\alpha \in \{-2, -1, -0.5, 0, 0.5, 1, 2\}$ for the primary (dim, layer 13) direction, $\{-1, 1\}$ for the others, on eight `word_rep` items and eight length-matched `control_word` items ($k \in \{16, 32\}$, four carrier templates). α=1 means "move the state by one full low-k-to-high-k step"; at layer 13 that step has norm 17.0 against a mean per-position state norm of 37.8, so α=1 already adds a vector that is 45% of a position's own norm, α=2 is 90% (`causal_steer_directions.json`). A direction that is about the count should move the repeated arm more than the length-matched control; one that is a generic "say more speech" knob should move both.

26.2 The sanity gate, which passed

Both arms have a bitwise gate, run before either is interpreted. The no-op self-patch must reproduce its resume reference bitwise, and α=0 steering must reproduce the free baseline bitwise. Both passed at 100%:

gate	n	result
Arm A: self-patch = resume reference	18	18/18 bitwise identical
Arm B: α=0 = free baseline	18	18/18 bitwise identical

`sanity_gate_pass` is `true` in the JSON. The intervention machinery – the hook, the position bookkeeping, the resume protocol – works exactly as designed. Everything that follows is a property of the result, not of the instrument.

26.3 The verdict: too disruptive to interpret

`causal_count.json`'s `verdict` field reads: *"intervention too disruptive to interpret: states from an unrelated item move the rendered count 134% as much as a donor that actually differs in k … neither a causal hit nor a null can be read off it."* This is a third, pre-committed outcome (docstring of `causal_count.py`), checked *before* either a hit or a null is considered, because a broken instrument and a true null look identical in a table of medians. It is triggered by `D1`, the unrelated-donor check: at the one cell where every donor kind was run (layer 7, P=128), the median absolute paired shift from an unrelated donor is 1.037 in log(1+count) against 0.771 for the informative cross-k donor at the same cell – a ratio of 1.34× (`disruption_index`: `"L7|P128": 1.34`). States that carry no information whatsoever about the receiver's k move the rendered count *more* than states that do. `D2` (degeneracy inflation) did not fire – degenerate-audio rate is 0.0% in both the reference and patched arms – so the disruption is not the patch turning speech to noise; it shows up in the count and the stop decision while the audio itself stays intelligible.

Two further numbers corroborate the same reading without being part of the pre-committed test. Pooled over every non-self patched condition against every resume reference:

arm	n	cap-hit %	stop %	degenerate %
reference (resume, no hook)	19	15.8	84.2	0.0
patched (any non-self donor)	163	37.4	62.6	0.0

Patching more than doubles the cap-hit rate (a run that never emits its own end-of-speech token and is truncated by the token budget) and drops the fraction that stop on their own from 84.2% to 62.6% – and this is not unique to the unrelated donor: even the length-matched *control* donor, which shares the receiver's k and carries no count prediction at all, hits the cap on 44.4% of its 18 runs. Splicing donor states into this decoder disrupts whatever process decides to stop, essentially regardless of what the donor's states actually encode.

Against this background, no cell clears the pre-committed hit bar. At the central cell (layer 7, P=128, n=36 paired cross-k patches), the higher-k donor does move the count up (median +0.118, raw sign-test p=0.039) and the lower-k donor down (median −0.717, p=0.541) – the opposite-directions requirement (`A1_opposite_directions`) is satisfied – but the cell's own Holm-corrected p is 0.261, so `A1_signed_shift` is `false`. The same-k-different-seed control moves the count by −0.077, which is *not* comfortably smaller than half the cross-k cell's own median (0.070), so `A2_not_generic_state` is `false` too: a donor that agrees with the receiver about the count moves the rendered count almost as much as a donor that disagrees. Restricted to the clean subset (no cap-hit, no degenerate run on either side of the pair, n=19), the median shift flips sign to −0.123, failing `A3_survives_cleaning`. The two other depths tested fare no better – layer 3 (n=36, median +0.197, p_{holm}=0.449, opposite-directions `false`) and layer 11 (n=36, median +0.080, p_{holm}=1.0). `hits` is the empty list at every layer, and `causal_hit` for Arm A is `false` – but given the disruption index above, that `false` is not evidence of a null: the protocol that produced it cannot be trusted to distinguish a null from a broken splice.

26.4 Arm B's outcome

The primary direction (dim, layer 13) clears none of its three pre-committed conditions on the repeated arm:

α	median count	degenerate %	cap-hit %	stop %
-2.0	0	100.0	100.0	0.0
-1.0	0	100.0	100.0	0.0
-0.5	0	55.6	100.0	0.0
$+0.0$	13	0.0	22.2	77.8
$+0.5$	13	0.0	0.0	100.0
$+1.0$	0	22.2	0.0	100.0
$+2.0$	0	66.7	0.0	100.0

B1 (dose-response) fails. The rendered count collapses to 0 at *both* extremes rather than rising monotonically toward the high-k end, and the pooled Spearman correlation over all (item, α) pairs is ρ=0.187 (p=0.142), short of the pre-committed $|\rho| > 0.5$, $p < 0.05$ bar, despite 87.5% of items agreeing in sign (`B1_dose_response`: `false`). **B2 (not degenerate) fails.** Degenerate-plus-empty rate at $|\alpha|$=2 reaches 100.0% (-2.0) and 66.7% ($+2.0$) against 0.0% at baseline, both far past the 15-point pre-committed tolerance (`B2_not_degenerate`: `false`). **B3 (specificity) fails,** and in the wrong direction. The one contrast that survives the extremes' degeneracy is the paired α=+1 vs. α=-1 shift within item: on the `control_word` arm it is a median +1.10 in log(1+count), 6 of 6 non-tied items positive, sign-test p=0.031; on the `word_rep` arm it is a median 0.0, 3 of 3 non-tied items positive (five items tied at zero on both sides), p=0.25. The control that carries no count prediction moves *more*, and significantly so, than the repeated arm the direction is supposed to be specific to – backwards from what a count direction predicts (`B3_specific`: `false`). We flag one artifact in the JSON rather than let it stand uncommented: `effect_ratio_rep_over_ctl` reads `Infinity`, but this is a division-by-zero in the summary statistic (both arms' effect medians, taken between the α=$\pm$2 extremes, are exactly 0.0 because generation has already collapsed to empty output at both extremes in both arms), not evidence of an infinitely count-specific direction; the paired $\pm$1 contrast above is the number that is actually informative, and it argues against specificity.

26.5 Why this matters methodologically: the control that was missing

The repository's history preserves an earlier pass through this exact protocol, scored before the grid was filled out: 6 resume references, 13 patched runs, cap-hit 50.0% (reference) and 46.2% (patched), spread thin over three window lengths ($P \in \{32, 128, 384\}$) that the script's own comments call "the pilot's first pass" – about two pairs per cell, "which cannot decide anything." Critically, that pass had not yet reached the unrelated-donor condition: its cell table has no `unrelated` entry at all. Scored as it stood, the verdict was *"count decodable but not causally used: the sanity controls pass, so the intervention machinery works, and neither patching donor states nor steering along the count direction moves the rendered count toward the donor's k at any locus tested"* – a clean, reportable null.

Continuing the same manifest to 163 patched runs and 19 references, concentrating the whole layer sweep on P=128 instead of splitting it three ways, and – the change that mattered – adding the unrelated-donor condition, overturned that reading. It was not that the informative donors started moving the count; `hits` is still empty. It is that the disruption control revealed the null could not be trusted: an uninformative donor moves the count as much as an informative one, so "neither donor moved the count" is indistinguishable, in this protocol, from "the protocol cannot register a donor-carried count difference at all." Without the unrelated-donor arm, this experiment would have shipped exactly the null quoted above – confidently, and wrongly, since a null that

cannot rule out a broken instrument is not a null. We record this rather than only the final verdict because the lesson is about the design, not just this result: a causal null is only as good as its disruption control, and this one did not exist until the second pass. (We can confirm from `data/results/causal_count.json`'s git history that the donor pool, sample size, and cell coverage changed between the two passes; we cannot independently confirm from version control alone that the per-run token budget itself was raised between them, since the committed script already sets it to the value used throughout – the two passes differ in how much of the same grid had been filled in, not necessarily in the generation budget per run.)

26.6 What would make this testable

The common thread across both arms is that the intervention itself is heavy enough to dominate whatever it is trying to isolate: Arm A overwrites the full residual-stream vector at every one of 128 consecutive positions at one layer, and even the same-k control donor drives cap-hit to 44.4%; Arm B's α=1 step is already 45% of a position's own state norm. A less disruptive design is the obvious next attempt: steering at a fraction of the magnitude used here (the $\alpha \in [0, 0.5]$ region is the only one that leaves Arm B's repeated arm non-degenerate); patching a narrower window or fewer dimensions per position rather than the whole state; or intervening directly on the stop logit – the end-of-speech decision itself – rather than on the residual stream several layers upstream of it, since the stop decision, not the count, is what this paper's account is ultimately about.

26.7 The follow-up: a rank-1 patch, and the escape clause it rests on

The previous subsection's "patch a narrower set of dimensions" is what we did next, and it is the experiment the main text actually reports, so it is documented here rather than left implicit. Instead of overwriting the residual stream, the follow-up transfers *only* the probe's count coordinate: the receiver's state is projected off the ridge direction and the donor's component along that same direction is written in its place, at every position of the window, leaving all other directions untouched. Everything else — donor pool, cell structure, scoring rule, sanity controls — is inherited unchanged from Arm A, so the two are directly comparable. The artifact is `data/results/causal_count_followup.json`.

The damage does go away. Degeneracy is 0.0% against a reference of 0.0%, the self-patch no-op is bitwise identical in 18/18 runs, and the informative cross-k cell moves the count by a median of exactly 0.0 in log(1+count) (n=36, 95% CI $[-0.09, 0.09]$). Steering along the same direction shows no dose-response.

That is the reading the main text gives, and it rests on an escape clause we state plainly rather than bury. The pre-committed gate P1 asks that an *unrelated* donor move the count less than half as much as an informative one; this arm's ratio is 1.74, so as a ratio it *fails*. What rescues it is that both sides of the ratio are noise: the unrelated donor's median |shift| is 0.259 and the cross-k donor's 0.149, against a full donor-to-receiver transfer of 0.992 — so the ratio is noise over noise, and the honest description of the arm is not "the disruption control passed" but "every cell sits at the arm's own noise floor; nothing moves, informative or not." On the published checkpoint that reading was reached *after* seeing the numbers. We therefore did two things: we wrote the escape into the next experiment's pre-commitment as an explicit, falsifiable test — the largest median |shift| across all donor kinds must be below 25% of a full transfer — and we report below which checkpoints pass it and which do not.

26.8 Replication on a second family: four checkpoints, and one that returns "cannot tell"

A null on one checkpoint is as easily a property of that checkpoint's probe geometry as of decoders in general. The replication puts the identical patch on four checkpoints across two families, 2,178 scored runs in all (`analysis/causal_count_second.py`, whose docstring carries the pre-commitment quoted above and predates the audio; `data/results/causal_count_second_checkpoint.json`).

checkpoint	P1 ratio	worst cell	bound	verdict
Qwen3-TTS-0.6B	1.38	14%	9%	readable null
Qwen3-TTS-1.7B	1.12	9%	6%	readable null
Llasa-1B	0.57	25%	20%	readable null
Llasa-8B	0.90	42%	—	cannot tell

"Worst cell" is the largest median |shift| over all donor kinds as a fraction of a full transfer — the pre-declared escape test, which passes below 25%. "Bound" is the 90% TOST equivalence bound on the cross-k cell in the same units: the smallest effect the arm can exclude, which is the deliverable rather than an interval containing zero. Both Qwen bounds are *tighter* than the published Llasa-1B one, 9% and 6% of a full transfer, TOST $p < 10^{-4}$ under both a row bootstrap and one clustered by receiver item; the one-sided bound toward the donor is 0.000 on both, i.e. no movement toward the donor at all. In interpretable units, at k=24 the Qwen-1.7B bound excludes any shift larger than 1.1 repetitions. Every checkpoint's no-op self-patch is bitwise identical (18/18; 36/36 on Llasa-8B) with $\max|\delta|$ exactly 0.0, and every α=0 steering run is bitwise identical to its baseline. The steering sweeps are nulls throughout (Qwen-0.6B ρ=+0.034, p=0.73; Qwen-1.7B ρ=+0.078, p=0.42; Llasa-8B ρ=+0.047, p=0.63).

Llasa-8B fails, and the diagnostic is the decisive kind. Its worst cell is 42% of a full transfer, so it fails P1 as a ratio *and* fails the noise-floor escape — the case the pre-commitment says yields no count claim. The specific reading: a *same-k, different-seed* donor, which carries exactly the count the receiver already has, moves the rendered count as much as a cross-k donor does (0.246 vs. 0.217 median |shift|). An arm in which a donor with the right answer is as disruptive as one with the wrong answer is measuring *foreign state*, not *different count*, and cannot return a null about counting. Doubling n (72 pairs at the central cell, seeds 0–3) left this unchanged, so it is not a power problem. The verdict is *cannot tell*, which is not *no*, and the main text says so.

26.9 Two things the replication learned about its own protocol

The protocol is not interchangeable, and we can prove it. Llasa-1B was run under both its published resume protocol (patch a resumed prefix, pair against a resume reference) and the decode-time protocol used for Qwen (patch during free generation, pair against the free baseline). Under its own protocol it reproduces its published null: median 0.0, inside the published 90% interval, bound 0.134. Under the decode-time protocol the *same checkpoint, same patch, same donors* becomes uninterpretable — worst cell 0.891, bound 0.236 against a published 0.089. The Qwen numbers are therefore usable not because the decode-time protocol is safe in general, but because those checkpoints' own diagnostics say the arm is readable on them (worst cells 0.107 and 0.071 against full transfers of 0.740 and 0.750). That is measured per checkpoint, never assumed, and a reader who distrusts the decode-time protocol can discard both Qwen rows and still have the Llasa-1B resume replication.

The published manifest is no longer bitwise re-derivable. Re-running `analysis/causal_count.py`'s own generator today returns 873 tokens where the committed manifest records 1,528, on the installed stack (`transformers` 4.57.3, `torch` 2.11.0+cu130). The replication above is therefore *statistical*, not bitwise: it re-runs the experiment and compares distributions, and it is reported that way. Three further traps are recorded for anyone re-running this: `qwen_gen.eos_trim_length` indexes `hidden_states[j+1]` over the full range and runs off the end (rendered frames are len(`hidden_states`)−1, verified against the ledger and the waveform); `soundfile.write` stores float32 input as PCM_16, so a bitwise no-op gate must compare in memory and never through the file; and probe grids differ per checkpoint (`every3` vs. `every4`), so a patch layer must be looked up in each file's own `probe_layers` rather than used as a column index.

27 The Spanish arm: the judge replicates, the counting statistic does not

Three review rounds made the same objection: the headline framing – periodicity, not length – is stated without linguistic qualification while every measurement behind it is English. This section renders the same repetition ladder in Spanish, with the same XTTS-v2 weights, the same speaker reference and the same decoding configuration, and reports what that arm can and cannot support. Code: `analysis/crosslingual_es.py` (a pre-committed docstring, fixed before any Spanish transcript existed) and `data/stimuli/make_stimuli_es.py`. Data: `data/results/crosslingual_es.json`, `data/results/ctc_validation_es.json`, `data/results/judge_vocab_audit_es.json`, `data/results/behavioural_es.csv`. One line summarises it: **the judge audit replicates and is now in the paper; the counting statistic does not survive the judge that has to score it, and that failure is itself the finding.**

27.1 Language and judge

Spanish, not German, was chosen for the reason a re-implementer would want stated rather than assumed: on greedy, LM-free decoding through the same XLSR-53 fine-tuning recipe, the Spanish community checkpoint (`jonatasgrosman/wav2vec2-large-xlsr-53-spanish`) scores 8.82% WER against 12.07% for the equivalent German checkpoint. This comparison is recorded in the project history (commit `939a1c8`) rather than in a result JSON; no separate WER-audit artifact backs it, and it is reported here on that basis. The judge is pinned at revision `96d7e9b4e4a78af515a3c6d3cee7c0826045d276` (`data/results/model_revisions.json`) – not `main`, which carries no `safetensors` file; see the third trap below.

27.2 Gate 1: the judge audit – reportable

The paper's central methodological claim is that an autoregressive judge is biased against the phenomenon under study. Adopting a Spanish judge without re-running that same audit would be the exact error the paper criticises. `ctc_validation_es.json` builds the identical concatenative construction used for the English audit (S9) in Spanish: one verified k=1 rendering spliced N times, so the true count is exact by construction, scored by both the Spanish CTC judge and Whisper-es.

	periodic ground truth					
judge	k=2	k=4	k=8	k=16	k=32	$k\geq4$ pooled
CTC (es)	1.00	1.00	1.00	1.00	1.00	**1.00**
Whisper (es)	1.00	0.25	0.125	error	error	**0.19**

Whisper-es returns no transcript at all (a hard decoder error) on every $k \geq 16$ periodic trial – at the top of the ladder there is no measurement rather than a poor one, exactly as S9 found for English Whisper. The rows below, built from six carrier/unit pairs (*tan*, *claro*, *hondo*, *verde*, *firme*, *muy*), are the same repeated-sentence construction as S9's English arm, not a genuinely aperiodic control – each is a single verified control utterance concatenated N times – and are labelled accordingly:

	repeated-sentence ground truth					
judge	k=2	k=4	k=8	k=16	k=32	$k \geq 4$ pooled
CTC (es)	1.00	1.00	1.00	0.969	0.984	**1.00**
Whisper (es)	1.00	0.25	0.125	error	error	**0.25**

Pooled over $k \geq 4$: CTC 1.00 against Whisper 0.1875 on periodic material, 1.00 against 0.25 on repeated-sentence material – the same qualitative pattern as the English audit (S9), in a language with different orthography and a judge fine-tuned on an entirely different corpus (Common Voice, not LibriSpeech). As in S9, these pooled figures are medians and understate the spread in the underlying trials, more so here than in English: over the same 24 $k \geq 4$ periodic trials the CTC(es) mean ratio is 0.931 (14/24 exact), driven by one donor (*veloz*) falling to 0.50–0.56 at $k \geq 8$; over the 24 repeated-sentence trials the mean is 0.806 (13/24 exact), because one donor (*tan*) is never recovered at any k (ratio 0.0 throughout) while the other five are close to exact. Gate 1's pass/fail check compares the medians above, not these means, against the pre-committed band ($[0.90, 1.10]$), and the medians pass: the Spanish judge is not merely undercounting less than Whisper. "Unbiased" is a claim about that gate, though, not about every trial underneath it – *veloz* and *tan* are donor-level counterexamples the median does not show. This is the result now in the main paper as 1.00 against 0.19 (S9, S27).

Gate 2, the vocabulary audit, also passes cleanly: all six Spanish templates (`et1`–`et6`) clear the 0.05 delivery floor with zero exclusions (`judge_vocab_audit_es.json`: `below_floor: []`, `excluded_templates: []`). This is a strictly better outcome than English, which loses template `t2` (the judge never transcribes *okay* or *hmm*). Nothing about the Spanish vocabulary needed to be discarded to run the counting arm below.

27.3 Gate 3 and the verdict: the counting arm is not reportable

The measurement is exactly the English statistic: over $k \geq 6$, after `population.panel()`'s exclusions, the fraction of items whose counted occurrences equal k, for `word_rep` and its length-matched `control_word`, and the difference.

arm	rep. exact	ctl. exact	gap	n
Spanish (XTTS-v2, es judge)	11.1%	20.4%	+9.3	216
English XTTS-v2 alone	16.7%	87.8%	+71.1	234
English panel (all checkpoints, reference)	–	–	+76.1	–

The pre-committed gate that decides whether this gap means anything is Gate 3: control exact-rate $\geq 50\%$ over $k \geq 6$, on the reasoning that if Spanish rendering is simply hard for XTTS-v2 and *both* families collapse, a shrunken gap is not evidence about periodicity. Spanish control exact-rate is 20.4% (0.2037), well under the 50% bar. Gate 3 **fails**. `crosslingual_es.json`'s own verdict field: *"UNINFORMATIVE – Spanish rendering fails on the control family too, so the gap is small for a reason unrelated to periodicity; this is a finding about XTTS-v2's Spanish, not about the periodicity*

claim." The +9.3-point gap is therefore not reported as a weaker replication, and it is not reported as evidence that the English effect is English-specific either – Gate 3 exists precisely to rule both readings out when it fails. This is what the main paper's limits paragraph means by "a Spanish arm came out unreportable rather than replicated" (S27).

27.4 The diagnosis, which is the actual finding

A control item is exact only if all k distinct fillers match as exact tokens, so control exact-match behaves roughly like (per-occurrence hit rate)^k. The post-hoc diagnostic in `crosslingual_es.json` (computed after the verdict, and not part of the pre-committed gates) reports the average fraction of a control's expected words that the judge actually delivers:

arm	rep. counted/k	ctl. counted/k
Spanish (all templates)	0.674	0.722
English XTTS-v2	0.873	0.973

The English judge's control side survives close to intact through k=32; the Spanish one does not, for a reason that traces to the judge's own word error rate rather than to the model. Both `implementation-notes.md` and the analysis script's own narrative put this at roughly 0.97 per-occurrence delivery for the English judge against roughly 0.80 for the Spanish one – we flag that these two rounded figures are a narrative estimate recorded in project history and the script's print output, not a field stored in any result JSON; the closest literal JSON quantity is the `ctl_counted_over_k` row above (0.722 for Spanish, 0.973 for English), which is somewhat lower than 0.80 for Spanish and should be read as the authoritative number if the two disagree. Either way the mechanism is the same: a single inflectional slip – *tenue* transcribed *tenues* – voids an entire control item's credit, because scoring is conjunctive across all k occurrences. Duration ratio on the failing controls is close to 1.0 and the transcripts carry the right number of words; the model said the fillers, the judge spelled one of them differently.

The exact-rate statistic is not transportable to a lower-accuracy judge. The paper's headline 76.1-point gap depends on a $\sim$ 2%-WER judge holding the control side near 1.0 across the whole ladder; an 8.8%-WER judge cannot do that, and no amount of correct counting by the model can rescue a statistic whose control side is capped below 1.0 by transcription noise alone. Critically, every artefact identified here (inflectional endings, short-word letter-shedding at long chunk lengths) depresses the *control* side only – nothing here inflates the repeated side's score – so the measured +9.3-point Spanish gap is a lower bound, not an estimate. That is simultaneously the least favourable reading for a replication and the most favourable one for "the effect is English-specific," which is exactly why the arm cannot be reported as either.

27.5 The generation ceiling

Zero of the 342 Spanish generations hit XTTS-v2's 602-mel-token cap in either family (`crosslingual_es.json`: `word_rep` rate 0.0, n=180; `control_word` rate 0.0, n=162). The ceiling was close, though: the longest Spanish item reaches 590 of 602 mel tokens, a `word_rep` item at k=32 (template `et1`) – one rung from censoring. Restricted to the same two families, English XTTS-v2's own worst case across the ladder is 581 of 602, reached at k=24 (its own k=32 worst case is lower, at 577). So k=32 is the effective Spanish ceiling for this arm: not because anything was truncated, but because the margin below the cap is thinner in Spanish than in English at the top of the ladder, and extending the Spanish ladder further would risk measuring the harness rather than the model.

27.6 Traps, which generalise beyond this arm

`AutoProcessor` silently reintroduces a language model. Community CTC checkpoints often ship a KenLM beside the acoustic model, and `AutoProcessor` resolves such a repository to `Wav2Vec2ProcessorWithLM`, whose `batch_decode` is beam search under an n-gram prior – precisely the dynamic this paper's judge exists to avoid, since it re-introduces exactly the kind of language-model-conditioned decoding bias the CTC judge was chosen to escape. It fails *open*: transcripts get better, not worse, so nothing about the output looks broken. It was caught only because `pyctcdecode` happened to be missing from the environment, turning silent contamination into an `ImportError`. `asr_ctc.py` now takes `--processor wav2vec2` explicitly and asserts the resolved class.

CTC chunk length is part of the instrument, and it is judge-specific. The English judge (LibriSpeech-trained) is validated at 25-second chunks. The Spanish XLSR-53 judge, fine-tuned on Common Voice clips mostly under 10 seconds, drops to 0.75 periodic accuracy at 25-second chunks and recovers to 1.00 at every k once chunked to 5 seconds – not by collapsing repetitions the way an autoregressive judge does, but by shedding letters from short words. That failure mode mimics the paper's central bias without being an instance of it, and the only way to tell the difference is to validate chunking per judge, on ground truth, rather than assuming one instrument's working configuration transfers to another.

A pinned revision is not necessarily where the weights came from. `main` for the Spanish judge carries no `safetensors` file, so `from_pretrained` fetched `model.safetensors` from a separate auto-conversion commit and cached it under a second snapshot directory. It was verified bit-identical to `main`'s `pytorch_model.bin` across all 424 tensors before pinning `96d7e9b4e4a78af515a3c6d3cee7c0826045d276` as the revision reported above. Reading a hash out of the local cache without first checking the snapshot count would have pinned the wrong provenance silently.

28 The period ladder: periodicity, or verbatim identity?

Every reviewer in one round made the same objection: the design never varied periodicity on its own. The published repeated arm is one word repeated k times – period 1 *and* verbatim token identity, confounded from the start – and the only controls on record were period-8 (cycled) or aperiodic (never-cycled). This section is the missing ladder: holding the carrier sentence, the word count and the requested count k fixed and varying only the period $p \in \{1, 2, 4, 8, k\}$, plus a second, independent arm that leaves the repetition verbatim and tests the companion account that survived every mechanistic probe in Sections 8–26. Code: `analysis/period_ladder.py` and `analysis/disambiguation.py`; stimuli: `data/stimuli/stimuli_period.jsonl` (`make_stimuli_period.py`) and `data/stimuli/stimuli_disambig.jsonl` (`make_stimuli_disambig.py`); results: `data/results/period_ladder.json` and `data/results/disambiguation.json`. Both analysis scripts carry a pre-committed docstring, written and thresholds fixed before any of this audio existed; every verdict below is read off those thresholds, not chosen after the fact.

28.1 Design

At a fixed carrier sentence, a fixed word count and a fixed requested count k, only the period of the repeating block changes:

$p = 1$: $A\,A\,A\,A\,\ldots$	(the published repeated arm)
$p = 2$: $A\,B\,A\,B\,\ldots$	no token adjacent to itself
$p = 4$: $A\,B\,C\,D\,A\,B\,C\,D\,\ldots$	
$p = 8$: $A\,B\,C\,D\,E\,F\,G\,H\,A\,B\,\ldots$	(the published cycled control)
$p = k$: k distinct words	(the published never-cycled control)

Five carriers (`t1`, `t3`–`t6`; `t2` is the template the CTC judge never transcribes the fillers of and is excluded from the whole panel, S10) × three requested counts $k \in \{16, 24, 32\}$ are rendered at every period. Vocabulary is balanced by rotation: $p \in \{2, 4, 8\}$ draw from one shared pool of eight words, so that pooled over rotations (4 at p=2, 2 at p=4, 1 at p=8) every arm uses the same eight words the same number of times and cannot differ lexically; $p = 1$ (the template's own target word) and $p = k$ (the 146-word extension pool, tail delivery 0.79–0.85) are inherited, reported confounds rather than controlled ones, so the vocabulary-matched comparison is $p \in \{2, 4, 8\}$ and $p = 1, k$ are attached with that caveat. Three seeds per item, three checkpoints (Llasa-1B, Qwen3-TTS-0.6B, XTTS-v2), scored under `population.panel()`'s standard exclusions. After exclusions, $n = 1134$ (`kept` in the JSON; dropped: 9 degenerate, 73 token-budget cap hits, 0 ablations, out of 1395 candidate rows).

The companion arm (`disambiguation.py`) holds the period at 1 and the count and word exact-match target fixed, and instead edits the *boundary* between successive copies: `bare` (*very very very…*, the published arm), `comma` (*very, very, very,…*), `stop` (*Very. Very. Very.…*) and `and` (*very and very and very…*). `comma` and `stop` hold the word count exactly fixed – punctuation attaches to a word, `text.split()` is unchanged – so they are the length-matched cells; `and` inserts $k - 1$ extra words and is reported beside them, never averaged in. This arm ran on two of the panel's three checkpoints: the behavioural CSV has no `pd_*` disambiguator rows for Llasa-1B at all (none were generated), so every number in §28.7 below is over Qwen3-TTS-0.6B and XTTS-v2 only, $n = 353$ after exclusions.

28.2 The full ladder

Exact rate (= k requested units delivered, in order, the paper's own statistic) by period and checkpoint, under the headline ordered-scan rule (`exact_by_period_scan`):

checkpoint	p=1	p=2	p=4	p=8	p=k	n $(1, 2, 4, 8, k)$
Llasa-1B	3.6	54.6	83.3	92.5	75.0	28, 130, 84, 40, 44
Qwen3-TTS-0.6B	6.8	41.3	73.3	100.0	86.7	44, 179, 90, 45, 45
XTTS-v2	11.1	51.7	75.6	82.2	57.8	45, 180, 90, 45, 45
MEAN	7.2	49.2	77.4	91.6	73.2	

Every one of the three checkpoints is individually monotone increasing across $p = 1, 2, 4, 8$ (3.6<54.6<83.3<92.5; 6.8<41.3<73.3<100.0; 11.1<51.7<75.6<82.2) – the deficit is graded, not a step function, at every checkpoint separately as well as on the pooled mean. The $p = k$ arm sits below $p = 8$ at all three checkpoints, as its inherited vocabulary caveat predicts (a 146-word tail delivering 0.79–0.85, against the balanced eight-word pool at $p = 8$), and is not part of the monotonicity claim above.

28.3 Five scoring rules: magnitude disagrees, ordering does not

`period_ladder.json` computes the deficit $D(p) = E(8) - E(p)$ under five rules and calls a verdict on each from the pre-committed thresholds ($D(2)/D(1) \geq 0.50 \rightarrow$ PERIODICITY, $\leq 0.25 \rightarrow$ VERBATIM TOKEN IDENTITY, between $\rightarrow$ GRADED), pooling the $D(2)/D(1)$ ratio over checkpoints:

rule	$D(1)$	$D(2)$	$D(4)$	$D(2)/D(1)$	verdict
ordered scan (headline)	84.4	42.4	14.2	0.502	PERIODICITY
unbounded recount	82.9	60.9	20.1	0.734	PERIODICITY
strict conjunction	82.9	61.2	21.2	0.738	PERIODICITY
cap hits kept (censored bound)	81.9	43.5	13.5	0.531	PERIODICITY
by family	84.4	42.4	14.2	0.502	PERIODICITY

$D(2)$ ranges 42.4–61.2 points across the five rules – the magnitude of the recovered effect is genuinely rule-dependent – but all five land on PERIODICITY. Two caveats a reader should have that the range alone does not convey. First, "by family" is not an independent computation: in this panel every architecture family contributes exactly one checkpoint (Llasa-1B→Llasa, Qwen3-TTS-0.6B→Qwen3-TTS, XTTS-v2→XTTS), so its numbers reproduce the ordered-scan row exactly rather than triangulating it; the row is included because the JSON stores it as a distinct verdict, but it should not be read as a fifth independent method – there are four. Second, the headline scan rule's pooled ratio, 0.502, clears the 0.50 PERIODICITY threshold by 0.002: per checkpoint the scan-rule ratio is 0.426 (Llasa-1B), 0.630 (Qwen3-TTS-0.6B) and 0.430 (XTTS-v2) – two of three checkpoints individually fall *below* 0.50 under this rule, and the pooled call is carried by Qwen3-TTS-0.6B. Under both recount-based rules (unbounded, strict) every checkpoint individually clears 0.50 comfortably (0.65–0.81), so the qualitative call is not fragile, but the margin on the rule the main text quotes as its headline is thin and should be read as such rather than as 0.502 implies at a glance.

28.4 A caution on the bootstrap interval

`period_ladder.json`'s `bootstrap` block is a cluster bootstrap (4000 replicates, resampling (template, k) cells) computed on the *strict conjunction* rule (`exact_both`), not the ordered scan:

quantity	point (strict conjunction)	95% CI
$D(1)$	82.9 pts	[75.0, 90.2] pts
$D(2)$	61.2 pts	[48.3, 72.9] pts
$D(2)/D(1)$	0.738	[0.633, 0.824]

The point estimate is inside its own interval only when read this way. The ordered-scan headline reported in the main text and above, $D(2) = 42.4$ points, is *not* the quantity this interval was built on, and 42.4 falls outside $[48.3, 72.9]$ – below its lower bound. Quoting the bootstrap interval beside the scan-rule point estimate would therefore make the headline number look like it misses its own confidence interval, when in fact the two are answers to differently-defined statistics computed on the same data. This is not a contradiction in the result; it is a labelling hazard in how the JSON is organised, and the read requested for this section – do not pair `bootstrap` with the ordered-scan $D(2) = 42.4$ – is correct and is exactly what the main text does instead by quoting the across-rule range (42.4–61.2) rather than this interval (`analysis/make_numbers.py`, the `PerDTwoLo`/`PerDTwoHi` block, records the same reasoning in a comment predating this section).

28.5 $p = 2$ by rotation

A single-rotation design would have let the decisive cell's answer depend on which two words happened to be picked. Pooled over the four rotations of the eight-word pool, the spread across rotations is the size of that confound:

checkpoint	rot. 0	rot. 1	rot. 2	rot. 3	range
Llasa-1B	69.7	55.6	42.9	51.4	26.8
Qwen3-TTS-0.6B	24.4	48.9	38.6	53.3	28.9
XTTS-v2	57.8	55.6	53.3	40.0	17.8

The within-checkpoint range (17.8–28.9 points) is comparable to or larger than the pooled $D(2)$ itself (42.4 points, headline rule): a $p = 2$ arm built on any single rotation could have read anywhere from roughly 25% to 70% exact depending on which pair of words it happened to use. Pooling over all four rotations, as the design does, is what makes the $p = 2$ cell interpretable at all.

28.6 Replication of the pre-existing arms

The $p = 1$, $p = 8$ and $p = k$ rungs are character-identical re-renders of the published `wr_*`, `ct_*` and `ap_*` items on the same seeds – a check that today's population is the same population, not a new pipeline:

checkpoint	arm	new	published	Δ	n_{new}	n_{pub}
Llasa-1B	p=1	3.6	6.5	−2.9	28	31
Llasa-1B	p=8	92.5	89.7	+2.8	40	39
Llasa-1B	p=k	75.0	73.3	+1.7	44	45
Qwen3-TTS-0.6B	p=1	6.8	6.8	0.0	44	44
Qwen3-TTS-0.6B	p=8	100.0	100.0	0.0	45	45
Qwen3-TTS-0.6B	p=k	86.7	86.7	0.0	45	45
XTTS-v2	p=1	11.1	11.1	0.0	45	45
XTTS-v2	p=8	82.2	82.2	0.0	45	45
XTTS-v2	p=k	57.8	57.8	0.0	45	45

Six of the nine rows reproduce the published value to the decimal stored in the JSON, exactly 0.0. The other three – all Llasa-1B, the smallest-n checkpoint here (n = 28–44) – do not: −2.9, +2.8 and +1.7 points, the worst being p=1 at $n_{\text{new}} = 28$ against $n_{\text{pub}} = 31$. `period_ladder.py`'s own docstring for this check does not claim exact reproduction ("not expected to be identical – generation is sampled"); it only flags a *large* disagreement as evidence of a changed pipeline, and 2.9 points on $n \approx 30$ is well inside sampling noise for a Bernoulli rate at that n. The replication therefore supports what it is meant to support – the ladder's rungs are commensurable with the published arms – but "deltas of 0.000" describes two of the three checkpoints, not all three, and should not be quoted as a blanket property of this check.

28.7 The disambiguation arm

The mechanistic account this arm was built to test is the paper's own, not a straw alternative: after contraction, attention dilution, a stop-head/duration account and a causal rank-1 patch on the count coordinate were each tested to a conclusion (Sections 8–26), the reading that survived was that

the decoder cannot individuate identical spans – the count is represented and stays decodable, but the generation policy runs on local context, and token-identical neighbours give it nothing to advance against. `make_stimuli_disambig.py`'s docstring states this account and the falsification thresholds before any disambiguator audio existed (`R`$\geq 0.50 \rightarrow$ SUPPORTED, $\leq 0.15 \rightarrow$ FALSIFIED). The experiment was run to test that account, not to defend it.

Exact rate by variant (period 1 throughout, $k \in \{16, 24, 32\}$; Qwen3-TTS-0.6B and XTTS-v2 only, per the scope note above):

checkpoint	bare	comma	stop	and
Qwen3-TTS-0.6B	6.8	6.8	9.8	15.6
XTTS-v2	11.1	8.9	13.6	11.1

Recovered fraction $R = (E(\text{dis}) - E(\text{bare}))/(E(8) - E(\text{bare}))$, against each checkpoint's own period-ladder $E(8)$ ceiling (100.0% for Qwen3-TTS-0.6B, 82.2% for XTTS-v2):

checkpoint	R_{comma}	R_{stop}	R_{and}
Qwen3-TTS-0.6B	+0.000	+0.031	+0.094
XTTS-v2	−0.031	+0.036	+0.000

Mean R over the length-matched cells (`comma`, `stop`): $R_{\text{matched}} = 0.009$, i.e. 0.9% of the deficit recovered (`R_matched` in the JSON; the main text's `\DisR` macro, 0.9, reads the same field). $R_{\text{and}} = 0.047$ (4.7%), reported beside it and never averaged in, since `and` is not length-matched. Both are far under the 0.15 falsification bar. **Verdict: INDIVIDUATION FALSIFIED** (`disambiguation.json`, `verdict` field, verbatim). The ordering prediction the account also made – `stop`$\geq$`comma` – holds (`ordering_as_predicted: true`), but a correctly-ordered near-zero recovery is not the confirmation the account needed; per its own pre-committed rule, no mechanism section is written on this result.

The confound the design handles: `comma` and `stop` add punctuation, which changes character count and spoken duration without changing word count, so the scoring target stays the repeated content word and its required count throughout:

variant	words	chars	duration (s)
bare	31.8	170.6	11.65
comma	31.8	194.3	13.11
stop	31.6	191.6	13.42
and	54.8	263.0	16.23

`bare`, `comma` and `stop` hold word count fixed to within 0.2 words while duration shifts by 1.5–1.8s from inserted pauses – exactly the length-matched, duration-varying design the account requires. `and` nearly doubles the word count ($k-1$ extra tokens) and is the one variant a text front-end cannot silently normalise away; its small positive R is reported as conservative rather than confirmatory, since added length is independently known to hurt counting.

28.8 Hygiene and audits

Front-end check (`disambiguation.json`): 0.0% of `comma`, `stop` and `and` waveforms are byte-identical to their `bare` twin, at $n = 88$, 84 and 89 compared cells respectively. No text front-end

silently stripped the disambiguator, so no cell is excluded as a pipeline null (`disambiguation.json` carries no `identical_cells` key, meaning the dead-cell set was empty) and all 353 scored rows are model measurements.

Judge audit (`disambiguation.json`, target-word delivery – does the counted word come back in the transcript at all, independent of count): `bare` 100.0% (n=89), `comma` 100.0% (n=89), `stop` 100.0% (n=85), `and` 93.3% (n=90). Spread across arms is 6.7 points (`judge_delivery_spread`), under the script's own 10-point warning threshold – the recovered-fraction numbers above are not an artefact of the judge reading some arms' target word less often than others.

Per-arm hygiene (`period_ladder.json`, cap hits excluded from every table above):

arm	n_{kept}	cap%	words	chars	dur. (s)	degen.%
p=1	117	11.4	31.8	170.3	13.79	0.0
p=2	489	8.8	31.6	187.8	16.10	0.0
p=4	264	2.2	31.7	188.6	16.08	0.0
p=8	130	2.3	31.8	188.8	15.55	0.0
p=k	134	0.7	31.7	214.9	16.93	0.0

Cap rate falls steadily from 11.4% at p=1 to 0.7% at p=k, matching the pre-committed hygiene reasoning: the $p = 1$ arm loops and hits the generation-length budget far more often than the higher-period arms, so excluding cap hits (as every table above does) removes more of $p = 1$'s worst items than any other arm's – which works against, not for, a finding that the deficit is largest at $p = 1$. Word count and duration are flat across $p \in \{1, 2, 4, 8\}$ (within 0.2 words, within 0.6s of one another); character count and duration rise at $p = k$ (214.9 chars, 16.93s) because the 146-word extension pool skews toward longer words, consistent with its already-flagged vocabulary caveat. Degenerate-output rate is 0.0% at every arm: none of the differences above are outputs the judge failed to score at all.